

ללמוד Angular על ידי בניית Compass

מסלול מבוסס-פרויקט: מהשורה הראשונה שלך
ב-TypeScript ועד לאפליקציית Angular פרוסה

מהדורה ראשונה, 2026

הוכן לשימוש אישי.

הקדמה

אם מעולם לא בנית אפליקציית ווב, הספר הזה נכתב עבורך. הוא מניח שאתה יודע לקרוא אנגלית, להשתמש במחשב, ומוכן להקליד דברים בתוך עורך קוד ולראות מה קורה. הוא לא מניח שאתה יודע מהו פריימוורק, מהו TypeScript, או למה מישוהו יבחר מרצון להשתמש במשהו שנקרא "הזרקת תלויות". עד לפרק האחרון, כל המונחים האלה ירגישו לך ככלים שאתה שולח אליהם יד, לא כז'רגון שאתה סובל.

הספר נבנה סביב אפליקציה יחידה בשם **Compass** – מעקב אישי אחר משימות והרגלים. בפרק 4 תקליד `ng new compass` ותראה תבנית התחלתית ריקה מופיעה על המסך. כל פרק שיבוא אחר כך מוסיף יכולת אמיתית אחת ל-Compass. בסוף הספר, תהיה לך אפליקציה אמיתית: היא תאפשר לך להתחבר, לתכנן את השבוע שלך, לעקוב אחרי רצפים, לעבוד ללא חיבור לאינטרנט, לסנכרן כשתתחבר מחדש, ולפעול בדפדפן באינטרנט הפומבי.

ללמידה על ידי בנייה יש שני יתרונות שספרי "קריאה תחילה" בדרך כלל מוותרים עליהם. הראשון הוא שכל מושג נוחת איפשהו. כשאתה פוגש רעיון חדש – נניח, פיירט (interceptor) – אתה לא סתם קורא הגדרה; אתה כותב אחד, רואה איך הוא משנה את התנהגות האפליקציה שלך, ואז שובר אותו בכוונה כדי לראות מה

קורה. השני הוא שאתה מסיים עם משהו. Compass קטן מספיק כדי להחזיק בראש, ועשיר מספיק כדי להראות לחבר. הוא גם שלך: אין בספר חידה שהתשובה שלה חבויה בסוף.

איך הספר מאורגן

לספר יש חמישה חלקים ונספחים.

חלק ראשון – יסודות. ארבעה פרקים שמעבירים אותך מאפס לפרויקט Angular רץ. תפגוש את TypeScript, את התכונות המודרניות של JavaScript ש-Angular באמת משתמש בהן, ואת ה-CLI של Angular. לא תיגע כאן בקומפוננטה; המטרה היא שיהיה לך סדנה לפני שתרים פטיש.

חלק שני – קומפוננטות ותבניות. המכניקה המרכזית: מהי קומפוננטה, איך תבניות וקישורי נתונים עובדים, ואיך סיגנלים – הפרימיטיב הריאקטיבי המודרני של Angular – מאפשרים לך לתאר מצב משתנה ללא טקסים. בסוף חלק שני, Compass ייראה וירגיש כמו אפליקציה אמיתית.

חלק שלישי – שירותים והעולם שבחוץ. אפליקציות לא חיות בקומפוננטה אחת. חלק זה מכסה הזרקת תלויות, HTTP, RxJS וטפסים – ארבעת הדברים שאתה צריך כדי לדבר עם שרתים, לאסוף קלט ממשתמשים, ולשתף התנהגות בין חלקי האפליקציה בלי להעתיק ולהדביק.

חלק רביעי – בניית אפליקציה אמיתית. ניתוב, מצב שחורג מקומפוננטה אחת, דירקטיבות ופייפים מותאמים אישית, וביצועים.

כאן Compass גדל מ"עמוד אחד שעובד" ל"כמה עמודים שנשארים מהירים".

חלק חמישי – הפקה. בדיקות, רינדור בצד השרת (SSR), פריסה, והמערכת האקולוגית שאליה אתה מצטרף. כשתסיים את חלק חמישי, יהיה לך מופע של Compass באינטרנט, ותדע מה לחפש הלאה.

נספחים. מדריך התקנה למחשב, רשימת בדיקות ל-debugging, וסיור קצר באידיומים ישנים של Angular שתפגוש בקוד קיים. קרא אותם כשתצטרך; הם חומר עזר, לא רצף.

איך לקרוא את הספר

קרא אותו ליד המקלדת. לכל פרק יש קוד שכדאי להקליד – לא להדביק. ההקלדה עצמה עובדת: היא מאיטה אותך מספיק כדי לראות מה השתנה, ומאלצת אותך לשים לב לתחביר שהעתקה הייתה מחליקה מעליו. כשפרק מסתיים בסעיף תרגילים, עשה אותם לפני שתתחיל את הפרק הבא. הם לא רשות במובן שהפרק הבא מניח שעשית אותם; הם רשות במובן שאם תדלג עליהם, תאבד את הזיכרון השרירי שהספר מנסה לבנות.

הודעות מודגשות מופיעות לאורך הספר:

הערה. צד-סוגריים שימושי – הקשר מעמיק, מושג קשור, או פיסת היסטוריה.

שים לב. טעות נפוצה או התנהגות עדינה שתחסוך לך אחר צהריים בהמשך.

נסה זאת. ניסוי של דקה לשינוי קוד וראייה של התוצאה. עשה את אלה; הם זולים.

מה צריך

- מחשב נייד או שולחני עם macOS, Windows או Linux.
- כ-5GB של שטח דיסק פנוי (בעיקר עבור `node_modules`, שנפגוש בפרק 4).
- חיבור אינטרנט יציב בזמן העבודה על חלקים ראשון וחמישי; שאר הספר עובד ללא רשת.
- בערך 30 עד 50 שעות פרוסות על פני מספר ישיבות שנוח לך. יש קוראים שסיימו בשני סופי שבוע; יש שפרסו על פני שלושה חודשים. הספר לא אכפת לו.

אתה לא צריך ניסיון קודם ב-Vue, React, Angular או כל פריימוורק אחר. אתה לא צריך לדעת TypeScript; פרק 2 מלמד מספיק ממנו. אתה לא צריך לדעת איך שרתים עובדים; כשנצטרך שרת בחלק שלישי, נרים אותו יחד בכמה שורות של קוד.

מילה על גרסאות של Angular

Angular משחררת גרסה ראשית חדשה בערך כל שישה חודשים, אבל הרעיונות בספר הזה – קומפוננטות, סיגנלים, הזרקת תלויות, ניתוב, טפסים – יציבים ומרכזיים כבר שנים ויישארו כאלה. התחביר הספציפי מיועד ל-Angular 18 ואילך, שהיא הגרסה הראשונה שבה מבנה הזרימה החדש (`@if`, `@for`, `@switch`), סיגנלים, וקומפוננטות עצמאיות הם ברירת המחדל המומלצת. אם אתה קורא את זה כש-Angular נמצאת בגרסה 22 עד אז, רוב הקוד עדיין ירוץ בלי שינוי; מה שכן ישתנה יהיה שינוי שם, לא שינוי חשיבה.

הפוך את הדף. פרק 1 קצר – בלי קוד, רק מפה של איפה אנחנו ולאן אנחנו הולכים.

פרק 1. מהי Angular, ומתי כדאי לפנות אליה

בפרק הזה אין קוד. הוא מפה. לפני שתתקין דבר, יעזור לך לדעת מה אתה עומד להתקין, איזו בעיה זה פותר, ואיזה סוג של אפליקציה אתה עומד לבנות. קרא אותו פעם אחת, לאט. כל מה שיבוא מפרק 2 ואילך מניח שהמפה הזאת בראש שלך.

הצורה של אפליקציית ווב

דמיין, לרגע, אתר שאתה משתמש בו לעתים קרובות. אולי זה יומן, רשימת משימות, שירות סטרימינג של מוזיקה, או תיבת דואר. עכשיו דמיין מה הוא עושה כשאתה מתקשר איתו.

אתה לוחץ על הודעה והטקסט המלא שלה מופיע בחלונית מימין. אתה מקליד שאילתת חיפוש ותוצאות מתסננות תוך כדי הקלדה, בלי שהעמוד נטען מחדש. אתה מסמן משימה כבוצעה והיא עוברת לרשימת "הושלמו"; משתמש אחר, ביבשת אחרת, רואה את השינוי שנייה לאחר מכן. אתה גורר אירוע ביומן שלך והוא מצטרף לזמן חדש.

שום דבר מזה אינו עמוד. בשנים הראשונות של הווב, כל אחת מהאינטראקציות האלה הייתה מחייבת מסע הלוך ושוב לשרת: לחיצה, המתנה, צפייה בעמוד מתלבן, המתנה, ראיית עמוד חדש. אפליקציית ווב מודרנית עושה משהו שונה. היא נטענת פעם אחת, ואז שומרת

תמונה חיה של מה שהמשתמש מסתכל עליו, מעדכנת חלקים מהמסך כשמידע חדש מגיע וכשהמשתמש פועל. כשהיא צריכה מידע או רוצה לשמור משהו, היא מדברת עם השרת בשקט ברקע, בלי לשבש את התמונה.

לשמור על התמונה החיה מעודכנת קשה יותר ממה שזה נשמע. כל חלק על המסך תלוי בפיסת מידע כלשהי – מספר ההודעות שלא נקראו בסרגל הצד תלוי ברשימת ההודעות, מצב הכפתור "השב" תלוי בכך שהודעה נבחרה, הצבע של משימה תלוי אם היא באיחור. כשכל פיסת מידע משתנה, כל חלק של המסך שתלוי בה חייב להשתנות איתה. אם היית מנסה לכתוב את זה ידנית – כל כפתור שצריך להיכנס לפעולה ולצאת ממנה, כל מונה שצריך לגדול, כל רשימה שצריכה לרנדר מחדש – היית מבלה את רוב הזמן שלך בהדבקת דברים במקום לבנות משהו חדש. גם היית מכניס באגים בלי הפסקה: שוכח לעדכן מונה, שוכח להשבית כפתור, שוכח להסיר פריט ישן מרשימה.

פריימוורק הוא תוכנה שמטפלת ברוב ההדבקה הזאת בשבילך. Angular היא פריימוורק.

מה "פריימוורק" באמת אומר

אם כתבת תוכנית מחשב בעבר – אפילו קטנה – כתבת את הקוד שלך ובאיזשהו שלב ייבאת את הקוד של מישהו אחר כדי לעשות עבודה שלא רצית לעשות בעצמך. הקוד של אותו מישהו נקרא **ספרייה**. הקוד שלך אחראי; כשהוא צריך עזרה, הוא קורא לספרייה.

פריימוורק הוא ההפך. בפריימוורק, הקוד של הפריימוורק אחראי; הקוד שלך ממלא את החלקים שהפריימוורק לא יודע. משל שימושי: ספרייה היא פטיש, ואתה מחליט מתי להניף אותו. פריימוורק הוא סדנה, ואתה מחליט מה לבנות בתוך הסדנה – אבל הסדנה כבר בחרה את פריסת שולחן העבודה, את מיקום השקעים, ואת ארונות הכלים על הקיר. אתה מרוויח הרבה: הסדנה מאורגנת, כלים סטנדרטיים תמיד במקום שאתה מצפה שיהיו, ואנשים אחרים שעבדו באותו סוג של סדנה יכולים להתמצא בקוד שלך מייד. אתה גם מוותר על משהו: אם אתה רוצה לעקור את שולחן העבודה ולשים את ארונות הכלים על התקרה, הסדנה תילחם בך.

הבחירה הזאת היא הבחירה המרכזית שאתה עושה כשאתה בוחר פריימוורק. בתמורה לווייתור על גמישות מסוימת, אתה מקבל עקביות, קהילה, והרבה החלטות שאינך צריך יותר לעשות בעצמך. Angular נמצאת בצד ה"בעל דעה מוצקה" של הספקטרום. יש לה דעות על איך לארגן קוד, איך קומפוננטות מדברות זו עם זו, איך לטפל בתופעות לוואי, איך לכתוב בדיקות, ואיך לבנות להפקה. חלק מהחדשים בעניין מוצאים את זה מחניק בימים הראשונים ומשחרר לשארית הקריירה שלהם.

אז – מהי Angular?

Angular היא פריימוורק, שנעשה על ידי Google, לבניית אפליקציות שרצות בדפדפן ווב. במשפט אחד: **אתה מתאר את המסכים של האפליקציה שלך כעץ של חתיכות קטנות ועצמאיות שנקראות קומפוננטות, ו-Angular דואגת להפוך את התיאור הזה לפיקסלים,**

לשמור על הפיקסלים בסנכרון עם הנתונים שלך, ולשלוח את כל העסק לדפדפן.

בואו נפרק את המשפט הזה חלק אחר חלק, כי לכל שם עצם בו יהיה פרק משלו בהמשך.

עץ של קומפוננטות. לא תבנה את Compass כקובץ ענק אחד. תבנה אותו כעץ של חלקים קטנים: מעטפת ברמה העליונה, בתוכה סרגל צד ואזור מרכזי, בתוך האזור המרכזי כותרת ורשימה, בתוך הרשימה שורות רבות של משימות, וכן הלאה. כל חלק – כל קומפוננטה – אחראי לדבר אחד: איך הוא נראה, איך הוא מתנהג כשמתקשרים איתו, ואיך הוא חושף את עצמו לחלקים שסביבו. קומפוננטות בעץ מדברות עם ההורים והילדים שלהן דרך חיבורים מוגדרים היטב וחד-כיווניים. Angular קוראת לחיבורים האלה inputs ו-outputs; תפגוש אותם כראוי בפרק 8.

עצמאיות. ה-HTML, ה-CSS וההתנהגות של הקומפוננטה חיים ביחד. זהו היפוך מכוון של המודל הישן של פיתוח ווב, שבו לפרויקט הייתה תיקייה של קבצי HTML, תיקייה של קבצי CSS, ותיקייה של קבצי JavaScript, וכל פיצ'ר יחיד נגע בקובץ אחד בכל אחת. הטיעון של Angular, שנולד מעשור של צפייה באנשים בונים אפליקציות גדולות, הוא שפיצ'רים לעתים רחוקות מכבדים את הגבולות האלה – הסימון של כפתור, הסגנון שלו, ומטפל הלחיצה שלו הם פיצ'ר אחד – ושמועיל יותר לשמור אותם יחד ולהפריד בין פיצ'רים במקום.

Angular הופכת את התיאור שלך לפיקסלים. אתה מתאר איך המסך צריך להיראות בכל רגע נתון, בהינתן נתונים מסוימים. Angular מבינה מה בעצם לשים על המסך, ומה לשנות כשהנתונים משתנים. זה

עדין. בווּב הישן, כתבת קוד שאמר "כשהמשתמש לוחץ 'השב', הסר את כפתור ההשבה, הוסף כפתור שלה, שים textarea באמצע, הצב עליו את המיקוד". ב-Angular, אתה כותב: "כפתור ההשבה גלוי כשאין טיוטה; כפתור השליחה גלוי כשיש טיוטה; תוכן ה-textarea קשור לטקסט של הטיוטה". אתה מתאר את המצבים, לא את המעברים. Angular עושה את המעברים.

Angular שומרת על הפיקסלים בסנכרון עם הנתונים שלך. כי התיאור שלך אומר איזה חלקים של המסך תלויים באיזה חלקים של הנתונים, Angular יודעת איך לעדכן אותם כשהנתונים משתנים. Angular מודרנית עושה את זה עם מנגנון שנקרא סיגנלים, שתפגוש בפרק 7. סיגנל הוא פיסת נתונים שיודעת מי מסתכל עליה; כשהנתונים משתנים, כל מי שמסתכל עליהם מקבל התראה, ורק החלקים של המסך שקוראים אותם מרונדרים מחדש. זה נשמע כמו נחמדות ומתגלה כמהפכני: זה אומר שהאפליקציה שלך נשארת מהירה גם כשהיא גדלה, כי הפריימוורק אף פעם לא צריך לבדוק שוב את כל המסך אחרי כל שינוי.

Angular שולחת את כל העסק לדפדפן. אפליקציות ווב מודרניות עוברות שלב בנייה שהופך את קוד המקור שלך (הרבה קבצים בפורמט חיבור ידידותי) למספר קטן של קבצים, ממוקטמים בכבדות, מוכנים להישלח לדפדפן. Angular מגיעה עם כלי בנייה שמטפל בזה עבורך. בפרק 4 תריץ את `ng build` בפעם הראשונה ותראה מזה-בייטים של קוד המקור שלך דחוסים למאות בודדות של קילו-בייטים של פלט מוכן להפקה.

מה הופך את Angular ל Angular

Angular היא לא הפריימוורק היחיד מסוגה. אם שמעת את השם Angular, כנראה שמעת גם את השמות של כמה אחרים. הספר הזה לא ישווה בין כולם לעומק – אתה לא צריך את ההשוואה הזאת כדי ללמוד Angular יותר משאתה צריך להשוות בין ספרדית לאיטלקית כדי ללמוד ספרדית – אבל הוגן לתת שם לתכונות שמבדילות את Angular מעמיתיה.

בעלת דעה מוצקה וכוללת סוללות. Angular מגיעה עם ניתוב, טפסים, HTTP, אנימציה, בדיקות, בינאום, ורינדור בצד השרת כבר בקופסה, כולם מעוצבים לעבוד יחד. פריימוורקים שנוקטים בגישה ההפוכה – שולחים ליבה קטנה ונותנים לך לבחור ספריות לכל השאר – נותנים לך יותר גמישות ומאלצים אותך לקבל יותר החלטות. ההימור של Angular הוא שלרוב ההחלטות האלה יש תשובות נכונות רוב הזמן, והיא עושה אותן עבורך כדי שתוכל להשקיע את תשומת הלב שלך באפליקציה שלך במקום זה.

TypeScript תחילה. Angular כתובה ב-TypeScript, וכל אפליקציית Angular נכתבת ב-TypeScript. TypeScript היא סופר-סט של JavaScript: זה JavaScript עם טיפוסים. אם זה נשמע מפחיד, אל דאגה – פרק 2 מלמד מספיק TypeScript כדי להיות פרודוקטיבי, והכלים של Angular גורמים לטיפוסים לעזור לך יותר משהם מפריעים. התמורה אמיתית: כשאתה משנה שם של שדה, העורך שלך משנה את שמו בכל השימושים שלו על פני האפליקציה כולה; כשאתה שובר את החוזה של קומפוננטה עם ההורה שלה, העורך שלך אומר לך לפני שאתה מרענן

את הדפדפן. באפליקציה גדולה, זה ההבדל בין קוד שאתה יכול להחזיק בראש לבין קוד שאתה נלחם בו.

הזרקת תלויות. ל-Angular יש מערכת מובנית לקומפוננטה אחת לומר "אני צריכה משהו שעושה X" ולקבל דבר מתאים בלי שתצטרך לדעת איך הוא נבנה. זה נשמע מופשט עכשיו; זה ירגיש מובן מאליו אחרי פרק 9. זו הסיבה הבודדת הגדולה ביותר לכך שאפליקציות Angular נוטות להיות בדיקות, כי "תן לי גרסה מזויפת של הדבר שמדבר עם השרת" מתגלה כשורת קוד אחת.

סיגנלים כפרימיטיב הריאקטיבי. רוב ההיסטוריה של Angular, המודל הריאקטיבי שלה נבנה על ספרייה שנקראת RxJS, שמייצגת שינויים לאורך זמן בזרמים נצפים. RxJS חזקה ועדיין חשובה – תפגוש אותה בפרק 11 – אבל צוק הלמידה היה תלול והטקסים היו כבדים למצב יומיומי. גרסאות אחרונות של Angular הציגו סיגנלים, פרימיטיב קל יותר שמרגיש כמו משתנים רגילים אבל מעדכן את המסך אוטומטית כשהם משתנים. סיגנלים הם איך שתנהל את רוב המצב בספר הזה; RxJS הוא איך שתטפל בזרמים שנפרסים במהותם לאורך זמן, כמו תגובות HTTP וזרמי אירועים.

CLI מונחה קונבנציות. Angular מגיעה עם כלי שורת פקודה, ng, שמייצר תבניות של פרויקטים, יוצר קומפוננטות ושירותים חדשים, מריץ את האפליקציה שלך, מריץ את הבדיקות שלך, ובונה להפקה. זה הכלי שמסיר את רוב הטקסים של ההקמה שפעם היו שער לפיתוח ווב. בפרק 4 תשתמש בו כדי ליצור את Compass בפקודה אחת.

מתי Angular היא בחירה טובה – ומתי לא

Angular היא לא הבחירה הנכונה לכל פרויקט בווב, וחלק מהלמידה של הפריימוורק הוא לדעת איפה הוא זורח.

Angular היא במיטבה כשהאפליקציה היא כזאת שאתה מצפה לעבוד עליה לאורך זמן, עם צוות, עם מורכבות עסקית אמיתית: דשבורדים, פאנלים של מנהלים, כלים פנימיים, פורטלים ללקוחות, אפליקציות בנקאות ובריאות ומסחר אלקטרוני שיצמחו פיצ'רים ברבעון במשך שנים. בהקשרים האלה, המבנה בעל הדעה של Angular, יסוד ה-TypeScript, וסיפור הבדיקות משתלמים פי כמה. מהנדסים חדשים יכולים להצטרף ולהתמצא; refactor בטוחים כי הקומפיילר תופס שבירות של חוזים; ניתן להוסיף פיצ'רים בלי לשבור בטעות שלושה פיצ'רים אחרים.

Angular היא כלי כבד יותר ממה שאתה צריך לעמוד שיווקי בודד, אתר נחיתה, בלוג, או הדגמה אינטראקטיבית חד-פעמית. עבור אלה, HTML פשוט – או מחולל אתרים סטטי קל משקל – יתחיל אותך מהר יותר, ישלח פחות קוד למשתמשים שלך, ויהיה קל יותר להעביר הלאה. לבחור ב-Angular לאתר שיווקי של חמישה עמודים זה כמו לשכור מחסן כדי להכין כריך. אתה יכול לעשות את זה; זה יעבוד; לא כדאי לך.

Angular היא לא הבחירה הטובה ביותר למנועי משחקים, ויזואליזציות כבדות, או אפליקציות שהמשטח העיקרי שלהן הוא canvas או סצנת WebGL. Angular לא יודעת ולא אכפת לה מה נראה פיקסל של ה-canvas שלך; היית משתמש בה רק לממשק שסביב

ה-canvas, וגם אז פריימוורק קל יותר או קוד רגיל אולי היה מתאים יותר.

Compass, האפליקציה שאתה עומד לבנות, יושבת במרכז מקום הכיף של Angular: כמה מסכים, נתונים אמיתיים, שרת מתמיד, טפסים, אימות, ורשימת פיז'רים שתגדל לאורך הפרקים. הוא מייצג את סוג האפליקציה שצוותי Angular בונים כל יום.

מה Compass יהיה, בסוף

עד לפרק האחרון, Compass יהיה אפליקציה אמיתית, אם גם קטנה. הנה תצוגה מקדימה כדי שתדע לאן הדרך מובילה.

תפתח את Compass בדפדפן שלך ותראה מסך התחברות. תתחבר עם כתובת מייל; קישור אישור יגיע; תלחץ עליו, ו-Compass יזכור אותך במכשיר הזה. תגיע לדשבורד שמראה את המשימות של היום ואת ההרגלים שאתה עוקב אחריהם השבוע.

תוסיף משימה בלחיצה על כפתור, מילוי טופס, ולחיצה על שמור. היא תופיע מיד. אם האינטרנט שלך כבוי, היא עדיין תופיע מיד, מסומנת באיקון קטן שאומר "עדיין לא סונכרן", וכשהאינטרנט יחזור היא תישמר בשקט ברקע. תגרור משימות בין "היום", "השבוע הזה", ו"מאוחר יותר". תסמן הרגלים כבוצעו בלחיצה; מונה רצפים ליד כל הרגל יתעדכן.

תנווט לעמוד סטטיסטיקות ותראה תרשים של השלמות ההרגלים שלך על פני שלושים הימים האחרונים. תלחץ על הרגל כדי לראות את

ההיסטוריה שלו. תבקר בעמוד הגדרות שבו תוכל לשנות את שם התצוגה שלך ולבחור ערכת נושא.

כל אחד מהפיצ'רים האלה הוא מקום בספר. מסך ההתחברות ואישור המייל חיים בפרקי הניתוב והטפסים. ההתנהגות של offline-first חיה בפרקי ה-HTTP והמצב. התרשים חי בפרק הקומפוננטות והתבניות, ושוב בפרק הביצועים כשנעשה אותו מהיר מספיק כדי לא להיות משנה בטלפון. הבדיקות שתכתוב לכל זה חיות בחלק חמישי.

תסיים את הספר עם אפליקציה שקטנה מספיק כדי להחזיק בראש ועשירה מספיק כדי להצדיק כל רעיון ש-Angular מציעה.

מה בא הלאה

שלושת הפרקים הבאים מכינים אותך להקליד `ng new compass`. הם יוצאי דופן עבור ספר "בונים אפליקציה" כי הם מכילים כמעט אפס Angular. זה במכוון. Angular יושבת על גבי TypeScript, TypeScript יושבת על גבי JavaScript מודרני, ושניהם יושבים על גבי סט של כלים – מנהל חבילות, סביבת ריצה, עורך. דילוג עליהם, כמו שהרבה ספרים עושים, מייצר קוראים שיכולים להעתיק דוגמאות של Angular אבל שקופאים בפעם הראשונה שהודעת שגיאה מזכירה מונח שהספר מעולם לא הגדיר.

פרק 2 מכסה את TypeScript. לא את כולו – TypeScript הוא שפה גדולה עם סט טיפוסים אפשריים גדול עוד יותר – רק את החלקים שבאמת תפגוש כשתכתוב Angular. פרק 3 מכסה את התכונות של JavaScript המודרני ש-Angular מסתמכת עליהן: `destructuring`, `spread`,

פונקציות חץ, מודולים, promises ו- `await / async`. פרק 4 יעביר אותך דרך התקנת Node והתקנת Angular CLI, הרצת `ng new` `compass`, וסיור איטי על כל קובץ שה-CLI מייצר כדי שכלום בפרויקט החדש שלך לא ייראה מסתורי.

בסוף חלק ראשון, תהיה לך אפליקציית Angular רצה על המסך שלך ותבין כל חלק ממה שיצר אותה. אז, בחלק שני, נתחיל לבנות.

תרגילים

1. פתח אפליקציית ווב שאתה משתמש בה לעתים קרובות – לקוח מייל, שירות מוזיקה, כלי לניהול פרויקטים, כל דבר עם ממשק עשיר. שים לב מה קורה כשאתה לוחץ מסביב. נסה לזהות שלוש אינטראקציות שהיו דורשות טעינה מלאה של עמוד באתר בסגנון ישן ובמקום זה מעדכנות רק פיסה של המסך. רשום אותן.

2. במילים שלך, בשלושה או ארבעה משפטים, תאר את ההבדל בין ספרייה לפריימוורק למישהו שמעולם לא כתב קוד. אל תשתמש במילים ספרייה או פריימוורק בתשובה שלך. אם אתה יכול להסביר את החליפין בלעדיהן, אתה מבין אותם.

3. חשוב על רעיון אפליקציה שאולי תרצה לבנות יום אחד. הוא לא חייב להיות מקורי או טוב; קופסת מתכונים אישית, יומן תרגול מוזיקה, או תזכורת ליום הולדת זה בסדר. שאל את עצמך: האם האפליקציה הזאת נמצאת במקום הכיף של Angular, או שמשוהו קל יותר ישרת אותה יותר טוב? רשום

את התשובה שלך ואת ההיגיון שלך. בעוד כמה חודשים,
חזור וראה אם אתה עדיין מסכים.

פרק 2. TypeScript מספיק ל-Angular

הפרק הזה הוא ידע עבודה של TypeScript, שנכתב לאנשים שלא השתמשו בה קודם ולא רוצים להפוך לחוקרי TypeScript במשרה מלאה. בסוף תוכל לקרוא כל ביטוי TypeScript שהספר משתמש בו, ולכתוב משלך בלי לנחש. לא תדע – ואל תשאף לדעת – כל מה ש-TypeScript יכולה לעשות. מערכת הטיפוסים שלה עמוקה מספיק כדי שיהיה לה מדף ספרים משלה; אנחנו לא מבקרים במדף הזה היום.

הדוגמאות בפרק הזה מונעות על ידי Compass. בכל פעם שנציג תכונה חדשה של TypeScript, נראה אותה עם צורה שתהיה לך אחר כך באפליקציה שלך: משימה, הרגל, משתמש, אירוע השלמה. כך, עד שנתחיל לכתוב קומפוננטות ב-Angular בפרק 5, הטיפוסים כבר ירגישו מוכרים.

למה טיפוסים בכלל

אם כתבת סקריפט קטן בשפה כמו Python או JavaScript רגיל, כנראה כתבת משהו כזה:

```
function totalMinutes(sessions) {
  let sum = 0;
  for (const s of sessions) sum += s.duration;
  return sum;
}
```

זה עובד. זה גם לא נותן לך שום עזרה. במקום אחר בתוכנית, קולגה מעביר לך מערך של מחרוזות בטעות, או אובייקט שמאיית את השדה `durationMinutes` במקום `duration`. הפונקציה עדיין רצה; היא מייצרת `NaN` או `"0[object Object]"` או משהו אחר סוריאליסטי, ועד שאתה שם לב, אתה צד אחר המקור של המוזרות בפלט.

TypeScript מוסיפה כמות קטנה של טקסים כדי למנוע את הציד הזה. אתה מוסיף הערה למה שפונקציה מצפה; הקומפיילר של TypeScript בודק כל מקום שהפונקציה נקראת בו; אם קורא מפר את החוזה, אתה מגלה מייד, לפני שהקוד רץ. בעורך, זה מתגלה כקו אדום גלי מתחת לשורה הבעייתית והודעה ספציפית על מה שגוי. בבנייה, זה עוצר את הבנייה.

אותה פונקציה ב-TypeScript:

```
interface Session {
  duration: number;
}

function totalMinutes(sessions: Session[]): number {
  let sum = 0;
  for (const s of sessions) sum += s.duration;
  return sum;
}
```

עכשיו הקומפיילר יודע: הפונקציה מקבלת מערך של דברים שיש להם שדה `duration` שהוא מספר, ומחזירה מספר. תעביר לה משהו אחר ו-TypeScript תתנגד. אין שום קסם בזה – הקומפיילר פשוט בודק ששמות וצורות מסכימים – אבל התמורה בקוד גדול יותר מקובץ אחד משמעותית. Angular בנויה להישען על התמורה הזאת.

הרצת TypeScript, בפסקה אחת

TypeScript היא לא משהו שדפדפנים מריצים ישירות. כלי שנקרא הקומפיילר של TypeScript (`tsc`) קורא קבצי `.ts`, בודק את הטיפוסים, ומייצר קבצי JavaScript רגילים שדפדפן (או Node.js) יכול להריץ. בפרויקטי Angular, אתה כמעט אף פעם לא מפעיל את `tsc` בעצמך; ה-CLI של Angular מריץ אותו כחלק מ-`ng serve` ו-`ng build`. לפרק הזה, אם אתה רוצה לנסות TypeScript בבידוד, השתמש ב-TypeScript Playground באתר typescriptlang.org/play – הוא נותן לך להקליד TypeScript משמאל ולראות את ה-JavaScript המהודר מימין, בלי התקנה. מפרק 4 והלאה, ה-CLI לוקח פיקוד.

הפרימיטיבים

הטיפוסים הבסיסיים של TypeScript הם `number`, `string`, `boolean`, `null` ו-`undefined`. יש אחרים, אבל אלה אלה שתקליד מאה פעמים ביום.

```
let title: string = "Buy milk";
let priority: number = 2;
let done: boolean = false;
let due: string | null = null; // עוד על "|" בעוד רגע
```

TypeScript חכמה בהסקת טיפוסים כשאתה מציב ערך בנקודת ההצהרה:

```
let title = "Buy milk"; // מוסק string
```

אתה לא צריך לכתוב `string` : כשהערך כבר אומר ל-TypeScript מה המשתנה מחזיק. בפועל, אתה מוסיף הערות טיפוסים על פרמטרים של פונקציות וטיפוסי החזרה, על צורות של אובייקטים, ועל דברים שאתה עומד להקצות אליהם אחר כך; אתה משאיר את כל השאר להסקה. קוד שמוסיף הערות לכל `let` טריוויאלי הוא קוד מלא רעש.

הערה. `null` ו-`undefined` שונים ב-TypeScript, כמו שהם ב-Angular. JavaScript. נוטה להעדיף `null` עבור "אנחנו יודעים שאין כאן ערך" ולשמור את `undefined` עבור "הערך מעולם לא הוצב". שדה שהוא אופציונלי הוא כזה שאולי הוא `undefined`; שדה שהוא `nullable` הוא כזה שקיים אבל אולי הוא `null`. זו הבחנה עדינה שתהיה חשובה בפרק 12 כשנדבר על טפסים.

מערכים וטאפלים

הטיפוס של מערך הוא `Type[]`, או במקביל `Array<Type>`:

```
const priorities: number[] = [1, 2, 3];
const tags: string[] = ["home", "work"];
```

מערכים ריקים צריכים הערה כי אין מה להסיק מהם:

```
const completed: string[] = []; // בסדר
const uh = [];                  // שאתה לא רוצה, any-מוסק כ
```

טאפלים הם מערכים באורך קבוע וטיפוסים לפי מיקום. תשתמש בהם לעתים רחוקות בקוד אפליקציה:

```
const point: [number, number] = [42, 7];
```

טיפוסי אובייקט עם interface

הלחם והחמאה של TypeScript באפליקציית Angular הוא תיאור הצורה

של אובייקטים. TypeScript מציעה שתי דרכים לעשות את זה – `interface` ו-`type` – ולצורות של אובייקטים הן כמעט חליפיות. השתמש ב-`interface` לצורות של אובייקטים; השתמש ב-`type` לאיחודים, לחיתוכים, ולדברים שהם לא סתם אובייקט.

הנה הטיפוס המרכזי של Compass, כפי שיופיע בפרק 5:

```
interface Task {
  id: string;
  title: string;
  done: boolean;
  createdAt: string; // ISO 8601 תאריך
  dueDate: string | null;
  tags: string[];
}
```

ערך שטוען שהוא Task חייב להכיל את כל שישה השדות האלה, עם הטיפוסים האלה. שדות נוספים הם שגיאה (ברוב ההקשרים). שדות חסרים הם שגיאה. טיפוסים שגויים הם שגיאה. הצורה הזאת הופכת לחוזה שכל חלק של האפליקציה יכול להסתמך עליו.

מאפיינים אופציונליים, readonly, וחתימות אינדקס

תפגוש שלוש הערות שמשנות את התנהגות השדה.

```
interface Habit {
  id: string;
  name: string;
  color?: string; // undefined עשוי להיות
  readonly createdAt: string; // לא ניתן להקצות מחדש אחרי בנייה
  metadata: { [key: string]: string }; // string→string מפה שרירותית
}
```

`color?: string` אומר שהשדה אופציונלי; ערך שמקיים את Habit לא חייב לכלול אותו, ואם הוא שם הוא עשוי להיות `string` או `undefined`. `readonly` מונע הקצאה מחדש דרך הטיפוס הזה – מועיל לשדות בסגנון `id` שאסור לשנות במקום. הצורה האחרונה, `{ [key: string]: string }`, נקראת חתימת אינדקס; היא

אומרת "כל מפתח מסוג string ממופה לערך מסוג string" ומועילה למילונים פתוחים.

איחודי טיפוסים וטיפוסים ליטרליים

טיפוס איחוד הוא ערך שיכול להיות אחד מכמה טיפוסים. כבר ראית אחד: `string | null`. איחודים הם "או" מוגבל: הערך הזה הוא או מחרוזת או הערך `null`.

איחודים של מחרוזות ליטרליות הם אחד הדפוסים הכי שימושיים של TypeScript:

```
type Status = "todo" | "doing" | "done" | "archived";

interface Task {
  id: string;
  title: string;
  status: Status;
}
```

עכשיו `status` לא יכול להיות מחרוזת שרירותית; הוא חייב להיות בדיוק אחד מארבעת אלה. נסה לכתוב `task.status = "in-progress"` ו-TypeScript תסרב. כך אתה אוכף סט סגור של אפשרויות — `enum` של עני, וברוב המקרים הבחירה הטובה יותר.

שים לב. ל-TypeScript יש מילת מפתח `enum`, והשניים הראשונות של Angular השתמשו בה בכבדות. הפרקטיקה המודרנית היא להעדיף איחודי מחרוזות ליטרליות. `enum` נהדק לאובייקטים JavaScript של זמן ריצה, שמגדילים את ה-bundle שלך; איחודי טיפוסים נהדקים לכולם (הם קיימים רק בזמן בדיקת הטיפוסים). פרק 5 משתמש באיחודים לכל אורכו.

צמצום טיפוסים

אם למשתנה יש טיפוס איחוד, TypeScript תסיק צמצום שלו בתוך ענף של קוד שהיא יכולה לומר בו מה הערך באמת:

```
function greeting(name: string | null): string {
  if (name === null) {
    return "Hello, stranger.";
  }
  // name יודעת ש string, לא string | null.
  return `Hello, ${name}.`;
}
```

שלושת דפוסי הצמצום הנפוצים ביותר:

- `typeof x === "string"` מצמצם את `x` ל-`string`.
- `x === null` או `x === undefined` מצמצמים דרך `if` ים.
- `"field" in x` מצמצם לתת-הקבוצה של איחוד שיש לה את השדה.

• `x instanceof Foo` מצמצם ל-`Foo`.

צמצום הוא אחת התכונות שגורמות ל-TypeScript להרגיש כאילו היא עוזרת במקום נלחמת. אם TypeScript אי פעם אומרת "אולי `undefined`" ואתה יודע שלא, התשובה כמעט תמיד היא לבדוק עם `if`, לא לעקוף את הבדיקה.

פונקציות

טיפוסי פונקציה מוסיפים הערות לפרמטרים, ואופציונלית לטיפוסי החזרה:

```
function isOverdue(task: Task, now: Date): boolean {
  if (task.dueDate === null) return false;
  return new Date(task.dueDate) < now;
}
```

פונקציות חץ עובדות אותו הדבר:

```
const isOverdue = (task: Task, now: Date): boolean => {
  if (task.dueDate === null) return false;
  return new Date(task.dueDate) < now;
};
```

אם TypeScript יכולה להסיק את טיפוס ההחזרה מהגוף, אתה יכול להשמיט את ההערה — אבל זו פרקטיקה טובה להוסיף הערות לטיפוסי החזרה בפונקציות מיוצאות, כי זה מכריח אותך לחשוב על מה שפונקציה בעצם מבטיחה להחזיר.

פרמטרים של פונקציה יכולים להיות אופציונליים או שיהיו להם ברירות מחדל:

```
function label(task: Task, prefix?: string): string {
    return `${prefix ?? ""}${task.title}`;
}

function paginate<T>(items: T[], page: number, size = 20): T[] {
    return items.slice(page * size, (page + 1) * size);
}
```

הדוגמה השנייה משתמשת ב-גנרי, שנסתכל עליו הבא.

גנריים, ברוגע

גנרי הוא פרמטר טיפוס – מציין מקום ל"טיפוס כלשהו שנמלא באתר הקריאה". השימוש הקלאסי הוא פונקציה שעובדת על כל מערך בלי לאבד מידע טיפוס:

```
function first<T>(items: T[]): T | undefined {
    return items[0];
}

first([1, 2, 3]); // T הוא number; מחזיר number | undefined
first(["a", "b"]); // T הוא string; מחזיר string | undefined
```

השם **T** הוא קונבנציה. השתמש באותיות בודדות לפרמטרים מאוד כלליים (**V**, **K**, **U**, **T**), או בשמות תיאוריים כשבהירות עוזרת (**<Payload>**, **<Item>**).

תראה גנריים כל הזמן ב-Angular. `signal<Task[]>([])` יוצר סיגנל שמחזיק מערך של `Task`. `HttpClient.get<Task[]>('/api/tasks')` אומר לקומפיילר איזה צורה לצפות בחזרה מהשרת. בכל מקרה, הגנרי הוא התשובה ל"איזה טיפוס נמצא בתוך הדבר?".

אל תכתוב פונקציות גנריות משלך כשאתה לומד. פנה אליהן בפעם השלישית או הרביעית שאתה מעתיק ומדביק פונקציה ורק משנה את הטיפוסים שלה. עד אז, פונקציות עם טיפוסים רגילים הן כמעט תמיד הכלי הנכון.

הסקת טיפוסים היא חברה שלך

TypeScript אגרסיבית בהסקת טיפוסים, וכדאי לך לתת לה. שקול:

```
const tasks = tasksFromServer(); // מוסק כ-Task[]
const titles = tasks.map(t => t.title); // מוסק כ-string[]
const activeCount = tasks.filter(t => !t.done).length; // מוסק כ-number
```

לא היו צריכות הערות — TypeScript צעדה דרך כל השרשרת והבינה. הכלל הכללי: הוסף הערות לכניסות וליציאות של הפונקציות הציבוריות שלך, ותן ל-TypeScript להסיק את כל השאר.

any, unknown, ופתחי המילוט הכנים

לפעמים אתה באמת לא יודע את הטיפוס של ערך – פשוט ניתחת קצת JSON מהחוט, או אתה קורא לספרייה ישנה בלי טיפוסים. ל-TypeScript יש שני פתחי מילוט:

- **any** מכבה בדיקת טיפוסים לערך הזה. הכל מותר. זה מסוכן; אם אתה כותב **any** אתה חותם ומוותר על כל היתרון של שימוש ב-TypeScript עבור הפיסת מידע הזאת.
- **unknown** היא החלופה הבטוחה. ערך של טיפוס **unknown** יכול להיות כל דבר, אבל אתה לא יכול לעשות שום דבר איתו עד שתצמצם אותו לטיפוס ספציפי קודם.

```
function parseTask(raw: unknown): Task {
  if (typeof raw !== "object" || raw === null) {
    throw new Error("Not an object");
  }
  const r = raw as { id?: unknown; title?: unknown; /* ... */ };
  if (typeof r.id !== "string" || typeof r.title !== "string") {
    throw new Error("Missing required fields");
  }
  // בדיקות נוספות...
  return { id: r.id, title: r.title, done: false, createdAt: "", dueDate: null };
}
```

פנה ל-**unknown** בגבול של האפליקציה שלך – ניתוח JSON, קריאת **localStorage**, טיפול בספריות בלי טיפוסים. פנה ל-**any** כמעט אף פעם.

דקורטורים, בקצרה

Angular מסתמכת בכבדות על דקורטורים – פונקציות שנכתבות עם קידומת `@` שמצמידות מטא-דאטה למחלקה או לחברים שלה. תראה אותם כל הזמן:

```
@Component({
  selector: "app-task-row",
  template: `<div>{{ task.title }}</div>`,
})
export class TaskRow {
  @Input() task!: Task;
}
```

לעת עתה, דע רק זאת: `@Component`, `@Input`, `@Output`, וחבריהם הם איך שאתה אומר ל-Angular "המחלקה הזאת היא קומפוננטה" או "השדה הזה הוא input מהורה". פרק 5 יסביר מה כל אחד עושה. תפקידה של TypeScript הוא פשוט לאפשר את התחביר; המשמעות היא של Angular.

הערה. דקורטורים סטנדרטים רשמית ב-JavaScript מודרני

עכשיו, אבל Angular עדיין משתמשת בטעם הניסיוני הישן יותר כי הוא נושא יותר מטא-דאטה ש-Angular מסתמכת עליה. ה-CLI של Angular מגדיר את הפרויקט שלך כך שזה פשוט יעבוד; אתה אף פעם לא צריך לחשוב על זה. אם אי פעם תראה `experimentalDecorators` ב-`tsconfig.json`, זו הסיבה.

האופרטורים ! ו- ?

שני סימנים קטנים ייחודיים ל-TypeScript יופיעו לעתים קרובות:

- `x!` הוא טענת non-null. הוא אומר לקומפיילר "סמוך עליי, זה לא null או undefined כאן". השתמש בו במידה; כל ! הוא מקום שבו יכולה להופיע שגיאת זמן ריצה שמערכת הטיפוסים לא מזהירה עליה. בקוד קומפוננטה, `@Input()` `task!: Task` משתמש ב-! כדי להבטיח "Angular תקצה את זה לפני שאשתמש בו", וזאת הבטחה ש-Angular מקיימת – אבל רק אחרי בניית הקומפוננטה, לא במהלכה.
- `x?.y` הוא אופרטור שרשור אופציונלי (שהוא בעצם JavaScript רגיל). הוא מחזיר `undefined` בקצר-מסלול אם `x` הוא `null` או `undefined`, במקום לזרוק. `"default" ?? x?.y` משלב אותו עם אופרטור `nullish coalescing` שנפגוש בפרק 3.

מה דילגנו עליו במכוון

ל-TypeScript יש טיפוסים מותנים, טיפוסים ממופים, טיפוס `template literal`, `infer`, איחודים מובחנים במלוא הכלליות שלהם, ומאה כלים אחרים. תיתקל בחלקם, בעיקר בהודעות שגיאה, ותפתה לצלול לחר הארנב הזה. התנגד. התכונות בפרק הזה הן אלה שתשתמש בהן כל יום ב-Angular. השאר זמין כשתצטרך אותו.

מה בא הלאה

פרק 3 מכסה את התכונות של JavaScript המודרני ש-Angular מניחה שאתה יודע: מודולים, פונקציות חץ, spread, destructuring ו-rest, await / async, ושיטות המערך שתישען עליהן ללא הפסקה. אם כבר כתבת JavaScript עדכני, אתה יכול לסקור אותו במהירות; אם לא, קח את זה לאט – פרק 4 יניח כל רעיון מפרק 3.

תרגילים

1. כתוב `Habit` interface עם השדות האלה: `id` (string), `name` (string), `frequency` (אחד מ-`"daily"`, `"weekly"`, `"custom"`), `color` (string אופציונלי), `createdAt` (string מסוג `readonly`). אז כתוב פונקציה `describe(habit: Habit): string` שמחזירה משהו כמו `"Read (daily)"`.

2. בהינתן `type Result<T> = { ok: true; value: T }` | `{ ok: false; error: string }`, כתוב פונקציה `unwrap<T>(r: Result<T>): T` שמחזירה את הערך אם `ok` הוא `true` וזורקת אחרת. נסה לכתוב את זה עם `as`.

3. ב-Playground (typescriptlang.org/play), הדבק אובייקט בצורה `{ title: "test", priority: "high" }` ונסה להקצות אותו למשתנה בטיפוס `{ title: string; priority: number }`. קרא את הודעת השגיאה בקפידה. אחר כך תקן את האובייקט כך

שיעבור בדיקת טיפוסים. הבנת פורמט השגיאה הזה היא מיומנות שכדאי שתהיה.

פרק 3. JavaScript מודרני שבאמת תשתמש בו

Angular היא TypeScript מבחוץ ו-JavaScript מבפנים. כל מה שהקומפיילר מייצר, הדפדפן מריץ. אם קטע של תחביר JavaScript ייראה לא מוכר בדוגמת Angular בהמשך הספר, הסיכוי הוא שהיא אחת מהתכונות בפרק הזה. יש עוד עשרות שלא נכסה – שפת JavaScript המודרנית גדולה מאוד – אבל תת-הקבוצה כאן היא בערך מאה אחוז ממה שקוד יישום Angular באמת משתמש בו.

אם כתבת JavaScript עדכני קודם, אתה יכול לקרוא את הפרק הזה כרשימת בדיקה ולעבור במהירות מעל מה שאתה כבר יודע. אם לא, תכנן לנסות כל דוגמה בקונסולת הפיתוח של דפדפן (פתח את הדפדפן שלך, לחץ F12, לחץ על "Console") כדי שהתחביר יגיע לאצבעות שלך.

מודולים: `import` ו-`export`

כל קובץ בפרויקט JavaScript או TypeScript מודרני הוא מודול. מודול הוא קובץ שיכול לייצא דברים (פונקציות, מחלקות, קבועים, טיפוסים) לקבצים אחרים לשימוש, ולייצא דברים ממודולים אחרים. זה המנגנון שבאמצעותו האפליקציה שלך מפוצלת לקבצים בלי להם namespace גלובלי.

יש שני טעמים של ייצוא:

```
// tasks.ts

// ייצוא בשם: מייצא סמל תחת שמו שלו.
export interface Task {
  id: string;
  title: string;
}

export function isOverdue(task: Task): boolean { /* ... */ return false; }

// ייצוא ברירת מחדל: אחד לקובץ, מיובא בלי סוגריים מסולסלים.
export default class TaskService {
  // ...
}
```

ובמקביל, שני טעמים של ייבוא:

```
// app.ts

import TaskService from "./tasks"; // ברירת מחדל
import { Task, isOverdue } from "./tasks"; // בשם
import * as tasks from "./tasks"; // namespace הכול, עם
```

מדריך הסגנון של Angular, והספר הזה, מעדיפים ייצוא בשם כמעט תמיד. ייצוא ברירת מחדל חוסך זוג סוגריים אבל עולה בבהירות: השם של ייצוא ברירת מחדל באתר הייבוא הוא מה שהמייבא מקליד, כך ש-`import Foo from "./bar"` ו-`import Baz from "./bar"` מתייחסים לאותו דבר. ייצוא בשם שומר על השמות ישרים לרוחב הקוד.

נתיבי ייבוא שמתחילים ב-`./` או `../` הם יחסיים – הם מצביעים על קובץ אחר בפרויקט שלך. נתיבים שמתחילים בכל דבר אחר הם

מזהים חשופים ומתמייחסים לחבילות מותקנות (@angular/core , rxjs , וכן הלאה).

שים לב. אל תשים את הסיומת `.ts` בייבואים שלך (`from "./"` `tasks.ts`). כלי הבנייה של Angular מטפל בסיומת עבורך. בסביבות JavaScript אחרות – בעיקר Node.js במצב ESM – אתה כן צריך את הסיומת. Angular הוא המקרה הראשון, לא השני.

פונקציות חץ

פונקציית חץ היא דרך קצרה יותר לכתוב פונקציה, עם הבדל סמנטי חשוב אחד מפונקציה רגילה: אין לה `this` משלה.

```
// טופס ארוך
const isDone = function(task) { return task.done; };

// חץ, גוף בלוק
const isDone = (task) => { return task.done; };

// return חץ, גוף ביטוי – בלי סוגריים, בלי מילת מפתח
const isDone = task => task.done;

// מספר פרמטרים דורש סוגריים
const compare = (a, b) => a.priority - b.priority;
```

היעדר `this` משלה חשוב כשאתה מעביר פונקציית חץ כ-`callback`:

```

class TaskService {
  tasks: Task[] = [];

  loadTasks() {
    fetch("/api/tasks")
      .then(res => res.json())
      .then(data => {
        // `this` כאן עדיין מתייחס למופע של TaskService,
        // loadTasks-כי פונקציית החץ יורשת אותו מ
        this.tasks = data;
      });
  }
}

```

אם ה-callback הפנימי היה `function` רגיל, `this` בתוכו היה `undefined` (במצב strict) או האובייקט הגלובלי – מקור באגים שהציק ל-JavaScript עשור לפני שחצים הגיעו. בקוד Angular, כמעט תמיד תרצה חצים ל-callbacks.

Template literals

מחרוזות בגרשי קונטרס-אקצנט יכולות להכיל ביטויים אינטרפולציה ולהתפרש על פני מספר שורות:

```

const name = "Anna";
const greeting = `Hello, ${name}. Today is ${new Date().toLocaleString}`;

```

הן טובות באופן מוחלט מקישור מחרוזות לכל דבר ארוך יותר מטריויאלי. תבניות Angular עצמן הן template literals כשאתה כותב תבניות inline:

```
@Component({
  template: `
    <h1>Welcome, {{ user.name }}</h1>
    <p>You have {{ taskCount() }} tasks today.</p>
  `,
})
```

ה- `{{ }}` בפנים הוא תחביר התבנית של Angular, לא של JavaScript `.${{ }}`. הם חיים יחד כי הם אף פעם לא חופפים: הליטרל החיצוני הוא JavaScript, מוערך פעם אחת כשמחלקת הקומפוננטה מוגדרת; ה- `{{ }}` הוא של Angular, מוערך מחדש כשהנתונים משתנים.

Destructuring

Destructuring שולף שדות מאובייקט (או אלמנטים ממערך) לתוך משתנים בשמות בביטוי יחיד.

```
const task = { id: "t1", title: "Buy milk", done: false };

// במקום:
const id = task.id;
const title = task.title;

// אתה יכול לכתוב:
const { id, title } = task;
```

אתה יכול לתת שם חדש בדרך החוצה ולספק ברירות מחדל:

```
const { id: taskId, title, done = false } = task;
```

Destructuring עובד גם בפרמטרים של פונקציה:

```
function label({ title, done }: Task): string {  
  return done ? `[x] ${title}` : `[ ] ${title}`;  
}  
  
label({ id: "t1", title: "Buy milk", done: false, /* ... */ });
```

Destructuring של מערכים משתמש באותו רעיון עם סוגריים מרובעים:

```
const [first, second, ...rest] = [1, 2, 3, 4, 5];  
// first = 1, second = 2, rest = [3, 4, 5]
```

ה-`...rest` בשורה האחרונה הוא אופרטור `spread/rest`, שנסתכל עליו הבא.

Spread-1 rest

לאופרטור שלוש-הנקודות `...` יש שני שימושים קשורים.

`Spread` מרחיב את האלמנטים של מערך (או את המאפיינים של אובייקט) `inline`:

```
const priorities = [1, 2, 3];  
const withZero = [0, ...priorities];    // [0, 1, 2, 3]  
  
const base = { title: "Untitled", done: false };  
const withId = { ...base, id: "t1" };    // { title, done, id }
```

`Spread` של אובייקט הוא איך שאתה יוצר עותק רדוד של אובייקט, ואיך שאתה מייצר עותק ששונה בלי לשנות את המקור:

```
const task: Task = { /* ... */ };
const updated = { ...task, done: true };
// task.done עדיין מה שהיה; updated.done הוא true.
```

הדפוס הזה של "אל תשנה; ייצר אובייקט חדש" הוא הדפוס של קוד ריאקטיבי מודרני ב-Angular. כשנגיע לסיגנלים בפרק 7, תראה אותו כל הזמן.

Rest אוסף את האלמנטים הנותרים של מערך או את הפרמטרים הנותרים של פונקציה למשתנה יחיד:

```
function sum(first: number, ...others: number[]): number {
  return others.reduce((acc, n) => acc + n, first);
}

sum(1, 2, 3, 4); // 10
```

אותן שלוש נקודות, כיוון הפוך: spread מרחיב, rest אוסף.

פרמטרים ברירת מחדל

לפרמטרים יכולות להיות ברירות מחדל, וברירות המחדל יכולות להתייחס לפרמטרים קודמים:

```
function paginate(items: Task[], page: number, size: number = 20): Task[] {
  return items.slice(page * size, (page + 1) * size);
}

paginate(all, 0); // size = 20 משתמש ב-
paginate(all, 0, 50); // size = 50 משתמש ב-
```

פרמטרי ברירת מחדל נכנסים לפעולה כשהארגומנט הוא `undefined` – העברת `null` לא מפעילה את ברירת המחדל. שווה לזכור את זה בפעם הראשונה שזה מפתיע אותך.

שרשור אופציונלי ו-nullish coalescing

שני אופרטורים שהחליפו בשקט הרבה שרשראות `if` הגנתיות.

שרשור אופציונלי – `?.` – מחזיר `undefined` בקצר-מסלול אם הצד השמאלי הוא `null` או `undefined`, במקום לזרוק:

```
const city = user?.address?.city;
// שווה ל:
const city2 = user && user.address ? user.address.city : undefined;
```

Nullish coalescing – `??` – מחזיר את הצד הימני אם הצד השמאלי הוא `null` או `undefined`, אחרת את הצד השמאלי:

```
const display = user.nickname ?? user.name ?? "Anonymous";
```

ההבחנה החשובה מ-`||` (שאולי ראית משמש באופן דומה) היא ש-`??` נופל דרך רק על `null` ו-`undefined`, לא על ערכים אחרים. `10 || 0` נותן `10`; `10 ?? 0` נותן `0`. ב-`Compass`, `3 ?? task.priority` הוא "ברירת מחדל בטוחה של עדיפות ל-3 אם לא מוגדרת", בעוד `3 || task.priority` היה שוגה ומעדכן עדיפות מכוונת של `0`.

Promises

Promise הוא ערך שיהיה קיים בעתיד. כל פעולה שלוקחת זמן – בקשת רשת, קריאת קובץ, טיימר – מחזירה **Promise**. **Promise** נמצא תמיד באחד משלושה מצבים: פתוח (עדיין ממתין), ממומש (הפיק ערך), או נדחה (הפיק שגיאה).

אתה מצמיד **callbacks** עם **.then()** ו- **.catch()**:

```
fetch("/api/tasks")
  .then(res => res.json())
  .then(tasks => console.log(tasks))
  .catch(err => console.error("Something went wrong:", err));
```

כל **.then** מקבל את הערך שהשלב הקודם החזיר. אם שלב כלשהו זורק או מחזיר **Promise** שנדחה, השליטה קופצת ל- **.catch**. הקרוב ביותר.

סגנון ה"שרשרת של **thens**" הזה היה פעם הדרך הסטנדרטית לכתוב קוד אסינכרוני. הוא עובד, ומנגנון ה-HTTP הבסיסי של **Angular** עדיין משתמש ב-**Promises** במקומות, אבל הארגונומיה מסורבלת ברגע שהשרשרת גדלה. קוד מודרני משתמש ב- **await / async** במקום.

await / async

async מסמן פונקציה כמחזירה **Promise**, ומאפשר לך להשתמש ב- **await** בתוכה. **await** משהה את הפונקציה בביטוי הזה עד שה-**Promise** שממתין נפתר, ואז מתחדש עם הערך שלו.

```
async function loadTasks(): Promise<Task[]> {  
  const res = await fetch("/api/tasks");  
  const tasks: Task[] = await res.json();  
  return tasks;  
}
```

// בהמשך

```
const tasks = await loadTasks();
```

שגיאות ב-Promises שממתינים יוצאות כחריגות שנזרקות, כך שאתה יכול להשתמש ב- `catch / try` רגיל:

```
try {  
  const tasks = await loadTasks();  
} catch (err) {  
  console.error("Could not load tasks:", err);  
}
```

`await / async` היא דרך נחמדה יותר לכתוב את אותו החישוב ששרשרת של `.then()` ים מבטאת – אבל זה אותו החישוב. שום דבר לא רץ סינכרונית. `await` עדיין מוותר על שליטה לדפדפן בין שלבים. זה תחביר סוכר, וזה טעים.

הערה. Angular משתמשת לפעמים ב-Promises, לפעמים ב-Observables (טיפוס RxJS), ויותר ויותר בסיגנלים למצב ריאקטיבי. פרק 11 פורק את ההבדל בין Promises ל-Observables – לעת עתה, החשוב הוא שתבין Promises, כי כל השאר בעולם האסינכרוני בונה עליהם או מחליף אותם.

שיטות מערך שתשתמש בהן כל הזמן

שיטות המערך של JavaScript המודרני הן ארגז הכלים של קוד יישום יומיומי. תישען עליהן בכל יום.

```
const tasks: Task[] = /* ... */;

// map: הפוך כל אלמנט
const titles = tasks.map(t => t.title);

// filter: שמור את האלמנטים שתואמים
const active = tasks.filter(t => !t.done);

// find: undefined קבל את האלמנט הראשון שתואם, או
const buyMilk = tasks.find(t => t.title === "Buy milk");

// some / every: בדוק את המערך כמכלול
const anyOverdue = tasks.some(isOverdue);
const allDone = tasks.every(t => t.done);

// reduce: קפל את המערך לערך יחיד
const total = tasks.reduce((sum, t) => sum + (t.done ? 1 : 0), 0);

// includes: האם המערך מכיל את הערך?
const hasWork = tags.includes("work");

// sort: משנה! בדרך כלל תרצה להעתיק קודם
const byPriority = [...tasks].sort((a, b) => a.priority - b.priority);
```

שלוש אזהרות לגבי `sort` וחבריה:

1. `sort`, `reverse`, ו-`splice` משנים את המערך במקום.

אם אתה רוצה מערך חדש ממוין בלי להפריע למקורי, פזר

אותו קודם: `[...tasks].sort(...)`.

2. `sort` בלי פונקציית השוואה מכפה אלמנטים למחרוזות, שזה מקור באגים אגדי. מיונים מספריים דורשים פונקציית השוואה: `.sort((a, b) => a - b)`.
3. `reduce` חזקה ומופרזת בשימוש. אם אתה מוצא את עצמך כותב `reduce` שיכול להיות לולאת `for` עם שני משתנים, לולאת `for` בדרך כלל בהירה יותר.

לולאות `for`

יש לך שלוש דרכים עיקריות לעבור באיטרציה:

```
// for..of עובר על ערכים במערך או ב iterable
for (const task of tasks) {
  console.log(task.title);
}

// forEach בסדר לתופעות לוואי; לא ניתן לצאת ממנו
tasks.forEach(task => console.log(task.title));

// for קלאסי עדיין שימושי כשהאינדקס חשוב
for (let i = 0; i < tasks.length; i++) {
  console.log(`${i}: ${tasks[i].title}`);
}
```

`for..in` גם קיים אבל עובר על מפתחות של אובייקט, לא על אלמנטים של מערך. מאחר שלמערכים יש מפתחות מחרוזתיים ("0", "1", ...) `for..in` יעשה דברים מפתיעים על מערכים. העדף `for..of` למערכים ו-`Object.keys(obj)` / `Object.entries(obj)` לאובייקטים.

אובייקטים והעתקות רדודות

העתקת אובייקטים היא מקום שמתחילים נשרפים בו. הקצאה לא מעתיקה; היא יוצרת התייחסות חדשה לאותו אובייקט.

```
const original = { title: "Buy milk", done: false };
const alias = original;
alias.done = true;
console.log(original.done); // true – הפתעה
```

כדי להעתיק, השתמש ב-spread (או ב-Object.assign):

```
const copy = { ...original };
copy.done = true;
console.log(original.done); // false עדיין
```

זו העתקה רדודה: אובייקטים מקוננים עדיין משותפים. אם באובייקט שלך יש אובייקטים מקוננים ואתה צריך עותק עמוק, השתמש ב-`structuredClone(value)` (מובנה בדפדפנים מודרניים).

מחלקות, בקצרה

Angular היא class-forward: קומפוננטות, שירותים, דירקטיבות ופיפּים הן כולן מחלקות. אתה לא צריך ללמוד הכל על מחלקות היום – פרק 5 יעשה זאת – אבל התחביר לא מפתיע:

```
class Counter {  
  private count = 0;  
  
  increment(): void {  
    this.count += 1;  
  }  
  
  get value(): number {  
    return this.count;  
  }  
}  
  
const c = new Counter();  
c.increment();  
console.log(c.value); // 1
```

מילת המפתח `private` היא של TypeScript, לא של JavaScript – בזמן ריצה השדה נגיש, אבל הקומפיילר יסרב לתת לקוד חיצוני לגעת בו. ל-JavaScript המודרני יש גם שדות פרטיים אמיתיים `#`; קוד הבסיס של Angular משתמש ב-`private` של TypeScript לפי מוסכמה, אז גם אנחנו.

מה דילגנו עליו במכוון

Generators (`function*`), הטיפוס `Symbol`, `Proxy`, `Reflect`, `Map` ו-`Set` (תפגוש את אלה כשהם עוזרים), ביטויים רגולריים בפירוט, ורבות מהעדינויות של `this`. כולם שימושיים; אף אחד מהם לא נדרש כדי לבנות את `Compass`. כשתפגוש אחד מהם, חפש אותו ב-Mozilla Developer Network – MDN היא הפניה הקנונית ל-JavaScript.

מה בא הלאה

פרק 4 מכניס אותך לאפליקציית Angular רצה. תתקין את Node.js, תתקין את ה-CLI של Angular, תריץ `ng new compass`, ותעבור סיור איטי על כל קובץ שה-CLI מייצר. עד סוף פרק 4, יהיה לך מסך התחלה של Angular פתוח בדפדפן שלך ותבין מה כל חלק ממה שיצר אותו עושה.

תרגילים

1. בהינתן `const nums = [3, 1, 4, 1, 5, 9, 2, 6]` השתמש בשיטות מערך כדי לייצר (א) מערך של ריבועים, (ב) סכום הריבועים, (ג) המספר הגדול ביותר, ו-(ד) האם כל מספר קטן מ-10. כתוב כל אחד כביטוי יחיד.

2. כתוב פונקציה `updateTask(tasks: Task[], id: string, changes: Partial<Task>): Task[]` שמחזירה מערך חדש שבו למשימה עם ה-`id` התואם `changes` ממוזג לתוכה, וכל משימה אחרת נשארת ללא שינוי. השתמש ב-`map` וב-`spread`; אל תשנה את הקלט. (הטיפוס `Partial<Task>` פירושו "כל שדות של `Task`, אבל כל אחד אופציונלי").

3. המר את שרשרת ה-Promise הזאת ל-`await / async`:

```
function loadWithAuth(): Promise<Task[]> {  
  return fetch("/api/session")  
    .then(res => res.json())  
    .then(session => fetch("/api/tasks", { headers: { Authorization: sess  
    .then(res => res.json());  
}
```

אז חשוב מה קורה אם ה- `fetch` הראשון נכשל. איפה השגיאה עולה
על פני השטח בכל גרסה?

פרק 4. ה-CLI, האפליקציה הראשונה שלך, והאנטומיה של פרויקט Angular

הפרק הזה שונה משלושת הקודמים. הוא הפרק הראשון שכדאי לך לקרוא עם המחשב הנייד שלך פתוח. עד סוף הפרק, Compass יתקיים כפרויקט בדיסק שלך, שרת הפיתוח שלו ירוץ, ועמוד ההתחלה שלו יהיה פתוח בדפדפן שלך. ואז נעבור על כל קובץ שה-CLI של Angular ייצר, כך ששום דבר בפרויקט החדש שלך לא ייראה מסתורי בפעם הראשונה שתפתח אותו בעורך.

אם נתקעת בהתקנת Node.js, קפוף לנספח א' להערות ספציפיות לפלטפורמה. חזור כשיהיה לך `node --version` ו-`npm --version` עובדים במסוף שלך.

מה אנחנו מתקנים, ולמה

אתה צריך שלושה דברים כדי לעבוד על Angular:

1. Node.js — סביבת ריצה שמאפשרת ל-JavaScript לרוץ מחוץ לדפדפן. כלי הפיתוח של Angular (הקומפיילר, שרת הפיתוח, מריץ הבדיקות) הם תוכניות JavaScript; הן רצות על Node.js. תתקשר עם Node.js ישירות מעט מאוד, אבל הכל תלוי בו.

2. **npm** – מנהל החבילות של Node.js. הוא מגיע יחד עם Node.js. הוא מוריד ספריות – כולל את Angular עצמה – לתוך תיקייה שנקראת `node_modules` בתוך הפרויקט שלך.

3. **ה-CLI של Angular** – הפקודה `ng`. זה כלי שמייצר תבניות של פרויקטים חדשים, יוצר קבצים חדשים, מריץ את האפליקציה שלך, מריץ את הבדיקות שלך, ובונה להפקה. כל מה שתעשה עם Angular לשארית הספר עובר דרך `ng`.

התקנת Node.js

לך ל-`nodejs.org` והורד את גרסת ה-LTS (long-term support). בזמן כתיבת הספר זו Node.js 20; כשתקרא את זה זה כנראה יהיה Node.js 22 או מאוחר יותר. כל אחת מהן בסדר כל עוד היא 20 ומעלה; Angular מסרבת להתקין על גרסאות ישנות יותר.

עקוב אחרי המתקין לפלטפורמה שלך. ב-macOS וב-Windows, המתקין ממקם את `node` ו-`npm` ב-`PATH` שלך אוטומטית; ב-Linux, השתמש במנהל החבילות של ההפצה שלך או ב-tarball באתר Node.js (ראה נספח א').

פתח מסוף – אפליקציית Terminal ב-macOS, Windows Terminal, או PowerShell ב-Windows, אמולטור המסוף האהוב עליך ב-Linux – ואמת:

```
node --version # או גבוה יותר v20.x.x
npm --version  # או גבוה יותר 10.x.x
```

שניהם צריכים להדפיס מספר גרסה. אם אחד מהם אומר "command not found", ראה נספח א'.

התקנת ה-CLI של Angular

עם Node.js במקומו, התקן את ה-CLI גלובלית עם npm:

```
npm install --global @angular/cli
```

הדגל `--global` הופך את `ng` לזמין בכל מקום במחשב שלך, במקום רק בתיקייה של פרויקט אחד. אמת:

```
ng version
```

אתה אמור לראות את גרסת ה-CLI של Angular, בתוספת כותרת שמפרטת את החבילות שיותקנו. אם אתה מקבל "command not found", פתח חלון מסוף חדש – ה-shell צריך להבחין בבינארי המותקן החדש.

הערה. יש אנשים שמעדיפים אף פעם לא להתקין שום דבר גלובלית. אם זה אתה, אתה יכול להשתמש ב-`npx @angular/` במקום `cli` במקום `ng` בכל מקום בספר הזה. זה מוריד עותק זמני בכל פעם. זה איטי יותר אבל שומר על הסביבה הגלובלית שלך נקייה.

יצירת Compass

בחר תיקייה לשמור בה את הפרויקטים שלך. בספר הזה אשתמש ב-`~/dev`. תחליף בשלך אם אתה מעדיף.

```
cd ~/dev
ng new compass
```

ה-CLI שואל אותך כמה שאלות. תענה עליהן כך:

• **Which stylesheet format would you like to use?** →

`CSS`. לא נשתמש ב-Sass בספר הזה. אם אתה כבר יודע Sass, אתה יכול לבחור בזה; הקוד עדיין יעבוד.

• **...Do you want to enable Server-Side Rendering (SSR)?** →

`No`. נוסף SSR בפרק 18, ברגע שהאפליקציה תהיה שווה `prerender`.

— **Which type of routes would you like to configure?**

אם זה מופיע, בחר בברירת המחדל (`server routes with data prefetching` אם מוצע; אחרת השאר ניתוב מופעל).
CLIs חדשים יותר עושים את זה אוטומטית בלי לשאול.

ה-CLI יוצר תיקיית `compass` לצד המקום שבו הרצת את הפקודה, מוריד כמאתיים חבילות ל-`node_modules` (זה ייקח דקה או שתיים בפעם הראשונה), ועוצר. אז הוא משאיר אותך בפרומפט עם רמז קטן על `ng serve`.

קח את הרמז הזה:

```
cd compass  
ng serve --open
```

כמה שניות אחר כך, הדפדפן שלך ייפתח ב-`http://localhost:4200` ותראה את עמוד ההתחלה של Angular ברירת המחדל. ברכות – אתה מריץ אפליקציית Angular.

`ng serve` גם נשאר פתוח במסוף שלך, ומסתכל על קבצי המקור שלך. שמור שינוי, והדפדפן יטען מחדש אוטומטית. השאר את `ng serve` רץ בחלון מסוף אחד; פתח חלון מסוף שני לשארית הפרק.

סיור על כל קובץ שה-CLI ייצר

פתח את `~/dev/compass` בעורך שבחרת. אם אין לך עורך מועדף, התקן את Visual Studio Code מ-`code.visualstudio.com` ופתח את התיקייה עם `File → Open Folder`. ל-VS Code יש את התמיכה הכי טובה מקופסה ב-Angular מכל עורך חינומי; כשנתחיל לכתוב קומפוננטות בפרק 5, הודעות השגיאה שלו `inline` יחסכו לך הרבה זמן.

אנחנו הולכים לצעוד עכשיו דרך כל קובץ וכל תיקייה ברמה העליונה. זה הליכה ארוכה. אל תנסה לשנן; רק תיצור מפה מנטלית של מה כל דבר עושה. תפגוש כל אחד מהקבצים האלה שוב, בהקשר, בהמשך הספר.

`package.json`

זה התיאור של הפרויקט שלך מבחינת `npm`. שני חלקים שווים בדיקה.

`scripts` מפרט את הפקודות שאתה יכול להריץ דרך `npm run` `:<name>`

```
"scripts": {  
  "ng": "ng",  
  "start": "ng serve",  
  "build": "ng build",  
  "watch": "ng build --watch --configuration development",  
  "test": "ng test"  
}
```

`npm run start` ו-`ng serve` שווי ערך. יש קונבנציות שמעדיפות `npm run` כי הוא שומר על אותן פקודות עובדות בפרויקטים שמתמשים בכלים בסיסיים שונים; בפרויקט Angular אתה יכול להשתמש בכל אחד.

`dependencies` מפרט את הספריות שהאפליקציה שלך צריכה בזמן ריצה; `devDependencies` מפרט את הכלים שהיא צריכה במהלך פיתוח. כשגרסה חדשה של Angular יוצאת ואתה רוצה לעדכן, אתה עורך את מספרי הגרסה האלה או מריץ `ng update` – נושא ליום אחר.

`angular.json`

קובץ התצורה של ה-CLI. הוא אומר ל-Angular איך לבנות, להגיש, ולבדוק את הפרויקט. תערוך אותו ידנית לעתים רחוקות בפרקים הראשונים; נבקר בו בפרק 16 (ביצועים) ובפרק 19 (פריסה). לעת עתה, דע שהוא קיים.

tsconfig.json וחבריו

שלושה קבצי תצורת TypeScript:

- **tsconfig.json** – התצורה הבסיסית.
- **tsconfig.app.json** – התצורה שנמצאת בשימוש כשבונים את האפליקציה עצמה.
- **tsconfig.spec.json** – התצורה שנמצאת בשימוש כשמריצים בדיקות.

לא תיגע בהם עד שתתחיל לכתוב בדיקות בפרק 17. הקובץ הראשי מפעיל מצב strict כברירת מחדל בפרויקטי Angular מודרניים, שפירושו ש-TypeScript תהיה קפדנית לגבי בדיקות `any`, `null`, `implicit`, ורגלי-פיתיון אחרות. אל תכבה מצב `strict`. זו הסיבה שהאפליקציה שלך תישאר ניתנת ל-refactor שנה מהיום.

.editorconfig

קובץ זעיר שמספר לעורך שלך על הזחה, סופי שורה, ובסיסים אחרים. אם העורך שלך מבין אותו (רובם המודרניים מבינים, לפעמים עם תוסף), הפורמט שלך יתאים אוטומטית לברירות המחדל של ה-CLI.

.gitignore

קובץ שמפרט מה git לא צריך לעקוב אחריו. במיוחד הוא לא כולל את `node_modules/` (שהיא ענקית וניתן לשחזר אותה מ-`package.json`) ואת `dist/` (פלט הבנייה). ניצור מאגר git בעוד רגע.

`node_modules/`

החבילות המותקנות. Angular עצמה חיה כאן, יחד עם כל מה ש-Angular תלויה בו ובכל מה שתתקין מאוחר יותר. אל תבדוק את התיקייה הזאת לתוך git; היא לעתים קרובות גיגה-בייטים בגודל וניתן לשחזר אותה במלואה עם `npm install`. אם תמחק אותה, תוכל ליצור אותה מחדש עם `npm install`.

`public/`

קבצים

סטטיים שמוגשים כמו שהם בשורש האפליקציה שלך. אם תפיל `robots.txt` או `favicon.ico` כאן, הם יופיעו ב-`/robots.txt` וב-`/favicon.ico` באפליקציה הרצה. ה-CLI מפיל `favicon.ico` ברירת מחדל כאן.

`src/index.html`

פתח אותו. הוא מאוד קצר:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Compass</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

האלמנט `<app-root>` הזה הוא נקודת עגון יחידה לכל האפליקציה. כשהדפדפן טוען את הדף, Angular מתאחזת, מוצאת את האלמנט הזה, ומשתלטת עליו. כל מה שאתה רואה ב-`http://localhost:4200` מרונדר בתוך `<app-root>`.

תערוך את `index.html` לעתים רחוקות. תערוך את הכותרת שלו, מדי פעם תוסיף `<link>` לפונט, ותשאיר אותו לבד.

`src/styles.css`

CSS גלובלי. כל מה שיש כאן חל על כל האפליקציה. ייבואי פונטים, reset של CSS, ומאפיינים מותאמים אישית ברמת השורש (משתני CSS) חיים כאן. סגנונות ספציפיים לקומפוננטה – הסגנונות של שורת משימה, נניח – חיים בקובץ הקומפוננטה, שנפגוש בפרק 5.

`src/main.ts`

קובץ ה-`bootstrap`. מאוד קצר:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch(err => console.error(err));
```

`bootstrapApplication` אומר: "התחל את אפליקציית Angular, `App` היא השורשית שלה היא `appConfig`, באיזה אלמנט ב-HTML שיש לו את התג המוגדר על ידי הסלקטור של `App`". התוצאה היא ש-`<app-root>` ב-`index.html` מתמלא בקומפוננטה השורשית המרונדרת.

תערוך את `main.ts` כמעט אף פעם – אולי פעם אחת בפרק 18 כדי לאפשר רינדור בצד השרת.

`src/app/`

זה איפה שהאפליקציה שלך חיה. כל מה שתכתוב מפרק 5 והלאה נכנס ל-`src/app/` או לתת-התיקיות שלו. ה-CLI כבר שם כאן שלושה קבצים:

— `app.ts` (או `app.component.ts` בכמה גרסאות CLI) – הקומפוננטה השורשית. בערך:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('compass');
}
```

ארבעה דברים שכדאי לשים לב אליהם, שאף אחד מהם אתה לא צריך להבין עדיין:

- הדקורטור `@Component`. הוא אומר ל-Angular שהמחלקה הזאת היא קומפוננטה, ומגדיר את הקומפוננטה עם סלקטור (`<app-root>`), קובץ תבנית, קובץ סגנון, וכל קומפוננטות או דירקטיבות אחרות שהקומפוננטה הזאת משתמשת בתבנית שלה.
- גוף המחלקה. כל דבר שהקומפוננטה רוצה לחשוף לתבנית שלה נכנס כאן – שדות, מתודות, סיגנלים.
- הקריאה `signal(...)`. זה הפרימיטיב של מצב ריאקטיבי מודרני של Angular. התבנית יכולה לקרוא `title()` והיא תרונדר מחדש אם הסיגנל אי פעם ישתנה.
- `imports: [RouterOutlet]`. קומפוננטות עצמאיות מכריזות אילו קומפוננטות ודירקטיבות אחרות הן

משתמשות. RouterOutlet הוא ה-placeholder שבו
ה-router ירנדר עמודים.

כל פרק מפרק 5 והלאה מבלה זמן בתוך קומפוננטות כמו זו. אל
תדאג לפרש אותה עכשיו.

app.html – התבנית של הקומפוננטה השורשית. ה-CLI ממלא
את זה עם עמוד התחלה ממותג של Angular. תמחק את רובו בפרק 5.

app.css – סגנונות scoped ל-קומפוננטה השורשית. כל מה
שתכתוב כאן חל על `<app-root>` ולא על שום דבר אחר – גם אם
יש לך class בשם `.container`. בקובץ הזה ועוד class
בשם `.container`. במקום אחר, הם לא יתנגשו. זה נקרא view
encapsulation, וזה אחד הדברים הטובים בשקט לגבי כתיבת Angular.

app.config.ts – תצורה ברמת האפליקציה.

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
  ],
};
```

`providers` היא הרשימה של שירותים ותצורה שהאפליקציה שלך
הופכת לזמינים בכל מקום. ב-Angular מודרנית, רוב הדאגות
החוצות-רוחביות מוקמות על ידי קריאה לפונקציות `provideX()`

כאן – ניתוב, HTTP, אנימציות, וכן הלאה. נסיף כמה מהן בפרקים מאוחרים יותר.

app.routes.ts – רשימה ריקה בהתחלה של הגדרות ניתוב:

```
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

פרק 13 ממלא אותה והופך את Compass לאפליקציה רב-עמודית.

מה ה-CLI יכול לעשות מלבד `new` ו-`serve`

תרגל את ההרגל של שימוש ב-`ng generate` לבניית תבניות. זה חוסך הקלדה, ויותר חשוב, זה מציב קבצים חדשים במקום הנכון עם הייבואים הנכונים.

```
ng generate component task-row      # יוצר src/app/task-row/*
ng generate service tasks           # יוצר src/app/tasks.service.ts
ng generate pipe date-friendly     # יוצר src/app/date-friendly.pipe.ts
```

אתה יכול לקצר ל-`ng g c task-row`. בספר הזה תמיד נקליד את הצורה הארוכה כדי שתדע מה אתה עושה; בפרקטיקה יומית תקצר.

פקודות אחרות ששוות לדעת:

```
ng serve      # שרת פיתוח; מסתכל על קבצים, טוען מחדש בשמירה
ng build      # dist/ בנייה להפקה לתוך /
ng test       # מריץ את בדיקות היחידה
ng lint       # אם מוגדר, linter-מריץ את ה
ng update     # וחבילות קשורות Angular מעדכן את
```

Git, ברגע זה

לפני שאתה כותב שורת קוד יחידה, יצור מאגר git. האני העיתידי שלך יודה לך בפעם הראשונה שתשבור משהו ותרצה לראות מה היה לך לפני חמש דקות.

```
git init
git add .
git commit -m "Initial commit from ng new"
```

אם מעולם לא השתמשת ב-git, בלה עשרים דקות עם שני הפרקים הראשונים של כל מדריך git ראשוני (git-scm.com/book) הוא חינם). תשתמש ב- `git status`, `git add`, `git commit`, ו-`git diff` בערך חמישים פעם כל אחד בספר הזה.

לוודא שהכל עובד

עם `ng serve` עדיין רץ מקודם, פתח את `src/app/app.html` בעורך שלך. מצא את ה- `<h1>` הראשון (או מקום עם טקסט גלוי) ושנה טקסט כלשהו ל- `Hello, Compass.` שמור את הקובץ.

טאב הדפדפן שלך אמור להתרענן תוך שנייה, והטקסט החדש אמור להופיע. אם כן, יש לך סביבת פיתוח Angular עובדת.

אם לא – העמוד לא מתעדכן, או שהמסוף מדפיס שגיאה – ראה את סעיף פתרון הבעיות בנספח א', או את נספח ב', שהוא מדריך לקריאת הודעות שגיאה של Angular.

מה בא הלאה

חלק שני. בפרק 5 נמחק את רוב תוכן ההתחלה של ה-CLI ונבנה את הקומפוננטה האמיתית הראשונה של Compass: רשימה של משימות. תפגוש תבניות, קישורי נתונים, ואת תחביר בקרת הזרימה החדש. עד סוף פרק 5, Compass תהיה רשימת מטלות – פשוטה מאוד, רצה כולה בדפדפן, בלי כל השמירה, הניתוב או האימות של המוצר המוגמר. כל פרק עוקב מוסיף עוד יכולת אמיתית אחת.

תרגילים

1. הרץ `ng generate component about --skip-tests` וקרא את הפלט. הדגל `--dry-run` אומר ל-CLI להדפיס מה הוא היה עושה בלי לעשות זאת בפועל. אילו קבצים היו נוצרים? עשה את אותו הדבר עבור `ng generate service auth --dry-run`. זה הרגל שימושי לתצוגה מקדימה של מה ה-CLI רוצה לעשות לפני שהוא עושה זאת.

2. פתח את כלי הפיתוח של הדפדפן (F12 ברוב הדפדפנים) ולחץ

על טאב "Elements" או "Inspector". מצא את האלמנט `<app-root>` ב-DOM. שים לב שהוא אלמנט אמיתי בעמוד, והתוכן של האפליקציה מרונדר בתוכו. זה שווה לראות פעם אחת כדי שתדע ש-Angular לא עושה שום דבר קסום ל-HTML.

3. ב-`src/app/app.html`, שים `{{ title() }}` איפשהו שהטקסט יהיה גלוי, ושמור. שים לב לסוגריים אחרי `title` – זה לא טעות הקלדה. סיגנלים הם פונקציות שאתה קורא להן כדי לקבל את הערך הנוכחי שלהן. בפרק 7 נבחן למה. לעת עתה, פשוט תבחין שזה עובד.

פרק 5. קומפוננטות: האטום של אפליקציית Angular

הקמנו את הסדנה; עכשיו נתחיל לבנות. זה הפרק הראשון שבו אתה כותב קוד Angular אמיתי. עד סוף הפרק תמחק את התבנית ההתחלתית של ה-CLI, תחליף אותה בקומפוננטה משלך, ותציג בדפדפן רשימה של משימות מקודדות. זה הישג קטן בפני עצמו, וזה גם הרגע שבו הצורה של אפליקציית Angular מתבררת.

השאר את `ng serve` רץ במסוף אחד, ואת העורך שלך פתוח על תיקיית `compass`. כל בלוק קוד בפרק הזה נועד להיות מוקלד ונצפה.

מהי קומפוננטה, בשנית

בפרק 1 תיארתי קומפוננטה כפיסה עצמאית של המסך: קצת HTML, קצת CSS, וקצת התנהגות, ארוזים יחד. עכשיו כשיש לך פרויקט בדיסק שלך, אנחנו יכולים להיות יותר קונקרטיים. קומפוננטה היא:

- **מחלקת TypeScript** שמחזיקה את המצב של הקומפוננטה

(שדות) ואת ההתנהגות (מתודות).

- **מקושטת ב-@Component** — פיסת מטא-דאטה שאומרת

ל-Angular "המחלקה הזאת היא קומפוננטה, והנה

הסלקטור והתבנית שלה".

- **מקושרת לתבנית**, שהיא HTML מוגדל עם תחביר הקישור

של Angular.

- **מקושרת אופציונלית לסגנונות**, `scoped` כך שלא ידלפו לשאר

האפליקציה.

- **בשימוש בתבנית של קומפוננטה אחרת לפי שם**, דרך

הסלקטור שלה.

הנקודה האחרונה היא המנגנון שדרכו נוצר עץ קומפוננטות. הקומפוננטה השורשית של Compass (App) תכלול בסופו של דבר `<app-task-list>` בתבנית שלה. ה-`<app-task-list>` הזה הוא קומפוננטה נוספת שאנחנו עומדים לכתוב. בתוך `<app-task-list>` יהיו מופעים רבים של `<app-task-row>`, אחד למשימה. כל אלמנט שמתחיל ב- `app-` בתבניות שלך הוא קומפוננטה שהגדרת; כל אלמנט שלא (כמו `<div>` או `<button>`) הוא HTML רגיל.

הדקורטור `@Component`, שדה אחרי שדה

פתח את `src/app/app.ts` (או `app.component.ts`; שם הקובץ תלוי בגרסת ה-CLI, ואת ברירות המחדל של ה-CLI השתנו). הוא נראה בערך כך:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('compass');
}
```

בואו נצעד דרך שדות ה- `@Component` בזה אחר זה.

selector הוא שם התג HTML שמייצג את הקומפוננטה הזאת. `'app-root'` פירושו שהקומפוננטה הזאת מופיעה כל מקום ש- `<app-root>` כתוב. סלקטורים הם איך שקומפוננטות מוצאות זו את זו; הם גם איך שהעולם החיצוני (במקרה הזה, `index.html`) יכול להשליט את האפליקציה שלך לתוך דף. הקונבנציה של Angular היא קידומת קצרה ספציפית לאפליקציה (`app-` כאן) ואחריה תפקיד הקומפוננטה. תקרא לקומפוננטות בשמות כמו `<app-task-row>`, `<app-add-task-form>`, `<app-habit-badge>`.

למה הקידומת? כדי למנוע התנגשויות עם אלמנטים אמיתיים של HTML ועם קומפוננטות של צד שלישי. תג כמו `<button>` הוא אלמנט HTML native; תג כמו `<mat-button>` הוא מ-Angular; Material; תג כמו `<app-button>` הוא שלך. הקידומת שומרת אותם ישרים.

imports היא רשימה של קומפוננטות, דירקטיבות, ופינים אחרים שהתבנית של הקומפוננטה הזאת משתמשת בהם. ב-Angular מודרנית, כל קומפוננטה עצמאית מכריזה על התלויות שלה במפורש. אם התבנית שלך מכילה `<router-outlet>`, אתה חייב לייבא `RouterOutlet`; אם היא מכילה `<app-task-row>`, אתה חייב לייבא `TaskRow`. זה אחד הדברים שמפתיע חדשים ("אני באמת צריך לפרט הכל?") ואחר כך מרוויח את מקומו בשקט ("העברתי את הקומפוננטה הזאת לאפליקציה אחרת והיא עדיין עובדת").

templateUrl מצביע על קובץ ה-HTML שהקומפוננטה הזאת מרנדרת. לחלופין אתה יכול להשתמש ב-**template**, שמקבל מחרוזת:

```
@Component({
  selector: 'app-hello',
  template: '<h1>Hello.</h1>',
})
export class Hello {}
```

תבניות inline נוחות לקומפוננטות זעירות (מתחת לכ-10 שורות). כל דבר גדול יותר שייך לקובץ נפרד. הספר משתמש בשניהם, ומעדיף קבצים נפרדים לקומפוננטות אמיתיות ו-inline לשורות בודדות.

styleUrl (או **styles**, או **styleUrls**) מצביע על ה-CSS ה-scoped של הקומפוננטה. הכל בקובץ חל רק בתוך מופעים של הקומפוננטה הזאת – class בשם `.title`. כאן לא יתנגש עם class בשם `.title` במקום אחר באפליקציה. זה נעשה דרך view encapsulation, ש-Angular מיישמת על ידי הוספת attribute ייחודי לכל אלמנט שהקומפוננטה מרנדרת ושכתוב ה-CSS שלך לכוון ל-attribute

הזה. אתה לא צריך לחשוב על המנגנון; אתה רק צריך לדעת ש-CSS שאתה כותב בקומפוננטה נשאר בתוך הקומפוננטה.

בניית הקומפוננטה האמיתית הראשונה של

Compass

בואו נחליף את התחלת ה-CLI במשהו שאנחנו הבעלים שלו. נבנה קומפוננטה `TaskList` שמראה רשימה מקודדת של משימות. בפרקים מאוחרים נחליף את הנתונים המקודדים בסיגנלים, אז בשירות, אז ב-HTTP.

מתיקיית `compass` שלך:

```
ng generate component task-list
```

זה יוצר `src/app/task-list/` שמכיל:

```
task-list/  
├─ task-list.ts           (או task-list.component.ts)  
├─ task-list.html  
├─ task-list.css  
└─ task-list.spec.ts
```

פתח את `task-list.ts`:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-task-list',
  imports: [],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {}
```

נתן לקומפוננטה הזאת טיפוס למשימות ורשימה מקודדת. קודם, בואו נשים את הטיפוס בקובץ משלו כדי שנוכל לשתף אותו מאוחר יותר. יצור `src/app/task.ts`:

```
export interface Task {
  id: string;
  title: string;
  done: boolean;
  createdAt: string;
  dueDate: string | null;
  tags: string[];
}
```

עכשיו ערוך את `task-list.ts`:

```
import { Component } from '@angular/core';
import { Task } from '../task';

@Component({
  selector: 'app-task-list',
  imports: [],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  tasks: Task[] = [
    {
      id: 't1',
      title: 'Buy milk',
      done: false,
      createdAt: '2026-01-15T09:00:00Z',
      dueDate: null,
      tags: ['home'],
    },
    {
      id: 't2',
      title: 'Write chapter 5',
      done: true,
      createdAt: '2026-01-14T18:30:00Z',
      dueDate: '2026-01-16T00:00:00Z',
      tags: ['work'],
    },
    {
      id: 't3',
      title: 'Call the plumber',
      done: false,
      createdAt: '2026-01-15T11:00:00Z',
      dueDate: '2026-01-18T00:00:00Z',
      tags: ['home', 'urgent'],
    },
  ];
}
```

עכשיו ערוך את `task-list.html` כדי להציג אותם. פרק 6 מכסה את התחביר לעומק; לעת עתה, העתק אותו וצפה:

```
<h1>Tasks</h1>
<ul class="task-list">
  @for (task of tasks; track task.id) {
    <li class="task" [class.done]="task.done">
      <span class="title">{{ task.title }}</span>
      @if (task.dueDate) {
        <span class="due">due {{ task.dueDate | date: 'shortDate' }}</span>
      }
    </li>
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>
```

ותן לו קצת עיצוב ב-`task-list.css`:

```
:host {  
  display: block;  
  max-width: 480px;  
  margin: 2em auto;  
  font-family: system-ui, sans-serif;  
}  
  
h1 {  
  font-size: 1.4rem;  
  margin: 0 0 1em 0;  
}  
  
.task-list {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
}  
  
.task {  
  padding: 12px 14px;  
  border-bottom: 1px solid #eee;  
  display: flex;  
  gap: 12px;  
  align-items: center;  
}  
  
.task.done .title {  
  text-decoration: line-through;  
  color: #999;  
}  
  
.due {  
  margin-left: auto;  
  font-size: 0.85rem;  
  color: #666;  
}
```

```
.empty {  
  padding: 20px;  
  text-align: center;  
  color: #999;  
  font-style: italic;  
}
```

שימוש בקומפוננטה

שני שינויים נוספים ונוכל לראות אותה בדפדפן.

ראשית, אמור לקומפוננטה השורשית לרנדר `<app-task-list>`.
ערוך את `src/app/app.ts`:

```
import { Component } from '@angular/core';  
import { TaskList } from './task-list/task-list';  
  
@Component({  
  selector: 'app-root',  
  imports: [TaskList],  
  templateUrl: './app.html',  
  styleUrls: ['./app.css'],  
})  
export class App {}
```

שני שינויים חשובים: הסרנו את ייבוא `RouterOutlet` (נחזיר אותו בפרק 13), והוספנו את `TaskList` ל-`imports`. כל קומפוננטה שמופיעה בתבנית של הקומפוננטה הזאת חייבת להיות מיובאת.

שנית, ערוך את `src/app/app.html`. מחק את מה שה-CLI שם שם והחלף ב:

```
<app-task-list></app-task-list>
```

שמור. תסתכל על טאב הדפדפן שלך. אמורה להיות לך כותרת "Tasks" ושלושה פריטים – אחד עם קו חוצה (המושלם) ושניים עם תאריכי יעד.

בדיוק כתבת את הקומפוננטה הראשונה של Angular ורכבת אותה באפליקציה. קח דקה להעריך את זה.

מה `:host` עושה

בטח שמת לב לפסאודו-קלאס `:host` ב-`task-list.css`. כך גיליון הסגנונות של קומפוננטה מתייחס לאלמנט השורשי של הקומפוננטה עצמה – `<app-task-list>` עצמו, מבחוץ. בתוך קובץ ה-CSS, `:host` הוא ה-`wrapper`; כל מה שבתוך התבנית של הקומפוננטה הוא תוכן צאצא של `:host`.

ההבחנה חשובה כי Angular מרנדרת כל קומפוננטה לתוך אלמנט משלה. כש-Angular רואה `<app-task-list>` בתבנית של `App`, היא לא מחליפה את התג במרקאפ של רשימת המשימות; היא מכניסה את המרקאפ בתוך התג. זה משמר את הגבול של האנקפסולציה ונותן לך מקום לתלות סגנונות כמו `display: block` או `margin` על הקומפוננטה בכללותה.

מחזור חיים של קומפוננטה, בפסקה אחת

כל קומפוננטה עוברת כמה רגעים בחיים שלה: היא נבנית (המחלקה מקבלת מופע), אז מאותחלת (Angular חיברה את ה-inputs שלה), ובסופו של דבר מושמדת (הקומפוננטה מוסרת מה-DOM, וצריכה לנקות Angular subscriptions מספקת hooks של מחזור חיים – מתודות שאתה יכול להגדיר עם שמות ספציפיים – עבור הרגעים האלה: `ngOnInit`, `ngOnDestroy`, וכמה אחרים. בקוד Angular ישן יותר, אלה היו בכל מקום.

ב-Angular מודרנית, רוב הסיבות שהיית פונה ל-hook של מחזור חיים משורתות טוב יותר על ידי פרימיטיבים אחרים – סיגנלים, `computed`, `effect`, וריאקטיביות מבוססת DI. השימושים הכשרים שנשארים צרים: לעשות הקמה חד-פעמית שצריכה לראות inputs (נדיר – `computed` בדרך כלל טוב יותר), או ניקוי משאבים חיצוניים שלא מנוהלים על ידי Angular (נדיר – `takeUntilDestroyed` בדרך כלל טוב יותר). נפגוש את ה-hooks כראוי כשנצטרך אותם; אל תלך לחפש אותם.

ידידותי לסיגנלים כברירת מחדל

שים לב שבפרק הזה, `tasks` הוא שדה רגיל על המחלקה – `tasks`: `Task[] = [...]`. זה עובד: התבנית קוראת אותו ומרנדרת. אבל הוא לא ריאקטיבי. אם תוסיף מאוחר יותר כפתור שדוחף משימה חדשה לרשימה, Angular תרנדר מחדש את הרשימה (בזכות זיהוי שינויים מבוסס `zone`, שהוא המנגנון הקלאסי); אבל ההתנהגות הזאת

בדרך החוצה. Angular מודרנית זזה לעבר סיגנלים כפרימיטיב הריאקטיבי, ו-Compass יעשה את המעבר הזה החל מפרק 7.

לעת עתה, שדות רגילים מספיקים. פרק 6 מבלה את זמנו על תחביר התבנית (`@if`, `@for`, `{{ }}`); פרק 7 משכתב את `tasks` כסיגנל. הדפוס של הספר הוא: להציג את הגרסה הרגילה, להרגיש את הגירוד, ללמוד את הגרסה הריאקטיבית, לנשום לרווחה.

תבניות inline לעומת חיצוניות

שני סגנונות, שניהם בסדר:

```
// generate`-ברירת מחדל מ) חיצוני`
@Component({
  selector: 'app-task-list',
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})

// Inline
@Component({
  selector: 'app-empty-state',
  template: `
    <div class="empty">
      <p>{{ message }}</p>
    </div>
  `,
  styles: `
    .empty { padding: 20px; text-align: center; }
  `,
})
```

כללי אצבע: תבניות של עד כ-10 שורות של מרקאפ בסדר `inline`; כל דבר גדול יותר קל יותר לקרוא בקובץ נפרד. אם התבנית ה-`inline` שלך מתחילה להכיל `@for` ומספר ענפי `@if`, העבר אותה לקובץ `.html` – העורך שלך יודה לך (סימון תחביר, השלמה אוטומטית, הזחה כולם משתפרים).

View encapsulation, בקצרה

לכל קומפוננטה יש מצב `ViewEncapsulation`. ברירת המחדל היא `Emulated`, שזה מה שתיארנו למעלה – Angular מגבילה את ה-CSS שלך לקומפוננטה על ידי הוספת `attributes`. `None` משבית את ההגבלה

(ה-CSS שלך הופך גלובלי – בדרך כלל לא מה שאתה רוצה). `ShadowDom` משתמש ב-API האמיתי של `Shadow DOM` של הדפדפן להגבלה, שהוא חזק יותר אבל מגיע עם מוזרויות סביב עיצוב, פונטים, וספריות צד שלישי.

תמיד תשאיר את האנקפסולציה על ברירת המחדל. הפעם היחידה שתשנה אותה היא אם היית בונה ספריית קומפוננטות וצריך את הבידוד החזק ביותר האפשרי.

מה בא הלאה

פרק 6 מכסה את כל מה שקרה עכשיו בתבנית. ראית `{{ }}`, `[class.done]`, `@for`, `@if`, `@empty`, ו-`date:` | בפרק 6 נסתכל על כל אחד מהם כראוי, יחד עם

property bindings, event bindings, ו-two-way bindings. עד סוף פרק 6, Compass יגיב ללחיצות – תוכל להעביר משימות בין בוצע ולא בוצע.

תרגילים

1. הוסף משימה רביעית למערך המקודד – אחת שהיא `done: true` ויש לה `dueDate` בעבר. שמור וודא שהיא מרנדרת עם הקו החוצה ותאריך הפיגור שלה. אל תשתמש בשום תכונת Angular מלבד אלה שהפרק הזה כיסה.
2. שנה את ה-selector ב-`task-list.ts` מ-`'app'` ל-`'task-list'` ל-`'compass-task-list'`. שמור ורענן את הדפדפן. מה נשבר? תקן את זה. שים לב שסלקטורים הם חוזה קטן בין שני קבצים (הקומפוננטה והמשתמש בה).
3. הרץ `ng generate component habit-badge`. קרא את ארבעת הקבצים שה-CLI מייצר. אילו מהם הם מידע חדש אחרי הפרק הזה? אילו הם boilerplate שאתה יכול עכשיו לכתוב מהזיכרון?

פרק 6. קישור נתונים ובקרת הזרימה החדשה

תבניות הן איך ש-Angular מתארת ממשק משתמש. הן HTML בתוספת סט קטן של הרחבות: אינטרפולציות להצגת נתונים, קישורים לחיבור נתונים ל-attributes ולאירועים, ותחביר בלוקים לרנדור רשימות ותנאים. הפרק הזה מכסה כל הרחבה שתשתמש בה יומיומית. עד הסוף, Compass יאפשר לך ללחוץ על משימה כדי להעביר אותה בין בוצע ולא בוצע.

שמור על מצב Compass מפרק 5. נסיף ל-`task-list.ts` ול-`task-list.html` לאורך הפרק.

אינטרפולציה: הכנסת ערכים לדף

פגשת את `{{ }}` בפרק 5. הוא מעריך ביטוי TypeScript מול השדות של הקומפוננטה ומכניס את התוצאה כטקסט.

```
<h1>{{ heading }}</h1>
<p>You have {{ tasks.length }} tasks.</p>
<p>Active: {{ tasks.filter(t => !t.done).length }}</p>
```

כל מה שאתה שם בין הסוגריים הכפולים הוא ביטוי, לא הצהרה.
משמעות הדבר:

- אתה יכול לקרוא למתודות ולהשתמש באופרטורים:
`{{ user.firstName + ' ' + user.lastName }}`
- אתה לא יכול להקצות, לזרוק, או להשתמש ב-`new` `{{ x . new }}`
`{{ 5 }}` לא ידור כביטוי תבנית.
- הביטוי חייב להיות נטול תופעות לוואי, אחרת Angular
תקרא לו מספר פעמים בשנייה והאפליקציה תתנהג לא
כשורה.

אינטרפולציה תמיד מייצרת טקסט. כדי להקשר ל-`attributes` או ל-`properties` של אלמנט, תשתמש באחת מצורות הקישור למטה.

Property binding: [prop]="expression"

Property binding מגדיר את ה-`property` של אלמנט HTML (לא את ה-`attribute` – אלה שונים, עוד על זה בקרוב) לערך של ביטוי.

```
<img [src]="user.avatarUrl" [alt]="user.name" />  
<input [value]="task.title" [disabled]="task.done" />  
<a [href]=""/tasks/" + task.id">Details</a>
```

הסוגריים המרובעים משמאל מציינים את ה-`property`. הגרשיים הכפולים מימין מחזיקים ביטוי TypeScript שמוערך בהקשר של הקומפוננטה.

Property לעומת attribute

ל-HTML יש גם attributes (מה שאתה כותב במקור) וגם properties (מה שהאובייקט של ה-DOM בעצם מחזיק). לרוב האלמנטים הם מיושרים, אבל לא תמיד. `<input value="hello">` מגדיר את ה-attribute ההתחלתי; לאלמנט ה-DOM הרץ יש `.value` property שמשתנה כשהמשתמש מקליד. Property binding כותב ל-property.

במקרים הנדירים שאתה באמת צריך להגדיר attribute (בדרך כלל עבור ARIA), ל-Angular יש תחביר מיוחד:

```
<button [attr.aria-label]="Delete task ' + task.title">x</button>
```

`[attr.name]` מגדיר את ה-attribute של HTML בשם `.name`. אם Angular אי פעם נותנת לך שגיאה של "no such property" על אלמנט, זה לעתים קרובות התיקון.

קיצורי class ו-style

שתי צורות מיוחדות של property binding כל כך נפוצות שהן מקבלות תחביר קצר:

```
<li [class.done]="task.done" [class.overdue]="isOverdue(task)">  
<p [style.color]="task.done ? '#999' : 'inherit'">{{ task.title }}</p>
```

`[class.name]` מעביר את ה-class על סמך האמת של הביטוי. `[style.property]` מגדיר את property הסגנון inline. שתי הצורות מקבלות מספר ערכים – אתה יכול להחזיק גם `[class.done]` וגם `[class.overdue]` על אותו אלמנט.

לוגיקה יותר מורכבת של class או style, קשור אובייקט או מערך:

```
<li [class]="{ done: task.done, overdue: isOverdue(task) }">  
<li [class]="cssClasses()">
```

Event binding: "(event)='handler(...)"

Event bindings מריצים מתודה של קומפוננטה כשאירוע DOM נורה.

```
<button (click)="markDone(task)">Done</button>  
<input (input)="onSearch($event)" />  
<form (submit)="save()">...</form>
```

הסוגריים משמאל מציינים את האירוע (כל אירוע DOM: `click`, `input`, `focus`, `keydown`, כל דבר). הצד הימני הוא הצהרה – זה המקום היחיד בתבנית שאתה יכול לקרוא למתודה לתופעת הלוואי שלה. `$event` בתוך ההצהרה מתייחס לאובייקט האירוע (אותו אובייקט שהיית מקבל ב-JavaScript listener רגיל).

בואו נשתמש בזה ב-Compass. פתח את `task-list.ts` והוסף מתודה:

```
export class TaskList {  
  tasks: Task[] = [ /* ... */ ];  
  
  toggle(task: Task): void {  
    task.done = !task.done;  
  }  
}
```

וב- task-list.html, חבר את הלחיצה:

```
<li class="task" [class.done]="task.done" (click)="toggle(task)">
  <span class="title">{{ task.title }}</span>
  @if (task.dueDate) {
    <span class="due">due {{ task.dueDate | date: 'shortDate' }}</span>
  }
</li>
```

שמור. בדפדפן, לחץ על כל משימה. היא אמורה לעבור בין בוצע ולא בוצע, עם הקו החוצה מופיע ונעלם.

שני דברים ששווה לציין:

- מטפל הלחיצה משנה את task.done במקום. זה עובד,

אבל זה עובד רק כי זיהוי השינויים מבוסס zone של

Angular בודק מחדש תבניות אחרי כל אירוע DOM. זה

שביר: סיגנלים (פרק 7) יגרמו לאותו קוד להיות יותר

מפורש ויותר מתנהג היטב.

- כל ה- עכשיו לחיץ. זה בסדר לעת עתה אבל

פרקטיקת נגישות רעה – יישום ראוי היה משתמש ב-

<button> או מוסיף טיפול במקלדת. נתקן את זה

כשנחזור לנגישות.

Two-way binding: [(model)]="value"

Two-way binding הוא התחביר של Angular לדפוס הנפוץ של "לקשר

property פנימה ולהאזין לשינויים שיוצאים". השימוש הקנוני הוא inputs

של טפסים:

```
<input [(ngModel)]="task.title" />
```

בקריאה מימין לשמאל, הסוגריים בתוך הסוגריים המרובעים הם המזמון של "בננה בקופסה". התחביר מתרחב ל-property binding על ngModel ו-event binding על ngModelChange :

```
<!-- שווה ערך לזה למעלה -->  
<input [ngModel]="task.title" (ngModelChange)="task.title = $event" />
```

ngModel הוא יישום ספציפי אחד של דפוס ה-two-way. binding מבוסס סיגנלים, שנשתמש בו ב-Compass, משתמש ב-model input — מושג שנפגוש בפרק 8. לעת עתה, דע רק ש-`[(x)]` הוא התחביר של Angular ל-two-way binding, ושהסמנטיקה שלו היא תמיד "property פנימה, event החוצה". אין קסם — זו לא ספריית קישור נתונים — רק חבילה תחבירית.

בקרת הזרימה החדשה

תבניות צריכות לקבל החלטות ("הראה את זה אם המשתמש מחובר") ולחזור ("רנדר שורה אחת לכל משימה"). התחביר המודרני של Angular לאלה נקרא בקרת הזרימה החדשה: `@if`, `@for`, `@switch`, `@empty`, ו-`@let`. הם מחליפים סט ישן יותר של דירקטיבות מבניות — `*ngIf`, `*ngFor`, `*ngSwitchCase` — שאתה עדיין תראה בקוד קיים. שניהם עובדים; התחביר החדש הוא ברירת המחדל המומלצת בכל פרויקט שנוצר אחרי אמצע 2024.

@if -1 @else

```
@if (user.loggedIn) {  
  <p>Welcome back, {{ user.name }}.</p>  
} @else if (user.pending) {  
  <p>Confirming your email...</p>  
} @else {  
  <a routerLink="/login">Sign in</a>  
}
```

סוגריים נדרשים סביב הענפים. אתה יכול לקנן חופשית. אין דו-משמעות לגבי איפה ענף מסתיים.

@for -1 track

```
@for (task of tasks; track task.id) {  
  <li>{{ task.title }}</li>  
}
```

הפסקה `track` נדרשת. היא אומרת ל-Angular איך לזהות כל פריט לרוחב רנדורים. כשהמערך משתנה – אתה מוסיף משימה, מסיר אחת, מסדר מחדש – Angular משתמש ב-`track` כדי להחליט אילו צמתים קיימים של DOM מתאימים לאילו פריטים במערך החדש. `track` נכון פירושו ש-Angular משתמש שוב בצמתי DOM (מהיר); `track` שגוי פירושו ש-Angular יוצר מחדש כל צומת בכל פעם (איטי).

Track לפי מזהה יציב – `id` של מסד נתונים, מחרוזת ייחודית – כשיש לך אחד. Track לפי אינדקס (`track $index`) רק כמוצא אחרון, ורק כשפריטים ממש חליפיים.

בלוק `@for` יש משתני הקשר שימושיים אחרים:

```
@for (task of tasks; track task.id; let i = $index; let first = $first) {  
  <li>  
    #{{ i + 1 }}: {{ task.title }}  
    @if (first) { <span>(first)</span> }  
    @if (last) { <span>(last)</span> }  
  </li>  
}
```

זמינים: `$count`, `$odd`, `$even`, `$last`, `$first`, `$index`.

ובאופן חשוב, בלוק `@empty`:

```
@for (task of tasks; track task.id) {  
  <li>{{ task.title }}</li>  
} @empty {  
  <li>Nothing to do.</li>  
}
```

`@empty` מרנדר כשהמערך ריק. זה מחליף אידיום שהיית כותב עם `@if (tasks.length === 0)` נפרד – יותר נקי, ומאפשר ל-Angular לייעל את המקרה הריק.

`@default`, `@case`, `@switch`

לשיגור רב-ענפי על ערך יחיד:

```
@switch (task.status) {  
  @case ('todo') { <span class="badge todo">To do</span> }  
  @case ('doing') { <span class="badge doing">In progress</span> }  
  @case ('done') { <span class="badge done">Done</span> }  
  @default { <span class="badge">?</span> }  
}
```

אתה יכול לקנן `@switch` בתוך `@for`, `@if` בתוך `@switch`, וכן הלאה. אין הגבלות.

`@let`

`@let` קושר משתנה מקומי בתוך תבנית כדי שתוכל להשתמש שוב בחישוב:

```
@let activeCount = tasks.filter(t => !t.done).length;  
@let hasActive = activeCount > 0;  
  
@if (hasActive) {  
  <p>{{ activeCount }} tasks remaining.</p>  
} @else {  
  <p>All done for today.</p>  
}
```

שני דברים לדעת: קשרי `@let` הם `scoped` לבלוק שמסביב; והביטוי מוערך מחדש כשה-`inputs` משתנים, כך שהקישור נשאר טרי.

פייפים: הפוך במקום

כבר השתמשת בפייפ: `date: task.dueDate |` `shortDate'` } } . פייפ הוא פונקציה שאתה יכול להחיל בתוך ביטוי

תבנית, באמצעות |. תפקידו הוא עיצוב חד-כיווני וללא תופעות לוואי.

פיפים מובנים ששווים לדעת:

```
{{ task.title | uppercase }}
{{ task.title | lowercase }}
{{ task.title | titlecase }}
{{ task.createdAt | date: 'medium' }}
{{ percentage | percent: '1.0-2' }}
{{ price | currency: 'USD' }}
{{ someObject | json }}      <!-- נהדר לדיבוג -->
{{ items | slice: 0 : 5 }}
```

פיפים יכולים להשתרשר:

```
{{ task.title | slice: 0 : 40 | titlecase }}
```

תכתוב פיפים משלך בפרק 15. עד אז, המובנים מכסים את רוב הצרכים.

קישורי attribute שתצטרך

מעבר ל-class ו-style, כמה קישורי attribute מגיעים לעתים קרובות:

```
<button [disabled]="loading">Save</button>
<input [placeholder]="hint" [autofocus]="isNew" />
<a [href]="url" [target]="external ? '_blank' : '_self'">Link</a>
<img [src]="avatarUrl" [width]="size" [height]="size" [alt]="user.name"
```

לרוב ה-attributes של HTML יש property מתאים, ו-Angular חכם מספיק כדי לקשור ל-property. כשלא (המקרה של *-aria למעלה), השתמש ב-[attr.name].

הכל יחד: input "הוסף משימה"

בואו נוסיף טופס קטן שמוסיף משימה לרשימה. ערוך את task-
:list.ts

```
import { Component } from '@angular/core';
import { Task } from '../task';

@Component({
  selector: 'app-task-list',
  imports: [],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  tasks: Task[] = [ /* ... שלושת המשימות ההתחלתיות מפרק 5 ... */ ];

  draft = "";

  toggle(task: Task): void {
    task.done = !task.done;
  }

  add(): void {
    const title = this.draft.trim();
    if (title === "") return;
    this.tasks = [
      ...this.tasks,
      {
        id: crypto.randomUUID(),
        title,
        done: false,
        createdAt: new Date().toISOString(),
        dueDate: null,
        tags: [],
      },
    ];
    this.draft = "";
  }
}
```

שני פרטים שכדאי לתת להם שם:

- `this.tasks = [...this.tasks, newTask]` יוצר

מערך חדש במקום לשנות את הישן. ב-Angular

מבוסס-סיגנלים (פרק 7), הדפוס הזה נדרש; כאן הוא

אופציונלי אבל הרגל טוב לבנות.

- `crypto.randomUUID()` הוא מובנה של דפדפן שמייצר

id טרי. באפליקציות אמיתיות ה-id מגיע לעתים קרובות

מהשרת; לעת עתה זה מספיק.

וב- `task-list.html`, הוסף טופס בראש:

```
<h1>Tasks</h1>

<form (submit)="add(); $event.preventDefault()">
  <input
    type="text"
    placeholder="What needs doing?"
    [value]="draft"
    (input)="draft = $any($event.target).value" />
  <button type="submit" [disabled]="draft.trim() === ''">Add</button>
</form>

<ul class="task-list">
  @for (task of tasks; track task.id) {
    <li class="task" [class.done]="task.done" (click)="toggle(task)">
      <span class="title">{{ task.title }}</span>
      @if (task.dueDate) {
        <span class="due">due {{ task.dueDate | date: 'shortDate' }}</span>
      }
    </li>
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>
```

שמור. הקלד שם של משימה. לחץ על Add או תלחץ Enter. היא מופיעה בסוף הרשימה, מנוקה מה-input, והכפתור מושבת מחדש עד שתקליד שוב.

שני דברים לשים לב לגבי הטופס:

- `(submit)="add(); $event.preventDefault()"` מטפל בהגשת הטופס ועוצר את הדפדפן מטעינה מחדש של הדף. `$event.preventDefault()` היא אותה קריאה שהיית כותב ב-JavaScript רגיל.

- `(input)="draft = $any($event.target).value"` הוא two-way binding ידני. הוא קורא את ה-`value` של ה-`input` בכל הקלדה ומעתיק אותו ל-`draft`. בפרק 12 נחליף את זה בטפסים ריאקטיביים ראויים, ובפרק 8 תפגוש את `model()` – גרסה טובה עוד יותר. לעת עתה, נעשה את זה ידנית.

ה-`$any(...)` הוא מעקף: ל-`$event.target` יש טיפוס `EventTarget | null`, שלא יודע על `value` property. `$any` אומר לבודק התבנית "סמוך עליי, אל תתלונן". במתודת `קומפוננטה` עם `handler` מוקלד כראוי, נשתמש בהמרת `TypeScript` ממשית; בביטוי תבנית, `$any` הוא פתח המילוט.

מה אתה לא צריך לעשות בתבניות

תבניות צריכות להיות תיאורים דקלרטיביים של מה לרנדר בהינתן נתונים כלשהם. יש כמה דברים שעוגדים בתבניות אבל אתה לא צריך לעשות:

- **ביטויים ארוכים.** כל דבר יותר מורכב מ-`ternary` קטן או קריאת מתודה שייך למחלקת הקומפוננטה. תבניות מלאות בשרשראות `?:` וקריאות מתודה מקוננות הופכות לחפצים לקריאה בלבד.
- **קריאות מתודה שעושות עבודה אמיתית.** כל קישור מוערך מספר פעמים ככל שהאפליקציה רצה. אם תבנית אומרת `{{ expensiveComputation(item) }}`, הקריאה הזאת

קורה בכל זיהוי שינויים. השתמש בסיגנל computed (פרק 7) או העבר את העבודה למחלקה.

• **שינויים.** שום ביטוי תבנית לא צריך לשנות מצב. הצהרות

של אירועים (`click`) יכולות, אבל

אינטרפולציות וקישורי property חייבים להיות קריאות טהורות.

עקיבה אחר הכללים האלה גורמת לתבניות להיות מהירות, צפויות, וניתנות לדיבוג. שבור אותם, ותבלה שעות אחר צהריים תוהה למה משהו מרנדר מחדש ארבעים פעם בשנייה.

מה בא הלאה

פרק 7 מציג סיגנלים — פרימיטיב המצב הריאקטיבי המודרני של Angular. נשכתב את שדה `tasks` של `Compass` כסיגנל, והאפליקציה תתנהג אותו הדבר אבל מסיבות אחרות לגמרי. מתחת למכסה נעבור מ"Angular בודקת הכל אחרי כל אירוע" ל"Angular מעדכנת בדיוק את החלקים של המסך שהסיגנלים שלהם השתנו".

תרגילים

1. הוסף כפתור "Clear completed" מעל הרשימה שמסיר כל משימה שבוצעה. חבר אותו עם `event binding`; אל תשנה את המערך, ייצר חדש עם `filter`. הכפתור צריך להיות מושבת כשאין מה לנקות.

2. הצג ספירה מתחת לרשימה: "3 tasks, 1 done, 2 remaining".
השתמש ב-@let כדי לחשב את הספירות פעם אחת,
ובאינטרפולציה כדי להציג אותן.

3. ה-`<li>` הלחיץ הוא נגישות רעה. העבר את מטפל
הלחיצה לאלמנט `<button>` פנימי, והשתמש ב-CSS כדי
לגרום לכפתור להתאים ויזואלית לשורה. בונוס: הוסף
קישור (`keydown.enter`) לכפתור כדי שלחיצה על Enter
גם תעביר.

פרק 7. סיגנלים: מצב ריאקטיבי שמרגיש רגיל

הקוד של Compass שיש לך עכשיו עובד. לחץ על משימה, היא עוברת; הוסף משימה, היא מופיעה; נקה את ה-input, הכפתור מושבת מחדש. זה עובד כי Angular מריצה זיהוי שינויים – בדיקה של כל קישור בכל קומפוננטה נראית – אחרי כל אירוע DOM. באפליקציות קטנות זה בסדר. באפליקציות גדולות זה הופך לצוואר בקבוק, וזה מקשה על החשיבה על למה משהו רונדר מחדש.

סיגנלים הם התשובה של Angular לשתי הבעיות. סיגנל הוא פיסה של מצב ריאקטיבי שיודעת בדיוק מי מסתכל עליה. כשסיגנל משתנה, רק החלקים של המסך שקוראים את הסיגנל הזה מסומנים לרנדור מחדש – שום דבר אחר. ברגע שהקומפוננטות שלך קוראות סיגנלים במקום שדות רגילים, Angular יכולה לעדכן את המסך בדייקנות במקום בסחיפה. המודל המנטלי הוא ערכים שזוחפים.

הפרק הזה מציג סיגנלים, משכתב את הקוד של פרק 6 של Compass לשימוש בהם, ומכסה מתי לפנות ל-computed ול-effect. עד הסוף יהיה לך מודל מנטלי למצב ריאקטיבי שידריך כל פרק שנשאר.

הבעיה שסיגנלים פותרים

דמיין מסך Compass עם מאה משימות ושלוש קומפוננטות בעמוד: רשימת המשימות, ציפ "נותרו" בכותרת, ופס התקדמות בתחתית. כל השלוש תלויות באותם נתונים.

במודל של לפני סיגנל, הזרימה היא: אתה לוחץ על משימה, הדגל `done` שלה מתהפך, ו-Angular בודקת מחדש כל קישור בעמוד – מאה `[class.done]`, את `{{ remaining }}` של הכותרת, את `[style.width]` של פס ההתקדמות, וכל השאר. Angular מהירה מספיק כדי שזה בדרך כלל לא נראה, אבל המנגנון הוא: כשיש ספק, בדוק מחדש הכל.

עם סיגנלים, הזרימה היא: אתה לוחץ על משימה; הסיגנל `done` של המשימה משתנה; Angular יודעת בדיוק אילו קישורי תבנית קוראים את הסיגנל הזה (`[class.done]`), אחד, פריט אחד בחישוב `remaining`, פריט אחד בחישוב `progress`) ומעדכנת רק אותם. עשרים עדכונים במקום עשרת אלפים.

הזכייה בביצועים אמיתית אבל לא הסיבה לפנות לסיגנלים. הסיבה היא שהקוד שלך הופך מדויק. ל"מאיפה הערך הזה מגיע?" יש תמיד תשובה – שרשרת של סיגנלים וחישובים שאתה יכול לעקוב אחריה. ל"למה זה רונדר מחדש?" יש תמיד תשובה – סיגנל שהוא קורא השתנה.

יצירת סיגנל

הסיגנל הפשוט ביותר עוטף ערך יחיד:

```
import { signal } from '@angular/core';  
  
const count = signal(0);
```

כדי לקרוא את הסיגנל, קרא לו כפונקציה:

```
console.log(count()); // 0
```

זה במכוון. סיגנלים הם פונקציות, וקריאה היא פעולה. כל קריאה מנוטרת – Angular רושמת מי קרא את הסיגנל כדי שיוכל להודיע לו מאוחר יותר.

כדי לכתוב, השתמש ב- `set` או ב- `update` :

```
count.set(10);  
count.update(n => n + 1);
```

`set` מחליף את הערך; `update` מחשב את הערך הבא מהנוכחי. לרוב המצב, שניהם מתחלפים. `update` שימושי כשהערך החדש תלוי בישן – מונה, החלפה, הוספה למערך.

זה הכל. קרא עם `()`, כתוב עם `set` או `update`, ו-Angular עושה את השאר.

קריאה בתבניות

אותו כלל חל בתבניות: כדי לקרוא סיגנל, קרא לו כפונקציה.

```
<p>Count: {{ count() }}</p>
```

הסוגריים לא אופציונליים. `{{ count }}` מדפיס את אובייקט הפונקציה עצמה (משהו כמו `[object Function]`). הקומפיילר לא יתפוס את זה – זה נראה כמו TypeScript תקין – כך שגיאה עם הסוגריים

היא טעות נפוצה בהתחלה. אם האינטרפולציה שלך מדפיסה `function(){...}()`, שכחת את ה-`()`.

סיגנלים של מערכים ואובייקטים

סיגנל יכול להחזיק כל ערך, כולל מערכים ואובייקטים. אבל סיגנלים עוקבים אחר התייחסויות, לא אחר מצב עמוק. אם אתה משנה מערך במקום, לסיגנל אין מושג ששום דבר השתנה והוא לא יודיע לקוראים שלו.

```
const tasks = signal<Task[]>([]);
```

```
// לא נכון: משנה במקום, בלי הודעה
```

```
tasks().push(newTask);
```

```
// נכון: ייצר מערך חדש
```

```
tasks.update(current => [...current, newTask]);
```

זה אותו דפוס "ייצר אובייקט חדש" שפגשנו בפרק 3, עכשיו עם שם. עדכונים immutable הם איך שסיגנלים עובדים. זה נשמע לא נוח בפעם הראשונה; זה הופך לטבע שני במהירות, וזה גורם לשינויי מצב להיות נראים וניתנים לבדיקה.

computed : ערכים נגזרים

סיגנל computed הוא סיגנל שערכו נגזר מסיגנלים אחרים. הוא מחשב מחדש כשהקלטים שלו משתנים, ושומר את התוצאה במטמון עד שהם משתנים.

```
import { computed } from '@angular/core';

const tasks = signal<Task[]>([]);
const remaining = computed(() => tasks().filter(t => !t.done).length);
const done = computed(() => tasks().length - remaining());
```

אתה קורא computed באותה דרך שאתה קורא כל סיגנל: `remaining()`. אתה אף פעם לא כותב ל-`computed` – הערך שלו הוא פונקציה של הקלטים שלו ואי אפשר להגדיר אותו ישירות.

שני דברים הופכים את `computed` לחזק:

- **מעקב תלויות אוטומטי.** הפריימוורק צופה באילו סיגנלים הפונקציה שלך קוראת. אם הסיגנלים האלה משתנים, ה-`computed` רץ מחדש. אם לא, הוא לא. אתה אף פעם לא רושם תלויות ידנית.
- **הערכה עצלה ומטמון.** ה-`computed` לא רץ עד שמשהו קורא אותו. ברגע שהוא רץ, התוצאה שלו במטמון; קריאות עוקבות מחזירות את המטמון. המטמון מבוטל כשכל תלות משתנה. אם ה-`computed` `progress` שלך תלוי ב- `tasks`, ו- `tasks` לא משתנה, `progress()` לא עולה כלום. לקרוא לו חמישים פעם.

effect : תופעות לוואי בקצה

חלק מהדברים צריכים לקרות כשסיגנלים משתנים, אבל לא מייצרים ערך. שמירה ל- `localStorage`, עדכון כותרת הדף, רישום ניתוחים, קריאה לספרייה שאינה Angular – אלה הן אפקטים.

```
import { effect } from '@angular/core';

class TaskList {
  tasks = signal<Task[]>([]);

  constructor() {
    effect(() => {
      const current = this.tasks();
      localStorage.setItem('compass:tasks', JSON.stringify(current));
    });
  }
}
```

הפונקציה של האפקט רצה פעם אחת מיד (כדי לבסס תלויות) ושוב בכל פעם שכל סיגנל שהיא קוראת משתנה. כאן היא שומרת את רשימת המשימות הנוכחית ל- `localStorage` בכל שינוי.

כללי הדרך לאפקטים:

- **רשום אפקטים בהקשר DI.** בקומפוננטה, זה אומר הקונסטרוקטור או אתחול שדה מחלקה. מחוץ להם, השתמש ב- `runInInjectionContext`.
- **אף פעם אל תכתוב לסיגנל מתוך אפקט.** לעשות זאת יוצר לולאה שבה האפקט מפעיל את עצמו. Angular תסרב להריץ את האפקט אם היא מזהה את הדפוס הזה.

• **פנה קודם ל- computed** . אם אתה מוצא את עצמך כותב אפקט שקורא סיגנלים ומחשב ערך, זה `computed`. אפקטים הם לפלטים לעולם החיצוני.

אפקטים הם הדבר הכי קרוב לרגל-פיתיון שיש לסיגנלים. בשימוש טוב, הם מחליפים `hooks` של מחזור חיים ו-`subscriptions` ל-`observables`. בשימוש רע, הם יוצרים פעולות רפאים מרחוק. פרק 14 חוזר אליהם כשנבנה שכבת מצב מבוססת סיגנלים.

שכתוב Compass עם סיגנלים

זמן להמיר את `TaskList` מפרק 6. פתח את `src/app/task-`
`list/task-list.ts` ושנה אותו ל:

```
import { Component, signal, computed } from '@angular/core';
import { Task } from '../task';

@Component({
  selector: 'app-task-list',
  imports: [],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  tasks = signal<Task[]>([
    /* ... שלושת המשימות ההתחלתיות מפרק 5 ... */
  ]);

  draft = signal("");

  remaining = computed(() =>
    this.tasks().filter(t => !t.done).length
  );

  toggle(taskId: string): void {
    this.tasks.update(current =>
      current.map(t =>
        t.id === taskId ? { ...t, done: !t.done } : t
      )
    );
  }

  add(): void {
    const title = this.draft().trim();
    if (title === "") return;
    this.tasks.update(current => [
      ...current,
      {
        id: crypto.randomUUID(),
        title,
        done: false,
      }
    ]);
  }
}
```

```
        createdAt: new Date().toISOString(),
        dueDate: null,
        tags: [],
      },
    ]);
    this.draft.set("");
  }

  onInput(event: Event): void {
    this.draft.set((event.target as HTMLInputElement).value);
  }
}
```

ואת `task-list.html`:

```
<h1>Tasks – {{ remaining() }} left</h1>

<form (submit)="add(); $event.preventDefault()">
  <input
    type="text"
    placeholder="What needs doing?"
    [value]="draft()"
    (input)="onInput($event)" />
  <button type="submit" [disabled]="draft().trim() === "">Add</button>
</form>

<ul class="task-list">
  @for (task of tasks(); track task.id) {
    <li class="task" [class.done]="task.done" (click)="toggle(task.id)">
      <span class="title">{{ task.title }}</span>
      @if (task.dueDate) {
        <span class="due">due {{ task.dueDate | date: 'shortDate' }}</span>
      }
    </li>
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>
```

שמור, רענן. Compass נראה ומתנהג אותו הדבר כמו קודם. כל שורת קוד חשובה, אבל, שונה:

- `tasks` ו-`draft` הם סיגנלים, נקראים עם `()`.
- `toggle` ו-`add` משתמשים ב-`update` כדי לייצר מערך חדש; שום דבר לא משתנה במקום.
- `remaining` מחושב פעם אחת, ומחושב מחדש רק כש-`tasks` משתנה.

• התבנית קוראת לסיגנלים: `remaining()`, `draft()`, `tasks()`.

מתחת למכסה, Angular עכשיו יודעת בדיוק אילו חלקים של התבנית תלויים באילו סיגנלים. אם נגרום לקומפוננטה הזאת להיות — `changeDetection: ChangeDetectionStrategy.OnPush` — שפרק 16 מסביר ושנחיל גלובלית — Angular תסרב לבדוק שום דבר שלא היה יכול להשתנות. זה מודל הביצועים של אפליקציה מונעת סיגנלים.

Signal inputs

Inputs של קומפוננטה — ערכים שמועברים מהורה — הוכרזו היסטורית עם הדקורטור `@Input()`:

```
export class TaskRow {  
  @Input() task!: Task;  
}
```

Angular מודרנית מציעה חלופה מבוססת סיגנלים: `input()` ו-`input.required()`, שמייצרת סיגנל.

```
import { Component, input } from '@angular/core';

@Component({ /* ... */ })
export class TaskRow {
  task = input.required<Task>();

  // קרא אותו כמו כל סיגנל אחר, computed-במתודות וב
  isOverdue = computed(() => {
    const t = this.task();
    return t.dueDate !== null && new Date(t.dueDate) < new Date();
  });
}
```

שתי צורות שימושיות:

- **input<Type>()** – אופציונלי; ברירת המחדל היא undefined, והסיגנל מחזיר undefined | Type אלא אם תספק ברירת מחדל: **input<number>(0)**.
- **input.required<Type>()** – נדרש; הקומפיילר אוכף שכל הורה יספק ערך.

Signal inputs הם ברירת המחדל המודרנית. הם מתחברים באופן טבעי עם **computed** (כמו למעלה) ועם **effect**. כשפרק 8 יעצב מחדש את Compass לעץ של קומפוננטות, נשתמש ב- **input()** לכל אורכו.

Model inputs: two-way binding, כראוי

`ngModel` הוא דרך אחת לקבל two-way binding על `inputs native`; אחרת היא `[(value)]` על הקומפוננטות שלך, מופעל על ידי `model inputs`.

```
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-quantity-picker',
  template: `
    <button (click)="value.set(value() - 1)">-</button>
    <span>{{ value() }}</span>
    <button (click)="value.set(value() + 1)">+</button>
  `,
})
export class QuantityPicker {
  value = model.required<number>();
}
```

הורה משתמש בו עם התחביר `[(name)]`:

```
<app-quantity-picker [(value)]="count" />
```

`model` נותן לך סיגנל שניתן לכתובה שמפיץ אוטומטית שינויים להורה דרך `output` מותאם. זו החלפה מודרנית לאוג "input + output" שקוד Angular ישן יותר השתמש בו. פרק 8 חוזר ל-`model inputs` לצד `inputs` רגילים.

מתי להשתמש בסיגנלים – ומתי לא

כלל האצבע: סיגנלים למצב שחי בתוך האפליקציה שלך; RxJS לזרמים מהעולם החיצוני.

- **השתמש בסיגנלים ל:** מצב UI (פתוח/סגור, id שנבחר, טיטוט טופס), מצב נגזר (רשימה מסוננת, ספירות מחושבות), חישובים ריאקטיביים, ומצב חוצה שהיית שומר קודם בשירות (פרק 14 מכסה חנויות סיגנלים).
- **השתמש ב-RxJS ל:** תגובות HTTP (שהן באופן טבעי observables), זרמי אירועי DOM שצריכים debounce או switchMap, חיבורי WebSocket, זרמי נתונים ארוכי-טווח.
- **השתמש בשדות רגילים ל:** ערכים שאף פעם לא משתנים לאורך חיי הקומפוננטה, או שלא משפיעים על התבנית. אם כלום לא קורא את זה ריאקטיבית, זה לא צריך להיות סיגנל.

לפנות לסיגנלים בכל מקום היא טעות של מתחיל. לפנות ל-RxJS בכל מקום היא טעות של Angular בקריירה מוקדמת. שניהם עולים בפשטות. פרק 11 מכסה את ה-interop ביניהם (יש toSignal ו-toObservable, ושניהם שתי שורות קוד).

זיהוי שינויים בעולם של סיגנלים

זיהוי שינויים ב-Angular היה פעם מבוסס zone: ספרייה שנקראת Zone.js תיקנה כל API אסינכרוני בדפדפן (טיימרים, Promises, XMLHttpRequest, מאזיני אירועים) כדי ש-Angular תוכל ללמוד מתי כל אחד מהם נורה

ולבדוק מחדש את האפליקציה. זה עובד, אבל זה מברשת רחבה ועולה במידה של CPU בכל אירוע.

Angular מודרנית זזה לעבר אפליקציות ללא zone. באפליקציה ללא zone, Angular בודקת מחדש רק כשסיגנל משתנה. אין בדיקה מחדש סביבתית בכל `setTimeout`; אין `Promise` מוטלא. קישורים שקוראים סיגנלים מדויקים; שום דבר אחר לא רץ.

ברירת המחדל של `ng new` עדיין מפעילה `Zone.js` לתאימות לאחור, אבל הכיוון ברור. כל החלטת עיצוב בספר הזה – סיגנלים למצב, `input()` על פני `@Input()`, `model()` ל-two-way – היא במסלול ללא zone. אם אתה עוקב אחר הדפוסים של הספר הזה, הגירה ללא zone מאוחר יותר תהיה שינוי תצורה, לא שכתוב.

מה בא הלאה

פרק 8 שובר את `TaskList` המונוליתי של `Compass` לעץ של קומפוננטות: רשימה, שורות בודדות, וטופס להוספת משימה. תפגוש הרכבה של קומפוננטות כראוי – `input()`, `output()`, והזרקת תוכן – ועד הסוף יהיה לך `Compass` שמובנה כמו אפליקציית Angular אמיתית, לא כמוכחת קונספט.

תרגילים

1. הוסף סיגנל `computed` `overdueCount` שסופר משימות ש-`dueDate` שלהן בעבר ושהן לא `done`. הצג אותו מתחת לכותרת: "3 overdue". הצג את השורה רק כשיש לפחות משימה אחת שבאיחור.

2. הוסף `effect` שמתעד לקונסולה בכל פעם שרשימת המשימות משתנה, בפורמט `"tasks: 5 (2 done)"`.
אחר כך לחץ מסביב וקרא את הקונסולה. מתי האפקט נורה? האם הוא נורה כשאתה מקליד ב-input? למה או למה לא?

3. הוסף כפתור `"clear completed"`. הוא צריך להיות מושבת כשאין דבר שהושלם. חבר אותו לעדכון הסיגנל `tasks` על ידי סינון של משימות שבוצעו. השתמש ב-`update`, לא ב-`set`; אל תשנה את המערך.

פרק 8. הרכבת קומפוננטות

אפליקציית Angular אמיתית היא לא קומפוננטה גדולה אחת; היא עץ של קטנות. לכל אחת יש עבודה צרה, ויחד הן מתחברות למסך. הרכבה היא מה שגורם לאפליקציה להיות ניתנת לתחזוקה – אתה יכול לשנות שם, להחליף, או להשתמש שוב בכל קומפוננטה בלי לגעת בשאר – והיא מה שמאפשר לקוד לגדול מעבר לכמה אלפי שורות בלי להפוך לביצה.

בפרק הזה אנחנו מפצלים את הקומפוננטה `TaskList` של `Compass` לשלוש: רשימה, שורות משימה בודדות, וטופס להוספת משימה. תוך כדי כך אנחנו מכסים את כלי ההרכבה: `input()` להעברת נתונים למטה, `output()` לשליחת אירועים למעלה, `model()` ל-two-way binding, והזרקת תוכן להכנסת תוכן שרירותי בתוך קומפוננטת `container`.

עד הסוף, `Compass` ייראה ויתנהג בדיוק כמו עכשיו – אבל הוא יהיה מובנה כמו אפליקציות Angular ששורדות מגע עם צוותים אמיתיים.

איך הרכבה נראית

כרגע, `TaskList` הוא הבעלים של הכל: המערך של המשימות, מצב ה-draft של ה-input, ההיגיון של add ו-toggle, וכל התבנית שמרנדרת טופס ושורות. זה בסדר למאה שורות. זה נכשל בעשרת אלפים.

אנחנו הולכים לפצל אותו:

- **TaskRow** – מרנדר משימה אחת. הוא מקבל `task` כ-`input` ופולט אירוע כשהמשתמש מעביר אותה.
- **AddTaskForm** – מרנדר את ה-`input` ואת כפתור `Add`. הוא פולט אירוע כשהמשתמש שולח כותרת חדשה.
- **TaskList** – בעלים של מערך המשימות ומתאם בין שני החלקים. הוא הופך קצר וקריא.

החלוקה הזאת של "container / הצגה" היא הדרך הכי נפוצה להבנות קומפוננטות Angular. ה-`container` יודע על מצב ומתאם את החלקים; קומפוננטות ההצגה יודעות רק על מה שהן מרנדרות ואילו אירועים הן פולטות. קומפוננטות הצגה נוטות להיות קטנות, טהורות, קלות לבדיקה, וקלות לשימוש חוזר.

העברת נתונים למטה: `input()`

פגשנו בקצרה את `input()` בפרק 7. כאן אנחנו משתמשים בו ברצינות.

צור את `TaskRow` עם ה-`CLI`:

```
ng generate component task-row
```

ערוך את `src/app/task-row/task-row.ts`:

```

import { Component, input, output, computed } from '@angular/core';
import { Task } from '../task';

@Component({
  selector: 'app-task-row',
  imports: [],
  templateUrl: './task-row.html',
  styleUrls: ['./task-row.css'],
})
export class TaskRow {
  task = input.required<Task>();
  toggled = output<string>();

  isOverdue = computed(() => {
    const t = this.task();
    return t.dueDate !== null && !t.done && new Date(t.dueDate) < new Date();
  });

  onToggle(): void {
    this.toggled.emit(this.task().id);
  }
}

```

שלושה דברים לשים לב:

- `task = input.required<Task>()` מכריז שהקומפוננטה הזאת מקבלת `input` נדרש מסוג `Task`. אם הורה שוכח להעביר אחד, הקומפיילר של התבנית מתלונן.
- `toggled = output<string>()` מכריז על אירוע שהקומפוננטה הזאת יכולה לפלוט. פרמטר הטיפוס הוא הטיפוס של ה-payload — אנחנו פולטים את ה-id של המשימה.

• **isOverdue** הוא סיגנל **computed** שתלוי ב- `task()`. כי

`task` הוא סיגנל בעצמו, בכל פעם שההורה מעביר ערך חדש, `isOverdue` מחושב מחדש.

התבנית, ב- `task-row.html`:

```
<li class="task" [class.done]="task().done" [class.overdue]="isOverdue">
  <button class="toggle" (click)="onToggle()">
    <span class="title">{{ task().title }}</span>
    @if (task().dueDate) {
      <span class="due">due {{ task().dueDate | date: 'shortDate' }}</span>
    }
    @if (isOverdue()) {
      <span class="overdue-badge">overdue</span>
    }
  </button>
</li>
```

שים לב ל- `task().title` – אתה קורא `signal input` בדיוק כמו כל סיגנל אחר, עם `()`. התבנית חוזרת על `task()` כמה פעמים; זו קריאה זולה (סיגנלים במטמון), ויותר נקייה מלהקצות למשתנה מקומי קודם.

וגם `task-row.css`:

```

:host { display: block; }

.task { padding: 0; margin: 0; border-bottom: 1px solid #eee; }
.task.done .title { text-decoration: line-through; color: #999; }
.task.overdue .title { color: #c73a52; }

.toggle {
  width: 100%; text-align: left; background: none; border: 0;
  padding: 12px 14px; display: flex; gap: 12px; align-items: center;
  cursor: pointer; font: inherit; color: inherit;
}

.due { margin-left: auto; font-size: 0.85rem; color: #666; }
.overdue-badge {
  font-size: 0.75rem; padding: 2px 6px; border-radius: 4px;
  background: #ffe3e3; color: #c73a52; margin-left: 8px;
}

```

השורה עכשיו היא דבר עצמאי שאתה יכול להשליט לכל מקום.

שליחת אירועים למעלה: output()

output() מייצר משהו עם מתודת emit(value). קריאה לה יורה את האירוע על הקומפוננטה; ההורה מקשיב עם התחביר . (name)="handler(\$event)"

כבר כתבנו את this.toggled.emit(this.task().id) בתוך TaskRow. הורה ירשם עם:

```
<app-task-row [task]="t" (toggled)="toggle($event)" />
```

`$event` ב-handler הוא מה שהועבר ל- `emit(...)` – כאן, מחרוזת ה-id של המשימה. TypeScript זורם דרך הטיפוס, כך שהמתודה `toggle` של ההורה תקבל `string` מוקלד.

`output()` מחליף את הדפוס הישן יותר של `@Output()` `eventEmitter = new EventEmitter<T>()`. שניהם עדיין עובדים; קוד חדש משתמש ב- `output()`.

הוספת הטופס: `AddTaskForm`

צור אותו:

```
ng generate component add-task-form
```

ערוך את `src/app/add-task-form/add-task-form.ts`:

```

import { Component, signal, output } from '@angular/core';

@Component({
  selector: 'app-add-task-form',
  imports: [],
  templateUrl: './add-task-form.html',
  styleUrls: ['./add-task-form.css'],
})
export class AddTaskForm {
  draft = signal("");
  add = output<string>();

  submit(): void {
    const title = this.draft().trim();
    if (title === "") return;
    this.add.emit(title);
    this.draft.set("");
  }

  onInput(event: Event): void {
    this.draft.set((event.target as HTMLInputElement).value);
  }
}

```

תבנית : add-task-form.html

```

<form (submit)="submit(); $event.preventDefault()">
  <input
    type="text"
    placeholder="What needs doing?"
    [value]="draft()"
    (input)="onInput($event)" />
  <button type="submit" [disabled]="draft().trim() === "">Add</button>
</form>

```

סגנון, `add-task-form.css`:

```
:host { display: block; margin-bottom: 1em; }
form { display: flex; gap: 8px; }
input { flex: 1; padding: 10px; font: inherit; border: 1px solid #ccc; border-radius: 6px; }
button {
  padding: 10px 16px; font: inherit; border: 0; border-radius: 6px;
  background: #7a3fbf; color: white; cursor: pointer;
}
button:disabled { background: #ccc; cursor: not-allowed; }
```

`AddTaskForm` בעלים של מצב ה-draft שלו (הקלדת המשתמש בתהליך). הוא פולט אירוע `add` עם הכותרת הסופית כשהמשתמש שולח. ההורה לא צריך לדעת או להיות אכפת לו איך הטופס עובד – הוא מתחבר לשני חוזים: הפלט `add`, והתג `<app-add-task-form>`.

TaskList המעוצב מחדש

עכשיו `TaskList` הופך קצר. החלף את המחלקה שלו ב:

```

import { Component, signal, computed } from '@angular/core';
import { Task } from '../task';
import { TaskRow } from '../task-row/task-row';
import { AddTaskForm } from '../add-task-form/add-task-form';

@Component({
  selector: 'app-task-list',
  imports: [TaskRow, AddTaskForm],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  tasks = signal<Task[]>([
    /* משימות התחלתיות ללא שינוי מפרק 7 */
  ]);

  remaining = computed(() => this.tasks().filter(t => !t.done).length);

  onAdd(title: string): void {
    this.tasks.update(current => [
      ...current,
      {
        id: crypto.randomUUID(),
        title,
        done: false,
        createdAt: new Date().toISOString(),
        dueDate: null,
        tags: [],
      },
    ]);
  }

  onToggle(id: string): void {
    this.tasks.update(current =>
      current.map(t => (t.id === id ? { ...t, done: !t.done } : t))
    );
  }
}

```

```
}  
}
```

והתבנית `task-list.html`:

```
<h1>Tasks – {{ remaining() }} left</h1>  
  
<app-add-task-form (add)="onAdd($event)" />  
  
<ul class="task-list">  
  @for (task of tasks(); track task.id) {  
    <app-task-row [task]="task" (toggled)="onToggle($event)" />  
  } @empty {  
    <li class="empty">Nothing to do. Nice.</li>  
  }  
</ul>
```

שמור. האפליקציה נראית זהה. כל פיסת התנהגות ללא שינוי. אבל עכשיו:

- ניתן להשתמש שוב ב- `TaskRow` בכל מקום שמשימה צריכה לרנדור.
- ניתן להשתמש שוב ב- `AddTaskForm` בכל מקום שאתה צריך לאסוף כותרת.
- `TaskList` הוא חמש-עשרה שורות של תיאום.

זו הצורה של אפליקציית Angular.

Two-way עם model()

`model()` נותן לך זוג input-output עם קישור `[(name)]`, ארוזים כסינגל אחד. הוא שימושי כשילד הוא בעלים של חלק מפיסת מצב משותפת – המקרה הקלאסי הוא input טקסט כמו זה שרק בנינו.

נניח שרצינו ש-`AddTaskForm` יחשוף את ה-`draft` שלו להורה לתצוגה חיצונית. יכולנו לעשות את זה ידנית עם `output`; `model()` עושה את זה בשורה אחת.

שכתב את `add-task-form.ts`:

```
import { Component, model, output } from '@angular/core';

@Component({ /* ... */ })
export class AddTaskForm {
  draft = model("");
  add = output<string>();

  submit(): void {
    const title = this.draft().trim();
    if (title === "") return;
    this.add.emit(title);
    this.draft.set("");
  }

  onInput(event: Event): void {
    this.draft.set((event.target as HTMLInputElement).value);
  }
}
```

עכשיו הורה יכול לעשות:

```
<app-add-task-form [(draft)]="currentDraft" (add)="onAdd($event)" />
```

כל שינוי מכל צד משתקף בשני. אם ההורה לא אכפת לו מה-draft, הוא יכול פשוט להעביר handler של (add) ולדלג על [(draft)]. model() נותן לך אופציונליות שזוג input/output שנעשה ידנית לא נותן.

לעת עתה, TaskList לא צריך לראות את ה-draft – התבנית הנוכחית משמיטה [(draft)] והכל עובד. שקול model() כשאתה מוצא את עצמך כותב זוגות של inputs ו-outputs ששמותיהם רק היו מוסיפים "Change" אחד לשני.

הזרקת תוכן: <ng-content>

חלק מהקומפוננטות לא מוגדרות על ידי הנתונים שהן מקבלות, אלא על ידי התוכן שהן עוטפות. חלון מודאלי, כרטיס, סעיף מתקפל – כל אחד הוא מכל שהפנים שלו ממולא על ידי המזמין שלו. הכלי של Angular לזה הוא הזרקת תוכן, דרך האלמנט <ng-content>.

הנה קומפוננטת Card :

```

@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <h2 class="card-title">{{ title() }}</h2>
      <div class="card-body">
        <ng-content></ng-content>
      </div>
    </div>
  `,
  styles: `
    .card { border: 1px solid #ddd; border-radius: 8px; padding: 10px; }
    .card-title { margin: 0 0 8px 0; font-size: 1.1rem; }
  `,
})
export class Card {
  title = input.required<string>();
}

```

הורה משתמש בו כך:

```

<app-card title="Today">
  <p>Three tasks remaining.</p>
  <button>Refresh</button>
</app-card>

```

מה שיופיע בין `<app-card>` ו-`</app-card>` בתבנית של ההורה מסתיים איפה ש-`<ng-content>` יושב בתבנית של הילד. זה מאפשר לקומפוננטה אחת להגדיר פריסה בזמן שהמזמינים ממלאים תוכן.

חריצים בשמות

אם ל-container יש מספר נקודות הכנסה, השתמש ב-attribute `select`:

```
<!-- בתוך SplitCard -->
<div class="left">
  <ng-content select="[left]"></ng-content>
</div>
<div class="right">
  <ng-content select="[right]"></ng-content>
</div>
<div class="footer">
  <ng-content select="footer"></ng-content>
</div>
```

ההורה מתייג כל פיסה של תוכן כדי לנתב אותה לחריץ:

```
<app-split-card>
  <p left>Left column content.</p>
  <p right>Right column content.</p>
  <footer>Footer content.</footer>
</app-split-card>
```

תוכן שלא תואם לאף `select` הולך ל- `<ng-content>` רגיל אם יש אחד, או נזרק אחרת.

הזרקת תוכן היא איך שאתה בונה מערכות עיצוב: כרטיסים, דיאלוגים, טאבים, wrappers של טבלאות. Compass ישתמש בה בפרק 15 כשנציג קומפוננטת "מצב ריק" משותפת.

גישה לילדים: `viewChild` ו-`viewChildren`

לפעמים הורה צריך להושיט יד לתוך ילד – למקד `input`, להפעיל מתודה, לקרוא פיסת מצב שלא נחשפת דרך `Angular` `output`. מציעה `viewChild()` ו-`viewChildren()` לזה.

```
import { Component, viewChild, ElementRef, AfterViewInit } from '@angular/core';

@Component({
  selector: 'app-search',
  template: `<input #searchInput type="text" />`,
})
export class Search implements AfterViewInit {
  input = viewChild<ElementRef<HTMLInputElement>>('searchInput');

  ngAfterViewInit() {
    this.input()?.nativeElement.focus();
  }
}
```

`viewChild('searchInput')` מחזיר סיגנל שהערך שלו הוא כל אלמנט או קומפוננטה שתאם ל-`reference` התבנית `#searchInput`. אתה קורא אותו עם `()` כמו כל סיגנל; הוא עשוי להיות `undefined` אם האלמנט עדיין לא מרונדר.

`viewChild` הוא מוצא אחרון. אם אתה מוצא את עצמך פונה אליו, שאל קודם: האם זה יכול להיעשות עם `input`, `output` או סיגנל? תשע פעמים מתוך עשר, זה יכול. המקרה של אחת מתוך עשר בדרך כלל כולל APIs אימפרטיביים של דפדפן (`focus`, `scroll`), נגינת `<video>`.

הנחיות לפיצול קומפוננטות

מתי לפצל קומפוננטה אחת לשתיים? סימנים:

- התבנית ארוכה יותר ממסך. אם אתה גולל כדי לקרוא אותה, יהיה לך קושי לתחזק אותה.
- שני סעיפים ויזואליים נבדלים אין להם מה לעשות אחד עם השני. "כותרת" ו"רשימה" באותה קומפוננטה כמעט תמיד צריכים להיות קומפוננטות נפרדות.
- אתה עומד להעתיק-להדביק חתיכה של מרקאפ לשימוש שני. חלץ קודם.
- למחלקה יש יותר מכעשרה שדות. חלקם כנראה על תת-דאגה.

מתי לא לפצל? כשהחלקים חסרי משמעות בפני עצמם. "כותרת של שורת משימה" היא לא קומפוננטה; היא `<span>`. "שורת משימה שיש לה גם כפתור מחיקה ותג באיחור" היא קומפוננטה. המבחן: האם אתה יכול לדמיין שימוש בקומפוננטה הזאת במקום אחר, או בדיקה שלה בבידוד? אם כן, חלץ. אם לא, השאר.

מה בא הלאה

חלק שלישי מתחיל בפרק 9 על הזרקת תלויות – המנגנון שיאפשר לנו למשוך את נתוני המשימות מ- `TaskList` לתוך שירות שקומפוננטות אחרות יכולות לשתף. פרק 10 מחבר את השירות הזה לבקאנד HTTP; פרק 11 מציג RxJS לחלקים של האפליקציה שבהם סיגנלים לא

מספיקים; פרק 12 מחליף את הטופס שנעשה ידנית בטפסים ריאקטיביים.

עד סוף חלק שלישי, Compass ידבר עם שרת אמיתית, ישמור את המשימות שאתה מוסיף, ויפסיק לאבד את העבודה שלך כשאתה מרענן.

תרגילים

1. חלץ קומפוננטה `TaskCount` שמציגה `"tasks left 3"` ומקבלת `total` ו-`done` כ-`inputs`. חשב את המספר `"left"` בתוך הקומפוננטה עם `computed`. השתמש בה ב-`TaskList` במקום הטקסט הנוכחי של הכותרת.

2. הוסף כפתור מחיקה ל-`TaskRow`. הוא צריך לפלוט `output deleted` עם ה-`id` של המשימה. חבר את `TaskList` להסיר משימות באירוע הזה. אל תיתן ללחיצה על כפתור המחיקה גם לירות את ה-`toggle`; השתמש ב-`$event.stopPropagation()` ב-`handler` של הלחיצה.

3. בנה קומפוננטה `Panel` עם הזרקת תוכן: `container` עם כותרת, `body` חריץ, ו`actions` אופציונלי. התבנית שלו צריכה להיות בערך

```
<div class="panel">
  <header>{{ title() }} <ng-content
  <div select="[actions]" /></header>
</div> <div class="body"><ng-content /></div>
```

. השתמש בה כדי לעטוף את רשימת המשימות – העבר

"Today" כותרת וכפתור "Clear completed" בחריץ
ה-actions.

פרק 9. הזרקת תלויות, כוח העל של Angular

הגענו לגבול. Compass שומר את המשימות שלו בתוך `TaskList`. פירוש הדבר שהמשימות קיימות רק בזמן ש-`TaskList` על המסך; כשנתחיל להוסיף עמוד סטטיסטיקות, מונה בסרגל צד, או ניתוב בין מסכים, המצב יעלם בכל ניווט. אנחנו צריכים להעביר את המשימות מהקומפוננטה למשהו שכל האפליקציה תוכל לשתף.

ה"משהו" הזה הוא שירות. שירותים ב-Angular הם מחלקות TypeScript רגילות עם מאפיין מיוחד אחד: הן יכולות להיות מוזרקות. קומפוננטה מבקשת שירות, ו-Angular מספקת מופע — אותו מופע לכל מבקש באותו scope — בלי שהקומפוננטה תדע איך השירות נוצר או מי עוד משתמש בו. המנגנון הזה נקרא הזרקת תלויות (DI), וזו אחת הסיבות שאפליקציות Angular נשארות ניתנות לתחזוקה בקנה מידה.

הפרק הזה מציג DI, מעצב מחדש את מצב המשימות של Compass לשירות `TaskStore`, ונותן לך את אוצר המילים שאתה צריך לכל פרק עוקב. כל חלק של Angular שתפגוש מכאן והלאה — HTTP, ראוטר, טפסים, בדיקות, `guards`, `resolvers` — מגיע לקומפוננטה שלך דרך DI.

למה "הזרקה"?

דמיין את `TaskList` שצריך לדבר עם שרת. בלי DI, זה היה נראה כך:

```
class TaskList {  
  taskService = new TaskService(new HttpClient(new HttpBackend(new ...)))  
}
```

כל קומפוננטה שמשתמשת ב-`TaskService` הייתה צריכה לבנות את כל השרשרת. גרוע יותר, בבדיקות היית צריך לבנות מחדש את השרשרת עם `mocks`. זה הופך למוסרבל מהר.

DI הופך את הסידור. הפריימוורק מתחזק רישום (injector) של שירותים. כשהקומפוננטה שלך אומרת "אני צריכה `TaskService`", Angular מחפשת ברישום, בונה אחד אם נדרש (משתמשת ב-DI שלה עצמה כדי לספק את התלויות של השירות), שומרת אותו במטמון, ומחזירה אותו. הקומפוננטה שלך מקבלת את המוצר המוגמר בלי לדעת מה נכנס אליו.

לזה יש שלוש השלכות שחשובות לנו:

- **קומפוננטות נשארות קטנות.** הן לא בונות את העולם; הן מבקשות מה שהן צריכות.
- **מופעים משותפים כראוי.** כברירת מחדל, שירות שנרשם בשורש יש לו מופע אחד משותף לרוחב כל האפליקציה. שתי קומפוננטות שמבקשות `TaskService` מקבלות את אותו `TaskService`, וכל מצב עליו נראה לשתייהן.

- **בדיקות נהיות קלות.** בבדיקות אתה יכול לרשום `TaskService` מזויף לפני יצירת הקומפוננטה; הקומפוננטה לא יודעת ולא אכפת לה.

יצירת שירות

ל-CLI של Angular יש מייצר:

```
ng generate service task-store
```

זה מייצר `src/app/task-store.ts` (או `task-store.service.ts` בגרסאות CLI ישנות יותר) בערך כך:

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class TaskStore {}
```

שני מרכיבים:

- **@Injectable** הוא דקורטור שמסמן את המחלקה כזמינה ל-DI. הוא גם מאפשר למחלקה לקבל תלויות דרך DI בעצמה.
- **providedIn: 'root'** אומר ל-Angular לרשום את השירות ב-injector השורשי – הרמה העליונה של האפליקציה. פירוש הדבר מופע אחד משותף לכל האפליקציה. כל קומפוננטה שמזריקה `TaskStore` מקבלת את אותו מופע.

לרוב השירותים, `providedIn: 'root'` הוא בדיוק מה שאתה רוצה. זו ברירת המחדל המודרנית, זה `"tree-shakes"` (השירות נופל מה-bundle אם כלום לא משתמש בו), ואין לזה חסרונות לרוב המכריע של המקרים. פנה לחלופות (פרק 13 מכסה וריאנט `scoped` למסלול) רק כשיש לך סיבה ספציפית.

העברת המשימות של Compass לתוך TaskStore

ערוך את `src/app/task-store.ts`:

```
import { Injectable, signal, computed } from '@angular/core';
import { Task } from './task';

@Injectable({ providedIn: 'root' })
export class TaskStore {
  private readonly _tasks = signal<Task[]>([
    { id: 't1', title: 'Buy milk', done: false, createdAt: '2026-01-15T09:00:00Z' },
    { id: 't2', title: 'Write chapter 9', done: true, createdAt: '2026-01-15T09:00:00Z' },
    { id: 't3', title: 'Call the plumber', done: false, createdAt: '2026-01-15T09:00:00Z' }
  ]);

  readonly tasks = this._tasks.asReadonly();
  readonly remaining = computed(() => this._tasks().filter(t => !t.done).length);
  readonly done = computed(() => this._tasks().filter(t => t.done).length);

  add(title: string): void {
    this._tasks.update(current => [
      ...current,
      {
        id: crypto.randomUUID(),
        title,
        done: false,
        createdAt: new Date().toISOString(),
        dueDate: null,
        tags: [],
      },
    ]);
  }

  toggle(id: string): void {
    this._tasks.update(current =>
      current.map(t => (t.id === id ? { ...t, done: !t.done } : t))
    );
  }

  remove(id: string): void {
    this._tasks.update(current => current.filter(t => t.id !== id));
  }
}
```

```
}  
  
clearCompleted(): void {  
  this._tasks.update(current => current.filter(t => !t.done));  
}  
}
```

שלושה דפוסים שכדאי לתת להם שם:

- **סיגנל פרטי לכתובה, תצוגת readonly ציבורית.** `_tasks`

הוא פרטי וניתן לשינוי; `tasks` הוא הסיגנל הציבורי, לקריאה

בלבד שצרכנים רואים. זה מונע ממקוראים לעשות

`this.store.tasks.set(...)` ולעקוף את ה-API.

- **התנהגות על פני נתונים.** השירות חושף מתודות שמתארות

כיונה — `add`, `toggle`, `remove`, `clearCompleted` צרכנים לא יודעים איך משימות מאוחסנות, רק מה הם יכולים לעשות איתן.

- **מצב נגזר** `computed`. `remaining` ו-`done` חיים כאן, לא

בקומפוננטות. כל קומפוננטה יכולה לקרוא אותם;

כשמשמיות משתנות, כל הקוראים מתעדכנים.

הזרקת השירות לקומפוננטה

שתי צורות של הזרקת עובדות ב-Angular מודרנית. הפונקציה

`inject()` היא הצורה החדשה יותר, המועדפת; הזרקת

קונסטרוקטור היא הצורה הישנה יותר, שעדיין נתמכת. אנחנו

משתמשים ב-`inject()` לרוחב הספר.

ערוך את `src/app/task-list/task-list.ts`

```
import { Component, inject } from '@angular/core';
import { TaskStore } from '../task-store';
import { TaskRow } from '../task-row/task-row';
import { AddTaskForm } from '../add-task-form/add-task-form';

@Component({
  selector: 'app-task-list',
  imports: [TaskRow, AddTaskForm],
  templateUrl: './task-list.html',
  styleUrls: ['./task-list.css'],
})
export class TaskList {
  private store = inject(TaskStore);

  tasks = this.store.tasks;
  remaining = this.store.remaining;

  onAdd(title: string): void {
    this.store.add(title);
  }

  onToggle(id: string): void {
    this.store.toggle(id);
  }
}
```

הקומפוננטה עכשיו בעצם מתרגמת בין ה-DOM לבין השירות. כל המצב והלוגיקה שלה עברו ל-`TaskStore`. אם נרצה את אותם הנתונים בקומפוננטה אחרת – סרגל צד שמראה `remaining`, נגיד – אנחנו פשוט מזריקים `TaskStore` גם שם וקוראים `store.remaining`. אין מה לחבר דרך העץ.

התבנית לא צריכה להשתנות; `tasks` ו-`remaining` הם עדיין סיגנלים עם אותו ממשק.

שמור. האפליקציה מתנהגת באופן זהה. המשימות עכשיו חיות מחוץ לקומפוננטה, וקומפוננטות אחרות היו יכולות לשתף אותן אילו היו קיימות.

הפונקציה `inject()`

`inject(Class)` מחזירה מופע של `Class` מה-`injector` הנוכחי. אתה יכול לקרוא לה:

- באתחול שדה או בקונסטרוקטור של קומפוננטה (כמו למעלה).
- באתחול שדה או בקונסטרוקטור של שירות.
- בתוך פונקציות שנקראות מההקשרים האלה, בטרנזיטיביות.

אתה לא יכול לקרוא ל-`inject()` מפונקציה אקראית שלא בהקשר DI. Angular זורקת שגיאת זמן ריצה. אם אתה צריך להזריק מחוץ להקשר טבעי, עטוף את הקריאה ב-`runInInjectionContext(injector, () => { ... })` אבל זה נדיר בקוד יישום – תיתקל בזה אולי פעם בשנה.

ל-`inject()` יש גם צורות גנריות לאופציונליות ולמקרים של `:multi-token`:

```
private optionalThing = inject(TokenClass, { optional: true }); // להיות  
private allOfThem = inject(TokenClass, { self: true }); // inject-רק ה
```

לא נצטרך את אלה זמן מה. `inject(Something)` רגיל מכסה 95% מהשימושים.

Providers: איך שירותים נכנסים ל-injector

`providedIn: 'root'` היא דרך אחת לרשום שירות. יש אחרות, ותפגוש אותן אחת אחת.

Providers ברמת האפליקציה דרך `provideX(...)`

חלק מהשירותים מוגדרים עם נתונים; אתה לא יכול פשוט לרשום אותם עם דקורטור. הקונבנציה של Angular היא פונקציית `provide<Feature>(config)` שאתה מציב ב-`providers` ב-`app.config.ts`:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(),
    provideAnimations(),
  ],
};
```

`provideRouter`, `provideHttpClient`, וחבריהם הם איך שהפיצ'רים ה-opt-in של Angular מגיעים לאפליקציה שלך. כל פונקציה מחזירה סט של רישומי Angular provider; קוראת אותם ב-bootstrap.

Providers ברמת הקומפוננטה

קומפוננטה יכולה לרשום שירות `scoped` לעצמה ולילדיה:

```
@Component({
  selector: 'app-editor',
  providers: [EditorState],
})
export class Editor { }
```

עכשיו כל התייחסות ל- `EditorState` בתוך `Editor` (ב- `Editor` עצמו ובכל קומפוננטת ילד) פותרת לאותו מופע; התייחסות ל- `EditorState` מחוץ ל- `Editor` פותרת למופע אחר (או נכשלת אם אין `provider` שורשי).

זה שימושי למצב שהוא מקומי למסד: מצב שלב של אשף, טיוטת טופס של מודאל, בחירות מסנן של עמוד. נשתמש בזה בפרק 13.

Providers של ערכים ו-factory

לפעמים מה שאתה רוצה להזריק הוא לא מחלקה, אלא ערך או תוצאת פונקציה. ל-Angular יש תחביר גם לאלה:

```
providers: [
  { provide: 'API_BASE_URL', useValue: 'https://api.compass.example' },
  { provide: TaskLoader, useFactory: () => new TaskLoader(navigator.onLi
  ]
```

`InjectionToken<T>` היא הדרך הבטוחה מבחינת טיפוסים לזהות `provider` שאינו מחלקה:

```
import { InjectionToken } from '@angular/core';

export const API_BASE_URL = new InjectionToken<string>('API_BASE_URL');

// providers:
{ provide: API_BASE_URL, useValue: 'https://api.compass.example' }

// הזרקה:
private apiUrl = inject(API_BASE_URL);
```

נראה `InjectionToken` בקוד של Angular עצמה ובספריות. בקוד יישום, מחלקות רגילות עם `providedIn: 'root'` מכסות את רוב הצרכים.

Injectors היררכיים

ה-`injectors` של Angular מרכיבים עץ שעוקב אחר עץ הקומפוננטות. כשאתה `inject(X)` בקומפוננטה, Angular הולכת למעלה מה-`injector` של הקומפוננטה הזאת ומחפשת `provider` ל-`X`. היא משתמשת בראשון שהיא מוצאת.

זו הסיבה ש-`providers: [EditorState]` ברמת הקומפוננטה נותן לכל צאצא מופע משותף, בעוד שאותו קוד במערך `providers` של קומפוננטה אחות נותן לאחות מופע שונה. החיפוש מוצא את ה-`provider` הקרוב ביותר.

לעתים רחוקות תחשוב על זה במפורש. זה חשוב כש:

- יש לך שירות שאתה רוצה `scoped` לאזור פיצ'ר ספציפי (אשף, מודאל), לא לכל האפליקציה.

- אתה צריך לעקוף שירות בבדיקות (פרק 17).
- אתה רוצה שני מופעים של אותה מחלקת שירות לשתי מטרות שונות.

לכל השאר, `providedIn: 'root'` ושכח מזה.

תלויות מעגליות

שני שירותים שמזריקים זה את זה יוצרים מעגל. Angular מסרבת לבנות אותם וזורקת בזמן ריצה. אם אתה נתקל בזה, זה בדרך כלל אומר שיש לשירותים בעיית עיצוב: הם צריכים להתמזג, או שאחד צריך להחזיק את השני אבל לא להיפך, או שהמצב המשותף צריך להיות מחולץ לשירות שלישי ששניהם תלויים בו.

יש מעקף – הזרקת `Injector` וחיפוש עצל – אבל אתה לא צריך אותו. כשאתה מרגיש את המשיכה אליו, קח את זה כסימן לעצב מחדש.

תועלת בבדיקות, בתצוגה מקדימה

כי הכל שקומפוננטה משתמשת בו נכנס דרך `DI`, בדיקות יכולות להחליף את התלויות האלה:

```
TestBed.configureTestingModule({
  providers: [
    { provide: TaskStore, useValue: fakeStore },
  ],
});
```

ה- `inject(TaskStore)` של הקומפוננטה עכשיו מחזיר `fakeStore`. אין `monkey-patching`, אין ייבואים להסיט. פרק 17 פורק את זה בפירוט; לעת עתה, דע שהסיבה שבדיקות Compass שלך יהיו קלות מאוחר יותר היא כי עיצבת אותו סביב DI עכשיו.

מה בא הלאה

פרק 10 מחבר את `TaskStore` לבקאנד HTTP אמיתי. נציג את `HttpClient` של Angular, נכתוב פונקציות לשלוף, ליצור, לעדכן, ולמחוק משימות, ונחבר אותן לחנות. עד סוף פרק 10, ריענון הדפדפן ישמור את המשימות שלך – כי הן חיות בשרת, לא בזיכרון.

תרגילים

1. צור קומפוננטה שנייה `TaskCountBadge` שמראה רק את הספירה הנוטרת כגלולה קטנה. הזרק את `TaskStore` בתוכה ורנדר `{{ store.remaining() }}`. הצב אותה ליד הכותרת ב-`TaskList`. שים לב ששום נתונים לא זורמים דרך `props` – שתי הקומפוננטות חולקות מצב דרך השירות.

2. הוסף שירות `HabitStore` שנבנה באותה דרך כמו `TaskStore`. הוא צריך להחזיק סיגנל של הרגלים (המצא `interface Habit`), לחשוף `habits` לקריאה בלבד, ולספק `add`, `remove`, ו-`check`. אל תחבר אותו עדיין לאף קומפוננטה; רק תוודא שהוא מדור.

3. רשום את `TaskStore` ברמת הקומפוננטה במקום `providedIn: 'root'`. מה נשבר? החזר את זה, וחשוב למה: איזה מאפיין של מצב משותף תלוי איפה השירות רשום?

פרק 10. HTTP: לדבר עם APIs אמיתיים

Compass רץ עד עכשיו לגמרי בזיכרון של הדפדפן. זה מספיק כדי להדגים את האפליקציה, ולא ממש מספיק לשימוש אמיתי. בפרק הזה נציג את שירות `HttpClient` של `Angular`, נחבר את `Compass` לבקאנד קטן, ונאפשר להוסיף משימה במכשיר אחד ולראות אותה במכשיר אחר. עד הסוף, המשימות שאתה יוצר ישרדו ריענון של הדפדפן – כי הן חיות בשרת, לא בסיגנל בזיכרון של `TaskStore`.

לפרק יש שני חצאים. החצי הראשון מרים בקאנד מינימלי כדי שיהיה לנו עם מה לדבר. החצי השני מחבר את `HttpClient` ל-`TaskStore` ומכסה `interceptors`, טיפול בשגיאות, ואת ה-`httpResource` המודרני.

בקאנד זעיר ל-Compass

אנחנו צריכים שרת. כדי לשמור על הספר עצמאי, נשתמש ב-`json-server` – שרת REST ללא תצורה שקורא קובץ `JSON`. הוא חושף אוטומטית endpoints של `GET/POST/PUT/PATCH/DELETE`. בהפקה `Compass` היה מדבר עם בקאנד אמיתי (`Node`, `Python`, `Go`, כל אחד); הקוד של `Angular` זהה.

התקן אותו גלובלית:

```
npm install --global json-server
```

צור קובץ `compass/db.json` (ברמה העליונה של פרויקט Angular שלך, ליד `package.json`):

```
{
  "tasks": [
    { "id": "t1", "title": "Buy milk", "done": false, "createdAt": "2026-01-01" },
    { "id": "t2", "title": "Write chapter 10", "done": true, "createdAt": "2026-01-01" }
  ]
}
```

הרץ אותו במסוף שני:

```
json-server --watch db.json --port 3000
```

עכשיו `http://localhost:3000/tasks` מחזיר את מערך המשימות. עשה לו `curl`, או פתח את ה-URL בדפדפן, כדי לאשר.

`json-server` הוא שרת מזויף, וההתמדה שלו טריוויאלית – הוא כותב שינויים בחזרה ל-`db.json` – אבל ה-API שלו תואם לכל שרת REST JSON שאי פעם תקרא אליו. זו הנקודה.

הגדרת CORS היא לא הבעיה שלנו

כי אנחנו מריצים את Compass ב-`http://localhost:4200` ואת ה-API ב-`http://localhost:3000`, דפדפנים היו בדרך כלל חוסמים את הבקשות כ-`cross-origin`. `json-server` מאפשר אותן כברירת מחדל; API אמיתי בהפקה היה שולח כותרות `Access-Control-Allow-Origin`. בבקאנדים העתידיים שלך, זכור זאת:

הדפדפן מקפיד על בקשות cross-origin, ותצורה שגויה של CORS היא הסיבה שדברים "פשוט לא עובדים" בתשעים אחוזים מהמקרים כשמחברים frontend חדש ל-backend חדש.

הפעלת HttpClient ב-Angular

HttpClient הוא שירות ה-HTTP המובנה של Angular. הוא מגיע דרך DI, וצריך להיות מוגדר ברמת האפליקציה. ערוך את `src/app/app.config.ts`

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withFetch } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withFetch()),
  ],
};
```

`provideHttpClient()` רושם את השירות. `withFetch()` אומר ל-Angular להשתמש ב-API של `fetch` הנייטבי של הדפדפן מתחת למכסה; המימוש הישן יותר מבוסס XHR עדיין עובד, אבל `fetch` הוא כיוון ההתקדמות ומתחבר טוב יותר עם SSR (פרק 18).

חזרה ה-API

הגדר

מודול קטן שמתאר את ה-URLs ועושה את ה-HTTP הגולמי. צור

:src/app/tasks-api.ts

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { firstValueFrom } from 'rxjs';
import { Task } from './task';

const BASE_URL = 'http://localhost:3000';

@Injectable({ providedIn: 'root' })
export class TasksApi {
  private http = inject(HttpClient);

  list(): Promise<Task[]> {
    return firstValueFrom(this.http.get<Task[]>(`${BASE_URL}/tasks`));
  }

  create(task: Omit<Task, 'id'>): Promise<Task> {
    return firstValueFrom(this.http.post<Task>(`${BASE_URL}/tasks`, ta
  )

  update(id: string, changes: Partial<Task>): Promise<Task> {
    return firstValueFrom(this.http.patch<Task>(`${BASE_URL}/tasks/${
  }

  remove(id: string): Promise<void> {
    return firstValueFrom(this.http.delete<void>(`${BASE_URL}/tasks/$
  }
}
```

שלושה פרטים שתפגוש שוב:

- **פרמטרי טיפוס על מתודות ה-HTTP.**

`this.http.get<Task[]>(url)` אומר לקומפיילר שגוף התגובה הוא מערך של `Task`. `Angular` לא מאמתת שהשרת באמת החזיר את הצורה הזאת; זו טענה שאתה מוסיף. בפרק 17 נוסיף בדיקות שבודקות שהצורה של השרת תואמת לטענה הזאת.

- **`firstValueFrom` מ-RxJS. מתודות `HttpClient`**

מחזירות `Observables`, לא `Promises`. `Observable` הוא זרם של ערכים לאורך זמן; `firstValueFrom` מחכה לראשון ומחזיר `Promise`. עבור HTTP של בקשה/תגובה, זה מה שאתה רוצה. פרק 11 פורק `Observables` כראוי.

- **`Omit<Task, 'id'>` ו-`Partial<Task>`. אלה הם**

טיפוסי `utility` של `TypeScript`: `Omit<T, 'K'>` הוא `T` בלי השדה `K`; `Partial<T>` הוא `T` עם כל שדה אופציונלי. שניהם נפוצים בחתימות מתודות של API.

חיווט HTTP לתוך `TaskStore`

עכשיו עדכן את `TaskStore` לשימוש ב-API. החנות עדיין חושפת את אותם סיגנלים ריאקטיביים; היא רק שולפת ומסנכרנת עם השרת.

ערוך את `src/app/task-store.ts`:

```

import { Injectable, signal, computed, inject } from '@angular/core';
import { Task } from './task';
import { TasksApi } from './tasks-api';

@Injectable({ providedIn: 'root' })
export class TaskStore {
  private api = inject(TasksApi);
  private readonly _tasks = signal<Task[]>([]);
  private readonly _loading = signal(false);
  private readonly _error = signal<string | null>(null);

  readonly tasks = this._tasks.asReadonly();
  readonly loading = this._loading.asReadonly();
  readonly error = this._error.asReadonly();
  readonly remaining = computed(() => this._tasks().filter(t => !t.done));

  async load(): Promise<void> {
    this._loading.set(true);
    this._error.set(null);
    try {
      const tasks = await this.api.list();
      this._tasks.set(tasks);
    } catch (err) {
      this._error.set('Could not load tasks. Try again in a moment.');
```

```

    } finally {
      this._loading.set(false);
    }
  }

  async add(title: string): Promise<void> {
    const draft: Omit<Task, 'id'> = {
      title,
      done: false,
      createdAt: new Date().toISOString(),
      dueDate: null,
      tags: [],
    };
  }
};
```

```

// מיד הוספה אופטימית: שים שורה זמנית ב
const tempId = `temp-${crypto.randomUUID()}`;
const optimistic: Task = { ...draft, id: tempId };
this._tasks.update(current => [...current, optimistic]);
try {
  const saved = await this.api.create(draft);
  this._tasks.update(current =>
    current.map(t => (t.id === tempId ? saved : t))
  );
} catch {
  this._tasks.update(current => current.filter(t => t.id !== tempId));
  this._error.set('Could not save the new task.');
```

```

}

async toggle(id: string): Promise<void> {
  const current = this._tasks().find(t => t.id === id);
  if (!current) return;
  const next: Task = { ...current, done: !current.done };
  this._tasks.update(all => all.map(t => (t.id === id ? next : t)));
  try {
    await this.api.update(id, { done: next.done });
  } catch {
    // חזרה במקרה של כישלון
    this._tasks.update(all => all.map(t => (t.id === id ? current : t)));
    this._error.set('Could not save that change.');
```

```

  }
}

async remove(id: string): Promise<void> {
  const previous = this._tasks();
  this._tasks.update(all => all.filter(t => t.id !== id));
  try {
    await this.api.remove(id);
  } catch {
    this._tasks.set(previous);
    this._error.set('Could not delete the task.');
```

```
}  
}  
}
```

החנות עכשיו:

- **טוענת אסינכרונית.** ב- `load()`, היא שולפת מהשרת, מעדכנת את הסיגנלים שלה, וחושפת סיגנלי `loading` ו- `error` ל-UI.
- **מחילה שינויים בצורה אופטימית.** `add`, `toggle`, ו- `remove` מעדכנים את ה-UI לפני שהשרת מגיב; אם השרת נכשל, החנות חוזרת אחורה. זה מה שגורם לאפליקציות ווב להרגיש `snappy`.
- **חושפת רק סיגנלים ל-UI.** ה-UI אף פעם לא מחכה למשהו; הוא קורא מצב ריאקטיבי.

קריאה ל- `load()` בהפעלה

ל- `TaskStore` יש מתודת `load()`, אבל שום דבר לא קורא לה. הוסף קריאה מהקונסטרקטור של `TaskList`:

```
export class TaskList {
  private store = inject(TaskStore);
  tasks = this.store.tasks;
  remaining = this.store.remaining;
  loading = this.store.loading;
  error = this.store.error;

  constructor() {
    this.store.load();
  }

  onAdd(title: string): void { this.store.add(title); }
  onToggle(id: string): void { this.store.toggle(id); }
}
```

עדכן את התבנית להראות מצב טעינה ושגיאה:

```
<h1>Tasks – {{ remaining() }} left</h1>

@if (loading()) {
  <p class="status">Loading...</p>
}
@if (error()) {
  <p class="status error">{{ error() }}</p>
}

<app-add-task-form (add)="onAdd($event)" />

<ul class="task-list">
  @for (task of tasks(); track task.id) {
    <app-task-row [task]="task" (toggled)="onToggle($event)" />
  } @empty {
    <li class="empty">Nothing to do. Nice.</li>
  }
</ul>
```

שמור. רענן. אתה אמור לראות הבזק קצר של "Loading..." ואז את שתי המשימות שלך מ-db.json. הוסף משימה; פתח את db.json בעורך שלך כדי לאשר שהקובץ קיבל רשומה שלישית. העבר משימה; השינוי אמור להתמיד בין ריעננים.

Interceptors: התנהגות בקשה חוצת-רוחב

נניח שכל בקשה צריכה כותרת Authorization, או שכל תגובה צריכה להיות מתועדת, או שכל 401 צריך להפנות לעמוד התחברות. אתה לא רוצה לשים את הקוד הזה בכל מתודה של TasksApi. התשובה של Angular היא interceptors.

Interceptor הוא פונקציה שרצה על כל בקשת HTTP, יכולה לשנות את הבקשה, ויכולה לשנות את התגובה.

```
// src/app/auth-interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('compass:token');
  if (token) {
    req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  }
  return next(req);
};
```

רשום אותו ב-app.config.ts:

```
import { provideHttpClient, withFetch, withInterceptors } from '@angular  
import { authInterceptor } from './auth-interceptor';  
  
// ...providers:  
provideHttpClient(withFetch(), withInterceptors([authInterceptor])),
```

עכשיו כל בקשה שנשלחת על ידי `HttpClient` תופסת את הכותרת, בלי ש-`TasksApi` תצטרך לחשוב על זה.

`Interceptors` גם הופכים את טיפול השגיאות אחיד. `Interceptor` שמנתק מ-session ב-401:

```
export const authGuardInterceptor: HttpInterceptorFn = (req, next) =>  
  return next(req).pipe(  
    catchError(err => {  
      if (err.status === 401) {  
        localStorage.removeItem('compass:token');  
        window.location.href = '/login';  
      }  
      return throwError(() => err);  
    })  
  );  
};
```

נשתמש ב-`interceptors` יותר בחלק חמישי; לעת עתה, דע שהם קיימים והם הפטיש הנכון ל"כל בקשה צריכה X".

`httpResource` : ה-`fetch` המשולב-סיגנל המודרני

הדפוס של `firstValueFrom(this.http.get(...))` בסדר, אבל Angular מודרנית מציעה משהו יותר נחמד למקרה הנפוץ של "שלוף

והחזק את הנתונים האלה". `httpResource<T>` יוצר משאב HTTP מגובה סיגנל שעוקב אחר מצבי טעינה, שגיאה, וערך בשבילך.

```
import { httpResource } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class TaskStore {
  private baseUrl = 'http://localhost:3000';

  taskList = httpResource<Task[]>(() => `${this.baseUrl}/tasks`);

  // בקומפונטה: taskList.value(), taskList.isLoading(), taskList.error
}
```

`httpResource` מחזיר משאב עם `.value()`, `.isLoading()`, `.error()`, `.reload()`, וכמה אחרים. ה-URL של הבקשה שלו הוא פונקציה; כשכל קריאת סיגנל בתוך הפונקציה משתנה, הוא שולף מחדש. אז `httpResource(() => tasks?tag=${tag()}/${baseUrl}$=)` שולף מחדש בכל פעם ש-`tag` משתנה.

ל-Compass נשמור על ה-`TaskStore` שנעשה ידנית כי הוא גם מטפל במוטציות, אבל לנתונים לקריאה בלבד (סטטיסטיקות, דשבורדים, look-ups), `httpResource` חוסך הרבה קוד. פרק 14 חוזר אליו כשנוסיף את עמוד הסטטיסטיקות.

שגיאות, timeouts, ניסיונות חוזרים

כל קריאת HTTP יכולה להיכשל. `HttpClient` מדווח על כישלונות כשגיאות על ה-`Observable`, שהופכות לדחיות על ה-`Promise`, שהופכות

לחריגות שנזרקות ב- `await`. בחנות למעלה תפסנו את אלה והחזרנו את העדכון האופטימי.

לכישלונות חולפים, לוגיקת ניסיון חוזר יכולה לעזור. ל-RxJS יש אופרטור `retry`:

```
import { retry } from 'rxjs';

list(): Promise<Task[]> {
  return firstValueFrom(
    this.http.get<Task[]>(`${BASE_URL}/tasks`).pipe(retry({ count: 2, delay: 1000 }));
  );
}
```

זה מנסה מחדש את הבקשה עד פעמיים, עם עיכוב של חצי שנייה בין ניסיונות. פרק 11 מכסה אופרטורי RxJS כמו זה כראוי.

ל-`HttpContext`, `timeouts`, ו-`interceptors` יכולים להגביל משך בקשה, או שאתה יכול לעטוף את הבקשה ב-`Promise.race` עם טיימר. ב-Compass לא נוסף `timeouts` ידנית; אם בקשה נתקעת, הדפדפן של המשתמש יגלה את הכישלון בסופו של דבר, והעדכונים האופטימיים שלנו שומרים את ה-UI מגיב.

מטמון ונתונים מיושנים

`HttpClient` לא שומר במטמון. כל קריאה הולכת לרשת אלא אם שכבת דפדפן או `Service Worker` תופסת אותה. לרוב נתוני היישום, זה מה שאתה רוצה — אתה לא רוצה נתונים מיושנים שנשמרו במטמון בתוך ה-JavaScript שלך.

`httpResource` שולף מחדש כשסיגנל הבקשה שלו משתנה; הוא גם לא שומר במטמון בין עמודים. אם אתה צריך מטמון (endpoint של הגדרות שאף פעם לא משתנה במהלך session, נניח), פרק 14 מראה דפוס שמשלב `httpResource` עם מטמון מבוסס סיגנל.

מה בא הלאה

פרק 11 מכסה RxJS – הספרייה מתחת ל-`HttpClient` והדרך ש-Angular מייצגת זרמים שנפרסים לאורך זמן. אז פרק 12 מחליף את ה-`Add task` "input" שלנו בטפסים ריאקטיביים ראויים, שלמים עם ולידציה ובטיחות טיפוסים.

תרגילים

1. הוסף כפתור `"delete"` ל-`TaskRow` שפולט אירוע `deleted`. חבר את `TaskList` לקרוא ל-`store.remove(id)`. וודא שהמחיקה מתמידה בין ריעננים.

2. הפעל כישלון: עצור את `json-server`, ואז לחץ על משימה. מה ה-UI עושה? האם העדכון האופטימי מוחזר? האם הודעת השגיאה מוצגת? ברגע שווידאת, הפעל מחדש את `json-server` ולחץ שוב – הכל אמור להתאושש.

3. שכתב את `TasksApi.list()` לשימוש ב-`httpResource` במקום `http.get`. רגיל. איזו חתימה יש למתודה עכשיו? מה `TaskStore` צריך לשנות כדי לצרוך אותה? (אתה יכול להשאיר את `TaskStore` משתמש

בגרסת ה-promise אם אתה מעדיף; הפואנטה של התרגיל היא לראות את ההבדל.

פרק 11. הבסיס של RxJS

ראית את RxJS פעמיים כבר: `HttpClient` החזיר `Observables`, וגלגלנו אותם עם `firstValueFrom`. זה מספיק עבור HTTP של בקשה-תגובה, אבל לא לחלקים של `Compass` שעדיין לפניכם. חיפוש תוך כדי הקלדה, זרמים חיים, עדכוני טפסים מתואמים, ופרמטרי מסלול – כולם הם באופן טבעי זרמים של ערכים לאורך זמן, ו-RxJS היא הכלי של Angular לזרמים.

הפרק הזה הוא לא סיור מלא ב-RxJS. לספרייה הזאת יש מאה אופרטורים ותת-תרבות משלה. מה שאתה צריך ל-`Compass`, ולרוב קוד יישום Angular, הוא תת-קבוצה ספציפית: מהו `Observable`, איך הוא שונה מ-Promise ומסיגנל, תריסר אופרטורים, ונקודות ה-`interop` בין `Observables` לסיגנלים. זה מה שהפרק הזה מכסה.

מהו Observable

`Observable` הוא יצרן של ערכים לאורך זמן. הוא עשוי לייצר אפס ערכים (ואז להשלים), ערך אחד (כמו תגובת HTTP), רצף קבוע (כמו הספרות של פאי), או זרם ללא גבול (כמו אירועי עכבר עד שהחלון נסגר). הוא עשוי להשלים בצורה נורמלית או לצאת מכלל שגיאה. והוא לא עושה כלום עד שמשיהו נרשם אליו – `Observables` הם עצלים.

השווה עם שני הדברים שאתה כבר מכיר:

- **Promise** מייצר ערך אחד לכל היותר. ברגע שהוא נוצר, Promise מתחיל לעבוד מיד (הוא להוט) ובסופו של דבר נפתר או נדחה. אי אפשר לבטל אותו ואי אפשר לספק יותר מערך אחד.
 - **סיגנל** מחזיק ערך נוכחי בכל עת. קריאה שלו תמיד מחזירה את הערך. כתיבה שלו מודיעה לכל מי שעוקב אחריו.
- Observable הוא אף אחד מהם. הוא תוכנית לייצור ערכים, מופעלת על ידי הרשמה, יכולה לייצר רבים, וניתנת לביטול.
- באופן קונקרטי:

```
import { Observable } from 'rxjs';

const numbers = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  subscriber.complete();
});

numbers.subscribe({
  next: n => console.log('got', n),
  complete: () => console.log('done'),
});
// מדפיס: got 1, got 2, got 3, done
```

כמעט אף פעם לא תבנה Observables עם `new Observable`. בקוד יישום, הם מגיעים מפונקציות יצירה ומ-APIs של Angular.

יצירת Observables

הפונקציות שהופכות משהו ל-Observable:

```
import { of, from, fromEvent, interval, timer } from 'rxjs';

const three = of(1, 2, 3); // פולט 1, 2, 3 ומשלים
const fromArray = from([1, 2, 3]); // פולט אלמנטים של מערך
const fromPromise = from(fetch('/api/tasks')); // Promise עוטף

const clicks = fromEvent<MouseEvent>(document, 'click'); // על כל לחיצה
const ticks = interval(1000); // פולט 0, 1, 2, ... כל שנייה
const delayed = timer(500); // ומשלים מפולט 0 אחרי 500
```

שניים מאלה – `fromEvent` ו-`interval` – מייצרים זרמים ללא גבול. הירשם אליהם, והם פולטים לנצח, או עד שתבטל את ההרשמה.

Angular נותנת לך Observables לדברים מובנים רבים:

```
this.http.get<Task[]>('/api/tasks'); // פליטה אחת, אז השלמה
this.route.params; // של מסלול param פולט על כל שינוי
this.route.queryParams; // של שאילתה param פולט על כל שינוי
formControl.valueChanges; // פולט על כל שינוי של בקרת טופס
```

כל אחד מאלה הוא זרם מוגדר היטב, וכל אחד מזדווג באופן טבעי עם pipeline קטן של אופרטורים כדי לייצר את הערך שאתה באמת רוצה.

אופרטורים ו-pipe

אופרטור

הוא פונקציה שמקבלת Observable ומחזירה Observable חדש. pipe מרכיב אופרטורים לשרשרת.

```
import { of, map, filter } from 'rxjs';

of(1, 2, 3, 4, 5)
  .pipe(
    filter(n => n % 2 === 0),
    map(n => n * 10)
  )
  .subscribe(n => console.log(n)); // 20, 40
```

יש עשרות אופרטורים. תשתמש אולי בחמישה-עשר מהם באופן קבוע. הנה הרשימה המקוצרת, בערך בסדר של כמה פעמים תפנה אליהם.

map(fn) מהפך כל פליטה דרך fn. אם fn מחזיר Promise או Observable, זה לא ממתין – לזה, השתמש ב-switchMap וחבריו.

filter(fn) מפיל פליטות ש-fn מחזיר עליהן false.

tap(fn) מריץ את fn על כל פליטה עבור תופעות הלוואי שלו, בלי לשנות את הזרם. שימושי לרישום ולדיבוג. גם tap({ error }) גם רץ על שגיאות.

take(n) לוקח את n הפליטות הראשונות, אז משלים.

take(1) נפוץ כשאתה רוצה ש-Observable יתנהג כמו Promise.

takeUntil(otherObservable\$) משלים את זרם המקור כש-

otherObservable\$ פולט. ל-Angular יש

`takeUntilDestroyed()` מיוחד שקושר את חיי ה-Observable לחיי הקומפוננטה – נשתמש בו למטה.

`debounceTime(ms)` מחכה `ms` אחרי הפליטה האחרונה לפני שהוא מעביר אותה. מושלם לחיפוש תוך כדי הקלדה: המשתמש מפסיק, הבקשה יוצאת.

`distinctUntilChanged()` מדכא ערכים כפולים עוקבים. גם מאוד שימושי לחיפוש: אל תשלוף מחדש אם השאילתה לא באמת השתנתה.

`switchMap(fn)` משטח את המקור דרך Observable פנימי, מבטל כל Observable פנימי שנמצא בטיסה כשחדש מגיע. זה האופרטור שגורם לחיפוש תוך כדי הקלדה לעבוד: כל הקלדה מפעילה בקשת HTTP חדשה, והבקשה הקודמת מבוטלת.

`mergeMap(fn)` משטח, אבל מאפשר לכל Observables הפנימיים לרוץ במקביל. לעתים רחוקות מה שאתה רוצה עבור עבודה מונעת משתמש; שימושי כשאתה רוצה עיבוד מקביל.

`exhaustMap(fn)` משטח, אבל מתעלם מפליטות מקור חדשות בזמן ש-Observable פנימי עדיין רץ. האופרטור הנכון לכפתורי "save": אם המשתמש לוחץ פעמיים, אל תירה שתי שמירות.

`concatMap(fn)` משטח, מסדר Observables פנימיים בתור כך שהם רצים אחד אחרי השני. שימושי לכתיבות מסודרות.

`catchError(fn)` מיירט שגיאות ואו מחזיר Observable חלופי או זורק שוב. כך אתה מספק ערך ברירת מחדל על קריאת HTTP שנכשלה.

נרשם מחדש על שגיאה. השתמשנו `retry({ count, delay })` בו בפרק 10.

`combineLatest([a$, b$])` לוקח מספר Observables ופולט מערך של הערכים האחרונים שלהם בכל פעם שאחד מהם פולט. שילוב של route params ו-query params של Angular הוא שימוש נפוץ.

`startWith(value)` מוסיף ערך לתחילת הזרם, כך שמנויים מקבלים פליטה מיידית.

כלל הלמידה: אל תנסה לשנן את כולם. למד `map`, `filter`, `tap`, `take`. `debounceTime`, `distinctUntilChanged` ו-`switchMap`. חפש את השאר כשבעיה מזמינה אותם.

דוגמה עובדת: חיפוש תוך כדי הקלדה

נניח ש-Compass מקבל תיבת חיפוש. המשתמש מקליד "buy m", ו-Compass שולף `/tasks?q=buy%20m`. אם המשתמש עדיין מקליד כשתגובה מגיעה, התגובה צריכה להיזרק – השאילתה מיושנת.

```
import { fromEvent } from 'rxjs';
import { debounceTime, distinctUntilChanged, filter, map, switchMap } from 'rxjs';

const input = document.querySelector('input');

fromEvent<Event>(input, 'input').pipe(
  map(e => (e.target as HTMLInputElement).value.trim()),
  debounceTime(200),
  distinctUntilChanged(),
  filter(q => q.length >= 2),
  switchMap(q => http.get<Task[]>(`/api/tasks?q=${encodeURIComponent(q)}`)),
).subscribe(results => {
  render(results);
});
```

קרא את זה מלמעלה למטה:

1. כל אירוע input, קבל את הערך המנוקה.
2. חכה 200ms אחרי ההקלדה האחרונה.
3. המשך רק אם השאילתה השתנתה.
4. המשך רק אם יש לפחות שני תווים.
5. ירה את בקשת ה-HTTP. אם הקלדה חדשה מגיעה בזמן שהבקשה הזאת בטיסה, בטל אותה והתחל חדשה.

ה-pipeline של שש השורות הזה מחליף את מה שאחרת היה טיימר מנוהל ידנית, מעקב אחר בקשה בטיסה, ref של debounce, והשוואה לשינוי. זה מה ש-RxJS נועד לו.

סיגנלים ו-Interop: Observables

רוב מצב Compass חי בסיגנלים. רוב ה-APIs המובנים של Angular פולטים Observables. ה-interop הוא שתי פונקציות:

הופך Observable לסיגנל. `toSignal(observable, options)`

```
import { toSignal } from '@angular/core/rxjs-interop';

class TaskList {
  private route = inject(ActivatedRoute);
  filter = toSignal(this.route.queryParamMap.pipe(
    map(params => params.get('tag') ?? 'all')
  ), { initialValue: 'all' });
}
```

כשיו `filter` הוא סיגנל שאתה יכול לקרוא בתבנית כ- `filter()`, והוא מתעדכן בכל פעם ש-`query params` של המסלול משתנים. `toSignal` גם מנקה את ההרשמה כשהקומפוננטה מושמדת.

הופך סיגנל ל-Observable. `toObservable(signal)`

```
import { toObservable } from '@angular/core/rxjs-interop';

class TaskStore {
  filter = signal('all');
  filter$ = toObservable(this.filter);

  filteredTasks$ = this.filter$.pipe(
    switchMap(f => this.api.list(f))
  );
}
```

`toObservable` פולט בכל פעם שהסיגנל משתנה, והפליטה היא הערך החדש של הסיגנל. בשילוב עם אופרטורים, זה מאפשר לך לבנות pipelines של "שליפה מונעת סיגנל" בלי לעזוב את המרחב הריאקטיבי.

הרשמה בטוחה

כל הרשמה היא משאב. אם הקומפוננטה שלך נרשמת ואף פעם לא מבטלת הרשמה, ה-callback ממשיך לרוץ אחרי שהקומפוננטה עוזבת את המסך – דליפה. Angular נותנת לך שלוש אפשרויות טובות ואחת רעה.

הטוב ביותר (בדרך כלל): הפייפ `async`. קשור Observable ישירות בתבנית, ו-Angular נרשמת ומבטלת הרשמה בשבילך.

```
<p>{{ user$ | async }}</p>
```

השני הכי טוב: `toSignal`. לצרכנים של Observable שאינם בתבנית. Angular קורעת את ההרשמה בהשמדת הקומפוננטה.

שלישי: `takeUntilDestroyed()`. להרשמות מפורשות בהקשר קומפוננטה.

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

class TaskList {
  constructor() {
    this.route.params.pipe(takeUntilDestroyed()).subscribe(params => {
      // ...
    });
  }
}
```

`takeUntilDestroyed()` קורא את ה-`DestroyRef` הנוכחי (מ-DI) ומשלים את ה-`Observable` כשהקומפוננטה מושמדת. לא נדרש `ngOnDestroy` מפורש.

רע: `.subscribe()` חשוף בלי איזה קריעה שהיא. זו הדליפה. אם ה-linter שלך הגון, הוא יצעק עליך.

Cold לעומת hot, בקצרה

`Observable` הוא `cold` אם כל מנוי מקבל ריצה משלו של היצרן, ו-`hot` אם כל המנויים חולקים ריצה אחת.

`http.get()` הוא `cold`: כל הרשמה יורה בקשה חדשה. `fromEvent(document, 'click')` הוא `hot`: כל המנויים מקבלים את אותם אירועי לחיצה.

לעתים רחוקות תצטרך לחשוב על זה במפורש. איפה שזה חשוב: אם אתה `subscribe` לתוצאת `http.get` פעמיים, אתה יורה שתי בקשות. השתמש ב-`shareReplay(1)` כדי לחלוק בקשה אחת בין מנויים רבים, או המר לסינגל.

מתי לא להשתמש ב-RxJS

RxJS חזק. הוא גם מנוצל יתר על המידה. סימנים שהוגזמת:

- לקומפוננטות שלך יש חמש קריאות `pipe` מקוננות כדי לחשב משהו שסיגנל `computed` היה מבטא בשורה אחת.
- יש לך שדה `Subject` על שירות שנמצא בשימוש כמכל עם מצב. השתמש בסיגנל.
- אתה `.subscribe()` למשהו ומגדיר שדה עם התוצאה. השתמש ב- `.toSignal`.
- אתה מבצע `debounce` על סיגנל דרך `toObservable` ו- `debounceTime`. בסדר – אבל בדוק אם `linkedSignal` או `effect` קטן אינם ברורים יותר למקרה שלך.

כלל האצבע מפרק 7 עומד: סיגנלים למצב, RxJS לזרמים. כשאתה מוצא את עצמך משתמש ב-RxJS למצב, הגר; כשאתה מוצא את עצמך משתמש בסיגנלים לזרמים, שאל למה.

מה בא הלאה

פרק 12 מסיים את חלק שלישי עם טפסים ריאקטיביים. Compass יש לו כרגע "Add task" input שנעשה ידנית. בפרק 12 אנחנו מחליפים אותו ב- `FormGroup`, מוסיפים ולידציה, ובונים דיאלוג עריכה למשימות עם תאריכי יעד ותגיות. אחרי פרק 12, Compass יש לו בקאנד אמיתי, טפסים אמיתיים, והתנהגות ריאקטיבית אמיתית – כל

מה שאפליקציה קטנה צריכה לפני שהיא גדלה למספר עמודים, שזו העבודה של חלק רביעי.

תרגילים

1. כתוב pipeline של Observable שלוקח לחיצות על כפתור "load more" וכל לחיצה מוסיפה את העמוד הבא של משימות. השתמש ב- `exhaustMap` כך שמספר לחיצות מהירות לא יורו מספר בקשות. וודא על ידי לחיצה מהירה.

2. המר את ה-pipeline של חיפוש-תוך-כדי-הקלדה בפרק הזה לקומפוננטת Compass. צור `SearchBox` עם `input`, חבר את ה-pipeline בקונסטרקטור שלו, ופלט `output` `results`. השתמש ב-`takeUntilDestroyed` על ההרשמה.

3. `HttpClient` מחזיר `Observables` קרים – כל `subscribe` יורה מחדש את הבקשה. הוכח זאת: הירשם לאותו `Observable` של `http.get(...)` פעמיים ובדוק את ה-`tab.network`. אחר כך הוסף `shareReplay(1)` ואשר שהוא יורה פעם אחת. למה `shareReplay(1)` ולא `shareReplay()`?

פרק 12. טפסים: תפיסת קלט משתמש באופן אמין

ל-Compass יש input אחד עכשיו: השדה "Add task". הוא עובד כי הוא טריוויאלי. יישומים אמיתיים יש להם טפסים עם שדות מרובים, כללי ולידציה, מגבלות חוצות-שדות, בדיקות אסינכרוניות, והתנהגות submit מסובכת. לנסות לעשות את כל זה ידנית עם קישורי (input) בדרך שעשינו בפרק 6 אפשרי; זה גם מפסיד יותר משעה של אחר צהריים לטופס משמה שצריך.

ל-Angular יש שתי מערכות טפסים: טפסים מונעי-תבנית וטפסים ריאקטיביים. הם לא שווי ערך. טפסים ריאקטיביים חזקים יותר, ניתנים לבדיקה יותר, ומוקלדים טוב יותר. הספר הזה משתמש בטפסים ריאקטיביים באופן בלעדי. טפסים מונעי-תבנית קיימים ויש להם מעריצים; אם אתה יורש קוד ישן יותר מלא ב- ngModel, דע שהוא עובד ושהסירור האקוסיסטמי של פרק 20 יכוון אותך למקורות לסגנון הישן יותר.

בפרק הזה אנחנו מחליפים את ה-Add task "input" של פרק 6 בטופס ריאקטיבי, ואז בונים דיאלוג עריכה למשימות שמאפשר למשתמש לשנות את הכותרת, להגדיר תאריך יעד, ולתייג אותה.

הכנה

טפסים ריאקטיביים באים מ-`@angular/forms`. רשום את ה-providers הנדרשים על ידי ייבוא הדירקטיבות של `ReactiveFormsModule` בכל קומפוננטה שמשתמשת בהן. כל קומפוננטה עצמאית שמארכת טופס מייבאת מה שהיא צריכה:

```
import { FormBuilder, FormControl, FormGroup, ReactiveFormsModule, Val.  
  
@Component({  
  imports: [ReactiveFormsModule],  
  // ...  
})
```

— `ReactiveFormsModule` הוא חבילה של דירקטיבות `formGroupName`, `formControl`, `formGroup`, וכן הלאה. תבניות שמתייחסות לדירקטיבות האלה חייבות לייבא את המודול.

`FormArray`, `FormGroup`, `FormControl`

שלושת הפרימיטיבים של טפסים ריאקטיביים.

`FormControl<T>` עוטף ערך יחיד. הוא יודע את הערך הנוכחי שלו, את מצב הולידציה שלו, אם המשתמש נגע בו או הוציא ממנו פוקוס, והוא פולט שינויים.

```
const title = new FormControl("", { nullable: true, validators: [Valid

title.value; // "
title.setValue('Buy milk');
title.valueChanges; // Observable<string>
title.valid; // boolean
title.errors; // { required: true } או null
```

יש טיפוס `title.value` ל-`title`, בלעדין, חשוב. `nullable: true` מגדירה את הבקרה בחזרה ל-`null`. עם `nullable: true`, הערך הוא תמיד `string`, ו-`reset` משחזר את הערך ההתחלתי (" כאן). השתמש בו כמעט על כל בקרה.

FormGroup היא אוסף של בקורות בשמות.

```
const form = new FormGroup({
  title: new FormControl("", { nullable: true, validators: [Validators.
  dueDate: new FormControl<string | null>(null),
  tag: new FormControl("", { nullable: true }),
});

form.value; // { title: string, dueDate: string | null, tag:
form.valid;
form.get('title')?.setValue('...');
```

הטופס מוקלד: ל-`form.value` יש טיפוס שנגזר מהצורה של הבקורות. זו התכונה של טפסים ריאקטיביים שהכי שינתה את המשחק ב-Angular 14 – לפני, ערכי טופס היו כולם `any`, ושגיאות הקלדה ב-`formControlName` של תבנית היו נשברות בשקט.

FormArray היא רשימה דינמית של בקורות באותה צורה. שימושי לשורות חוזרות: "הוסף עוד תגית", "הוסף עוד סופר". נשתמש בה מאוחר יותר בפרק הזה עבור תגיות.

FormBuilder הוא יותר נחמד

בניית טפסים עם `new FormControl(...)` היא מפורטת. **FormBuilder** הוא עוזר קטן שירות מוזרק דרך `DI`:

```
private fb = inject(FormBuilder);

form = this.fb.nonNullable.group({
  title: ['', Validators.required],
  dueDate: [null as string | null],
  tag: [''],
});
```

`fb.nonNullable.group(...)` הופך כל בקרה ל-`non-nullable` כברירת מחדל. `fb.group(...)` (בלי `nonNullable`) משתמש בברירות המחדל הישנות שניתן לנו. העדף את `fb.nonNullable`. כל רשומה היא `[initialValue, validators]`. יכולים להיות פונקציה יחידה או מערך.

בנייה מחדש של "Add task" עם טפסים ריאקטיביים

פתח את `src/app/add-task-form/add-task-form.ts`. החלף אותו ב:

```
import { Component, inject, output } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';

@Component({
  selector: 'app-add-task-form',
  imports: [ReactiveFormsModule],
  templateUrl: './add-task-form.html',
  styleUrls: ['./add-task-form.css'],
})
export class AddTaskForm {
  private fb = inject(FormBuilder);
  add = output<string>();

  form = this.fb.nonNullable.group({
    title: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(50)]]
  });

  submit(): void {
    if (this.form.invalid) return;
    this.add.emit(this.form.getRawValue().title.trim());
    this.form.reset();
  }
}
```

תבנית:

```
<form [formGroup]="form" (ngSubmit)="submit()">
  <input
    type="text"
    formControlName="title"
    placeholder="What needs doing?" />
  <button type="submit" [disabled]="form.invalid">Add</button>
</form>

@if (form.controls.title.touched && form.controls.title.invalid)
  <p class="error">
    @if (form.controls.title.hasError('required')) { Title is requ
    @if (form.controls.title.hasError('minlength')) { At least two
    @if (form.controls.title.hasError('maxlength')) { Please keep
  </p>
}
```

שמור. זה עובד בדיוק כמו קודם, אבל:

• **צורת הטופס נאכפת על ידי TypeScript.** אם תקליד לא נכון

`formControlName="titel"`, הקומפיילר של התבנית

אומר לך.

• **הולידציה מרוכזת.** אין יותר חישוב `disabled` בתבנית –

היא קוראת `form.invalid`.

• **לשגיאות יש שמות.** הודעות שונות למגבלות שונות, בלי

לטפל במקרים מיוחדים של אורכים בתבנית.

• **`reset()` היא קריאה אחת.** אין יותר ניקוי ידני של

סיגנל.

`ngSubmit` (בלי סימן דולר לפניו) היא דירקטיבה שתופסת הגשות

טפסים וקוראת ל- `submit()`, והיא מונעת את הטעינה מחדש ברירת

המחדל של הדפדפן. אתה לא צריך יותר
`$.sevent.preventDefault()`

Validators מובנים

האובייקט `Validators` מייצא את המקרים הנפוצים:

```
Validators.required
Validators.requiredTrue // עבור תיבות סימון
Validators.min(0)
Validators.max(100)
Validators.minLength(3)
Validators.maxLength(200)
Validators.pattern(/^[a-z0-9-]+$ /i)
Validators.email
```

כל דבר אחר, כתוב `validator` ידנית. `Validator` הוא פונקציה
`.(control) => errors | null`

```
function noWhitespace(control: AbstractControl): ValidationErrors | null {
  const v = (control.value ?? '').trim();
  return v.length === 0 ? { whitespace: true } : null;
}

new FormControl('', [Validators.required, noWhitespace]);
```

המפתחות של אובייקט השגיאה הופכים לרשומות ב-
`control.errors`, שהתבנית יכולה לבדוק עם
`.hasError('whitespace')`

דיאלוג עריכה: FormGroup ו- FormArray יחד

Compass יצטרך לאפשר למשתמש לערוך משימה: לשנות את הכותרת, להגדיר או לנקות את תאריך היעד, ולנהל את רשימת התגיות. זה טופס אמיתי: FormGroup עם FormArray מקונן.

צור אותו:

```
ng generate component edit-task
```

```
: edit-task.ts
```

```
import { Component, inject, input, output } from '@angular/core';
import {
  FormArray, FormBuilder, FormControl, FormGroup,
  ReactiveFormsModule, Validators
} from '@angular/forms';
import { Task } from '../task';

@Component({
  selector: 'app-edit-task',
  imports: [ReactiveFormsModule],
  templateUrl: './edit-task.html',
  styleUrls: ['./edit-task.css'],
})
export class EditTask {
  task = input.required<Task>();
  saved = output<Partial<Task>>>();
  cancelled = output<void>();

  private fb = inject(FormBuilder);

  form = this.fb.nonNullable.group({
    title: ['', [Validators.required, Validators.minLength(2)]],
    dueDate: [null as string | null],
    tags: this.fb.array<FormControl<string>>([]),
  });

  constructor() {
    // מלא את הטופס כשמשוימה משתנה
    this.setFromTask();
  }

  private setFromTask(): void {
    const t = this.task();
    this.form.patchValue({
      title: t.title,
      dueDate: t.dueDate,
    });
  }
}
```

```
const tagsArray = this.form.controls.tags;
tagsArray.clear();
for (const tag of t.tags) {
  tagsArray.push(this.fb.nonNullable.control(tag, Validators.required));
}

addTag(): void {
  this.form.controls.tags.push(
    this.fb.nonNullable.control("", Validators.required)
  );
}

removeTag(index: number): void {
  this.form.controls.tags.removeAt(index);
}

submit(): void {
  if (this.form.invalid) return;
  const raw = this.form.getRawValue();
  this.saved.emit({
    title: raw.title.trim(),
    dueDate: raw.dueDate,
    tags: raw.tags,
  });
}

cancel(): void {
  this.cancelled.emit();
}
}
```

ותבנית שקושרת `FormArray`:

```
<form [formGroup]="form" (ngSubmit)="submit()" class="edit-task">
  <label>
    Title
    <input type="text" formControlName="title" />
  </label>

  <label>
    Due date
    <input type="date" formControlName="dueDate" />
  </label>

  <fieldset>
    <legend>Tags</legend>
    <div formArrayName="tags" class="tags">
      @for (control of form.controls.tags.controls; track $index;
      <div class="tag-row">
        <input type="text" [formControlName]="i" />
        <button type="button" (click)="removeTag(i)">Remove</button>
      </div>
    }
  </div>
  <button type="button" (click)="addTag()">Add tag</button>
</fieldset>

<div class="actions">
  <button type="button" (click)="cancel()">Cancel</button>
  <button type="submit" [disabled]="form.invalid">Save</button>
</div>
</form>
```

שתי דירקטיבות חדשות:

- `formArrayName="tags"` קושר container ל- `FormArray`.

• בתוכו, `formControlName="0"`, `"1"`, וכן הלאה, קושרים כל ילד לחרוץ שלו. השימוש ב-
`[formControlName]="i"` עם משתנה תבנית עושה זאת בצורה נקייה.

הטופס עכשיו תומך בהוספה והסרה של תגיות מהזבוב, כשכל תגית מאומתת בנפרד. הטופס המוקלד נותן לנו
`form.controls.tags.controls` כ- `FormControl<string>[]`,
ו-`TypeScript` יודעת ש-`raw.tags` היא `string[]`.

Validators אסינכרוניים, בקצרה

מדי פעם ולידציה צריכה להגיע לשרת: "האם שם המשתמש הזה זמין?"
`Validators` אסינכרוניים הם פונקציות שמחזירות `Observable` או `Promise` של `ValidationErrors | null`.

```
function uniqueTitle(api: TasksApi): AsyncValidatorFn {
  return control => {
    if (!control.value) return of(null);
    return timer(300).pipe(
      switchMap(() => api.check(control.value)),
      map(taken => (taken ? { unique: true } : null))
    );
  };
}

// בטופס:
title: ['', {
  validators: [Validators.required],
  asyncValidators: [uniqueTitle(this.api)],
}],
```

ה- `timer(300)` הוא `debounce` ידני כדי שלא נציף את השרת בכל הקלדה. `form.pending` הוא `true` בזמן שה-`validator` האסינכרוני רץ – קשור אותו בתבנית כדי להראות `.spinner`.

לא נוסף ולידציה של כותרת ייחודית ל-`Compass` – משימות עם שמות כפולים בסדר – אבל אתה עכשיו יודע לאן לפנות.

`valueChanges` ותגובה לקלט טופס

לכל בקרה וקבוצה יש `valueChanges` `Observable`:

```
this.form.controls.title.valueChanges
  .pipe(takeUntilDestroyed())
  .subscribe(v => console.log('title now:', v));
```

או, מומר לסינגל:

```
titleValue = toSignal(this.form.controls.title.valueChanges, {
  initialValue: this.form.controls.title.value,
});
```

שניהם עובדים. פנה ל- `toSignal` כשאתה רוצה שהמצב החי של הטופס יניע נגזרים של `computed` או אפקטים.

למקרה הנפוץ של "כשהבקרה הזאת משתנה, הרץ את תופעת הלוואי הזאת", השתמש ב- `valueChanges` עם `takeUntilDestroyed` ואל תפנה לסגנלים אלא אם תופעת הלוואי במקרה שייכת ל-`computed`.

ולידציה חוצת שדות

לפעמים תוקף תלוי ביותר מבקרה אחת. תאריך יעד חייב להיות אחרי "תאריך התחלה"; אישור-סיסמה חייב להתאים לסיסמה. הוסף `validator` לקבוצה, לא לבקרות.

```
form = this.fb.nonNullable.group({
  password: ['', Validators.required],
  confirm: ['', Validators.required],
}, {
  validators: (group) => {
    const p = group.get('password')?.value;
    const c = group.get('confirm')?.value;
    return p === c ? null : { mismatch: true };
  },
});
```

קרא אותו בתבנית כ- `form.errors?.['mismatch']`.

מתי להשתמש בטפסים מונעי-תבנית

התשובה הקצרה: בטפסים זעירים שבהם טפסים ריאקטיביים הם טקסי יתר. `input` חיפוש יחיד, מתג יחיד, תיבת סימון חד-פעמית: `[(ngModel)]` בסדר. כל דבר עם שדות מרובים, ולידציה ביניהם, או מניפולציה תוכניתית צריך להיות ריאקטיבי.

הספר משתמש בטפסים ריאקטיביים לאורך כי הם מתרחבים מזעיר לענק בלי לשנות פרדיגמה. כשתירש קוד `Angular` ישן יותר ותראה `ngForm`, `ngModel`, `ngModelGroup`, תוכל לקרוא אותם; כנראה לא תרצה לכתוב פיצ'רים חדשים בסגנון הזה.

חיווט `EditTask` לתוך `TaskList`

לא נבנה את חוויית המודאל המלאה בפרק הזה – ניתוב לדף פירוט הוא העבודה של פרק 13. לעת עתה, הוסף מצב "עריכה" זמני ל-`TaskList` שמראה את הטופס מתחת לשורה נבחרת.

```
export class TaskList {
  private store = inject(TaskStore);
  tasks = this.store.tasks;
  editingId = signal<string | null>(null);
  editing = computed(() => this.tasks().find(t => t.id === this.editingId));

  onEdit(id: string): void { this.editingId.set(id); }
  onCancel(): void { this.editingId.set(null); }
  onSave(changes: Partial<Task>): void {
    const id = this.editingId();
    if (id) this.store.applyChanges(id, changes);
    this.editingId.set(null);
  }
}
```

תצטרך להוסיף `applyChanges` ל-`TaskStore`. הוא קורא ל-`TasksApi.update` ומחיל את המשימה שהוחזרה על הסיגנל, עם חזרה אופטימית על כישלון – מאוד דומה ל-`toggle`).

תבנית:

```
@let e = editing();
@if (e) {
  <app-edit-task
    [task]="e"
    (saved)="onSaved($event)"
    (cancelled)="onCancel()" />
}
```

זה מספיק צנרת כדי להוכיח שהטופס עובד מקצה לקצה. פרק 13 הופך את זה למסלול עריכה ראוי.

מה בא הלאה

חלק רביעי. Compass היא עכשיו אפליקציה עובדת של מסך אחד: היא שולפת, מוסיפה, עורכת, ושומרת משימות מול בקאנד אמיתי. פרק 13 מוסיף ניתוב – מסך בית, עמוד סטטיסטיקות, תצוגת פירוט משימה עם טופס העריכה – ופרק 14 מקדם את TaskStore משירות יחיד לשכבת מצב מבוססת סיגנלים ראויה שחנויות אחרות יכולות לעקוב אחריה. פרק 15 מעצב מחדש UI לשימוש חוזר לדירקטיבות ולפייפים; פרק 16 גורם ל-Compass להיות מהיר בטלפון.

תרגילים

1. הוסף שדה notes ל-EditTask – `<textarea>` עם ולידציית `maxLength(1000)`. הצג את מספר התווים הנוותרים מתחת לשדה, מונע על ידי `.toSignal(form.controls.notes.valueChanges)`.

2. המר את הטופס "Add task" לכלול תאריך יעד אופציונלי. כשהמשתמש שולח, תאריך היעד צריך להישלח ל-`TaskStore.add` יחד עם הכותרת. עדכן את `TaskStore.add` לקבל ולשמור אותו.

3. חבר validator אסינכרוני לטופס "Add task" שמונע כותרות כפולות: אם מערך המשימות הנוכחי כבר יש בו משימה עם אותה כותרת מנוקה, סמן את הבקרה כלא תקינה עם שגיאת `{ duplicate: true }`. אתה לא צריך HTTP לזה – קרא את הסיגנל `tasks()` של החנות ישירות בתוך ה-validator באמצעות `computed`.